



南开大学
Nankai University



天津市智能机器人技术重点实验室
Tianjin Key Laboratory of Intelligent Robotics
南开大学机器人与信息自动化研究所
Institute of Robotics & Automatic Information System

第六章 移动机器人运动规划技术

Motion Planning Algorithms for Mobile Robot

孙雷 教授

Tel:13512967601

Email: sunl@nankai.edu.cn

2026年6月8日

本章内容



1. 移动机器人运动规划



2. 基于图搜索的静态路径规划



3. 基于图搜索的动态路径规划



4. 基于采样的路径规划算法



5. 面向避碰的局部路径规划算法



6. 路径平滑与轨迹规划算法

1. 运动规划(Motion Planning)

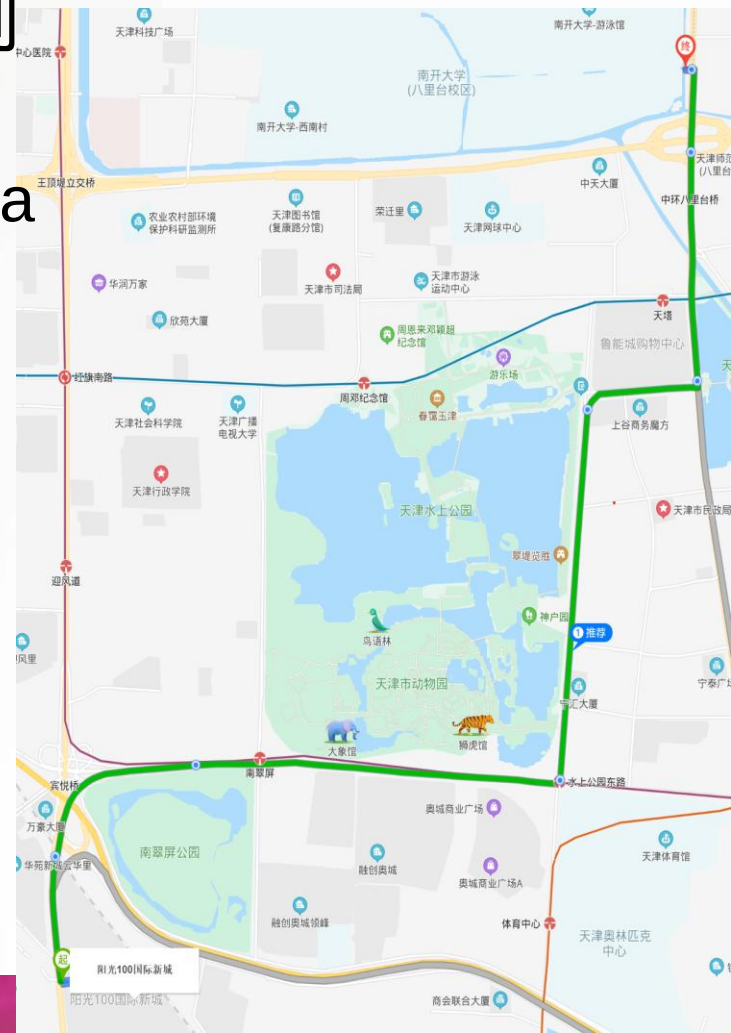
■ 何为运动规划:

- 运动规划是一个过程, 根据所给定的地图, 寻找从起始状态到目标状态的运动序列。运动规划常常需要在运动受到约束的条件下找到最优解。

——Wikipedia

■ 需求:

- 安全性: 规避障碍物
- 可达性: 从起点到终点的连通域
- 光滑性: 能量优化, 运行的平稳性, 舒适性
- 可执行性:
 - ✓ 机器人在动力学约束(Kinodynamic feasibility)下的运行可行性
 - ✓ 机器人执行单元的性能约束: 速度, 加速度, 加加速度(Jerk)



1.2 移动机器人运动规划方法

■ 分层规划思想:

□ 路径规划：只考虑工作空间（环境）的几何约束，不考虑机器人和执行器约束

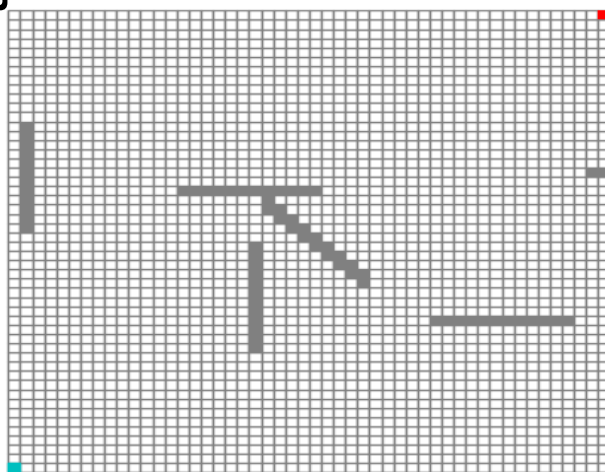
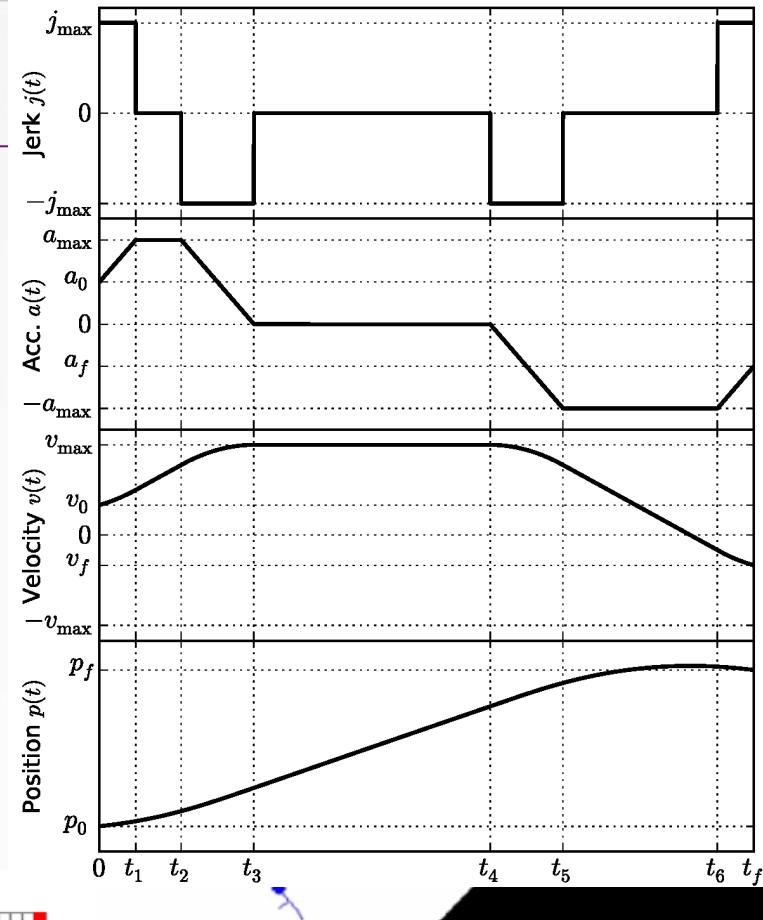
✓ 基于图搜索的路径规划算法

✓ 基于采样的路径规划算法

□ 轨迹规划：在给定路径上，考虑机器人和执行器约束生成随时间变化的运动序列

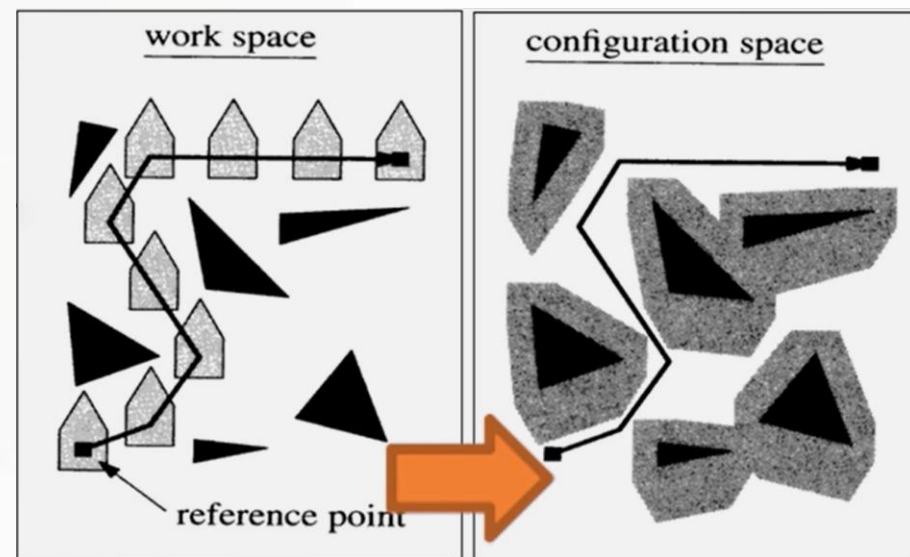
✓ 在线轨迹规划

✓ 离线轨迹规划



1.2 移动机器人路径规划方法

- 路径规划：在自由构型空间（Configuration Space, C-Space）中寻求一条从初始位置到目标位置的联通路径
 - C-Space：机器人简化为一个可移动的点，不考虑姿态和体积
 - ✓ 自由空间
 - ✓ 障碍物空间
 - 只考虑工作空间（环境）的几何约束，不考虑机器人和执行器约束
 - 完备性：当解存在时，能够在有限时间内找到解
 - 算法完备：基于图搜索方法
 - 概率完备：基于采样的方法



1.2 移动机器人路径规划方法

■ 图搜索的路径规划算法（算法完备）

- 如果在起始点和目标点间有路径解存在那么一定可以得到解，如果得不到解那么一定说明没有解存在。
- 完备的算法的优势在与它对于解的捕获能力是完全的，但是由此产生的缺点就是算法复杂度较大。

■ 基于采样的路径规划算法（概率完备）

- 这种算法一般是不直接在grid地图进行最小栅格分辨率的规划，它们采用在地图上随机撒一定密度的粒子来抽象实际地图辅助规划。
- 基于采样的算法速度较快，但是生成的路径代价（可理解为长度）较完备的算法高，而且会产生“有解求不出”的情况
- 这样的算法一般在高维度的规划问题中广泛运用

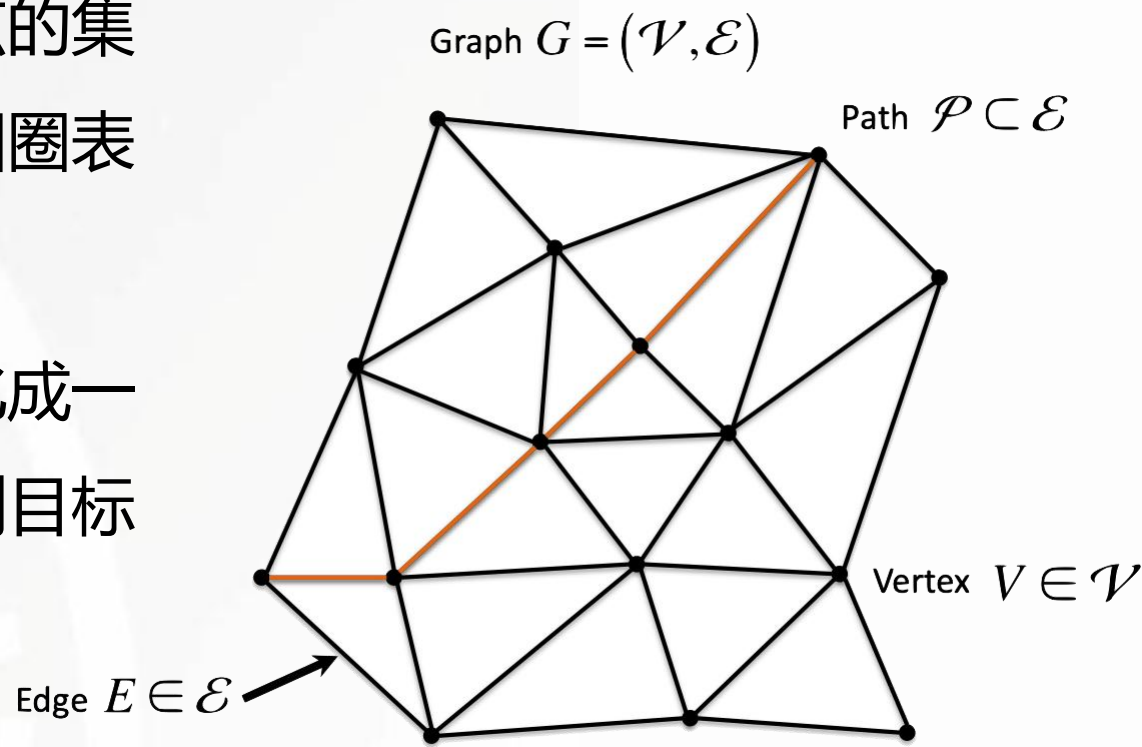
02

基于图搜索的静态路径规划

Graph Search-Based Static Path Planning

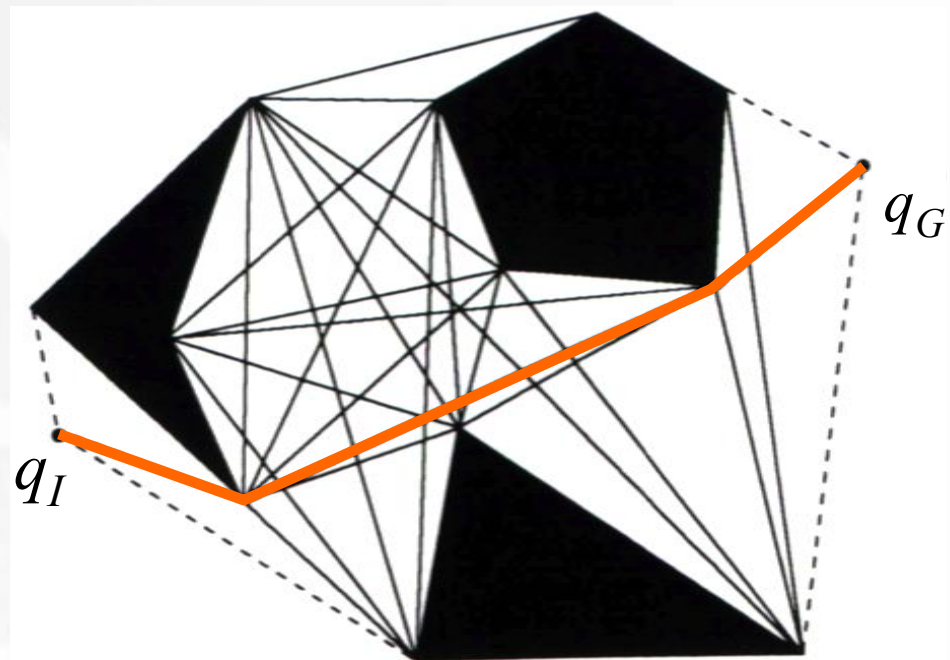
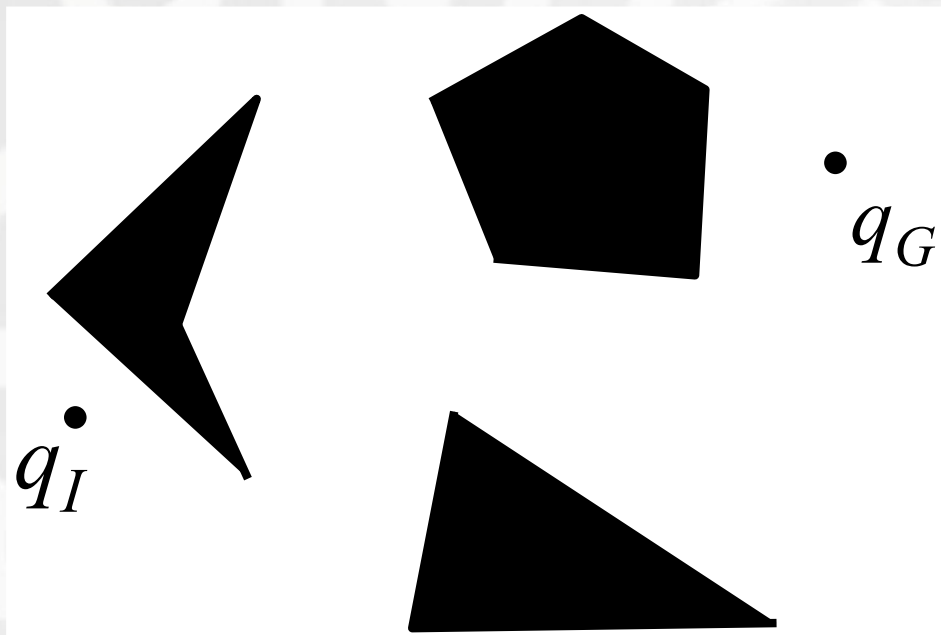
2.1 图搜索的基本概念

- 图：计算机科学中，一个图就是一系列顶点的集合，这些顶点通过一系列边连接。顶点用圆圈表示，边就是顶点间的连线。
- 图搜索：图搜索是一种将机器人的位姿简化成一个点，在图的空间内搜索一条从初始节点到目标节点的联通路径
 - 拓扑图的构建
 - 基于拓扑图的节点关联关系

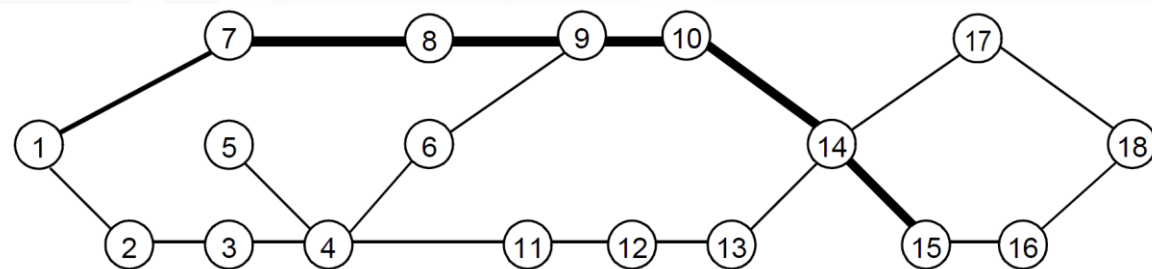


2.2 图的构建 (1)

可视图: visibility graph;

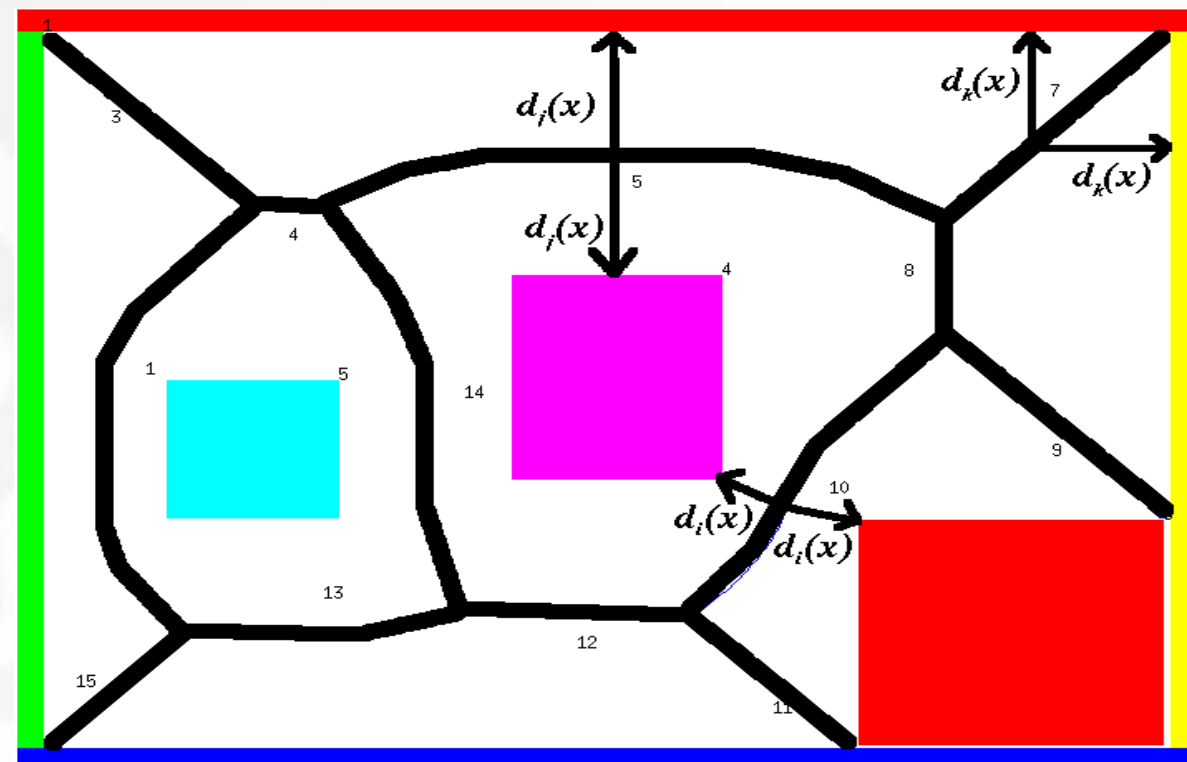
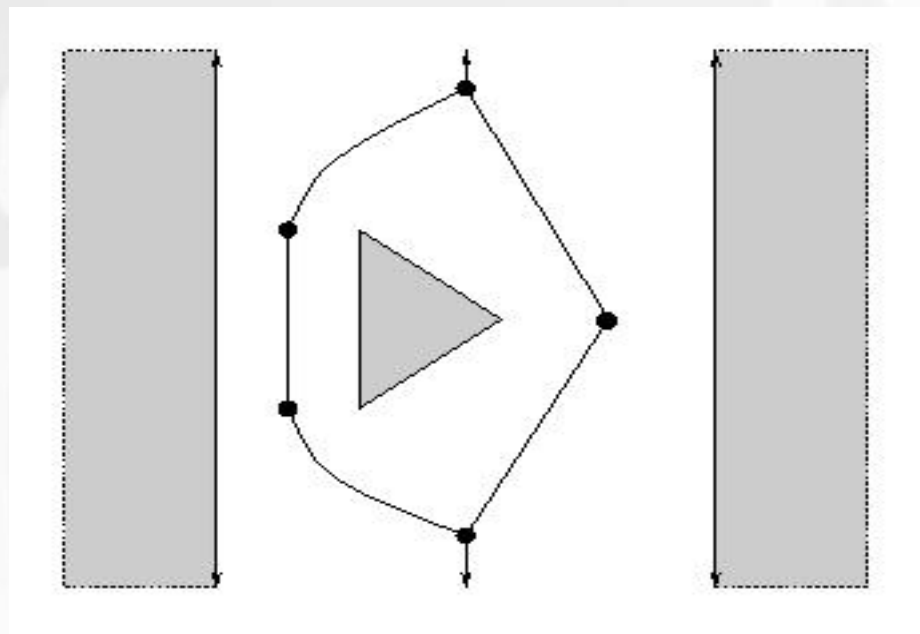
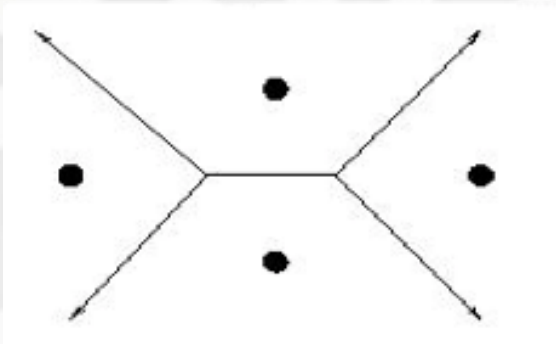


- 可实现无碰撞的最短距离规划
- 太过靠近障碍物



2.2 图的构建 (2)

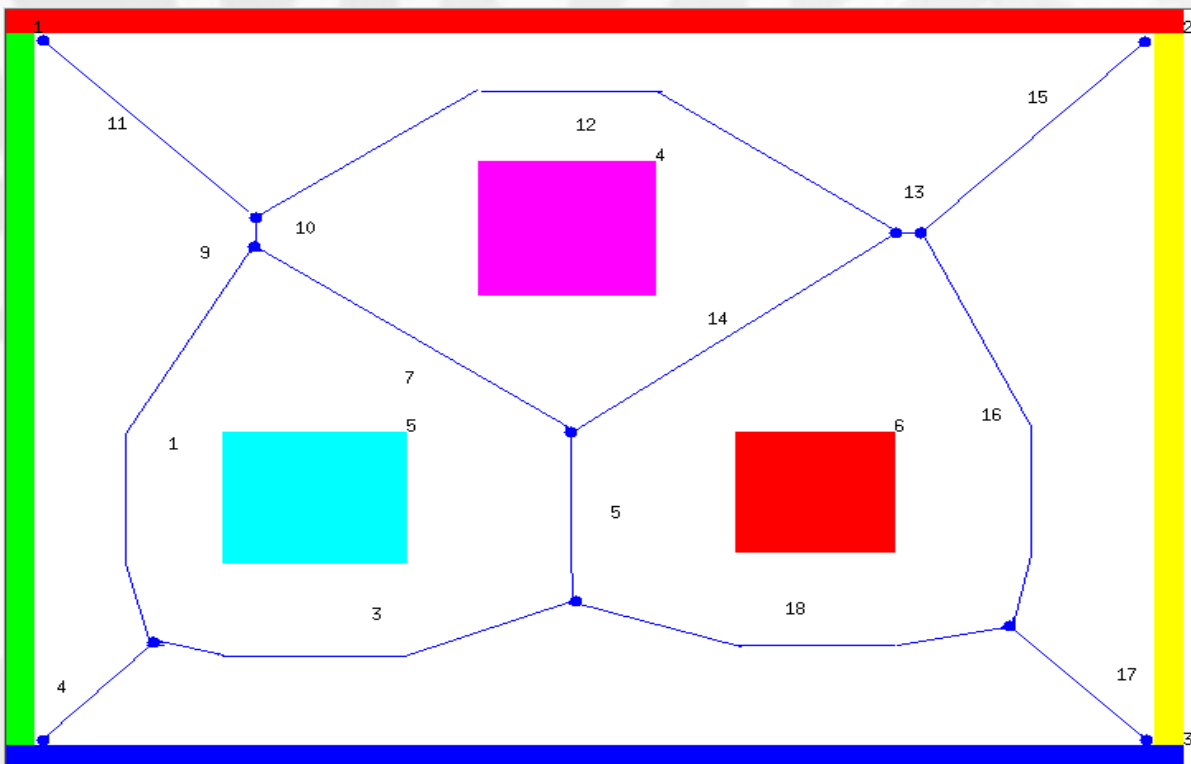
维诺图: Voronoi Graph



- 维诺图产生的路径距离周围两个最近的障碍物的距离相等
- 最大化与障碍物的距离
- 最为安全

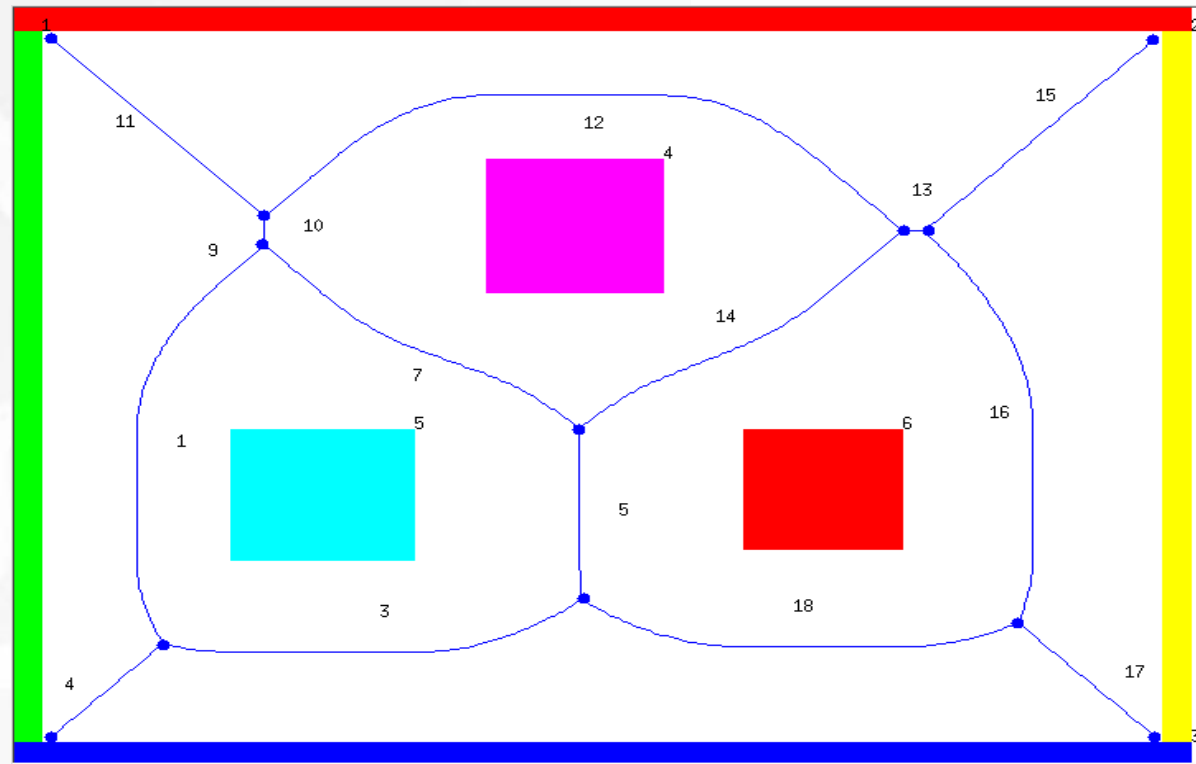
2.2 图的构建 (2)

维诺图: Voronoi Graph



L_1 距离下的维诺图路径

$$|x| + |y| = \text{const}$$

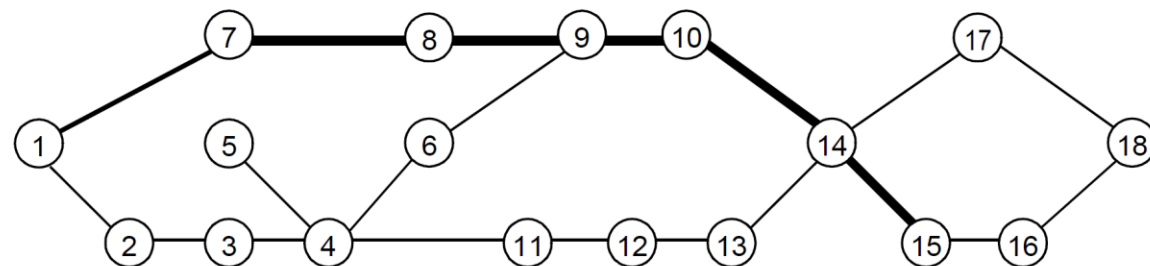
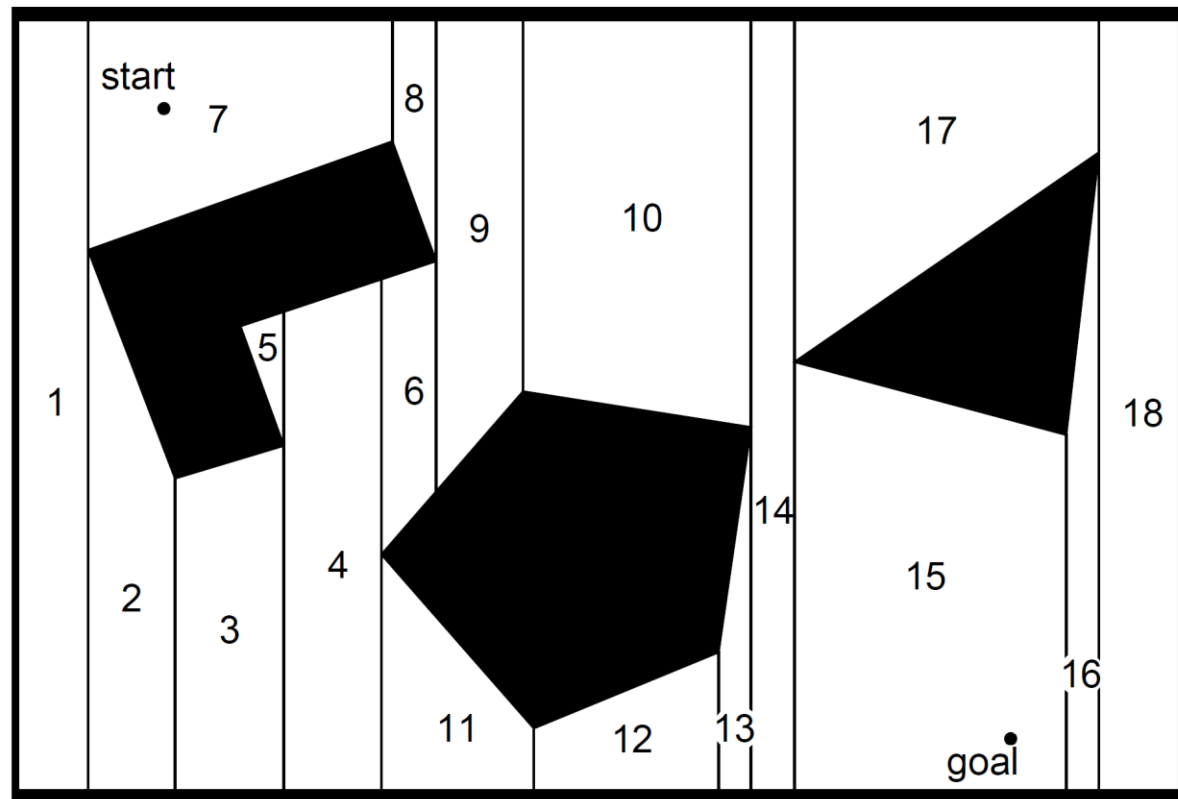


L_2 距离下的维诺图路径

$$x^2 + y^2 = \text{const}$$

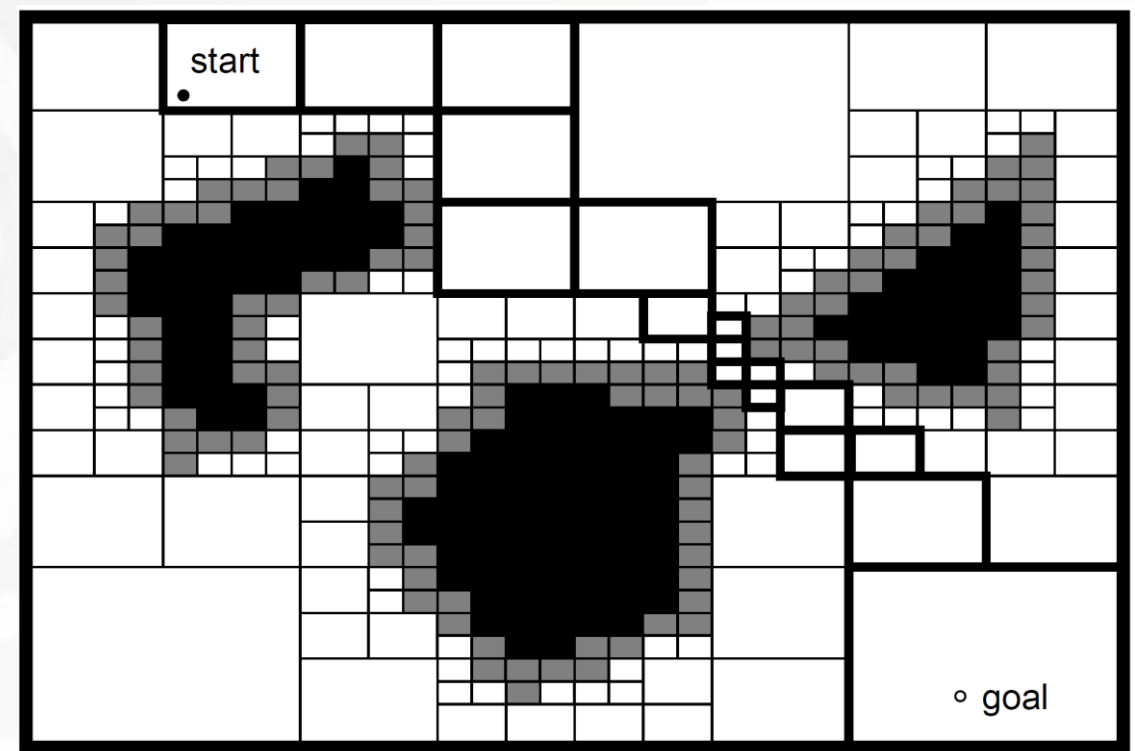
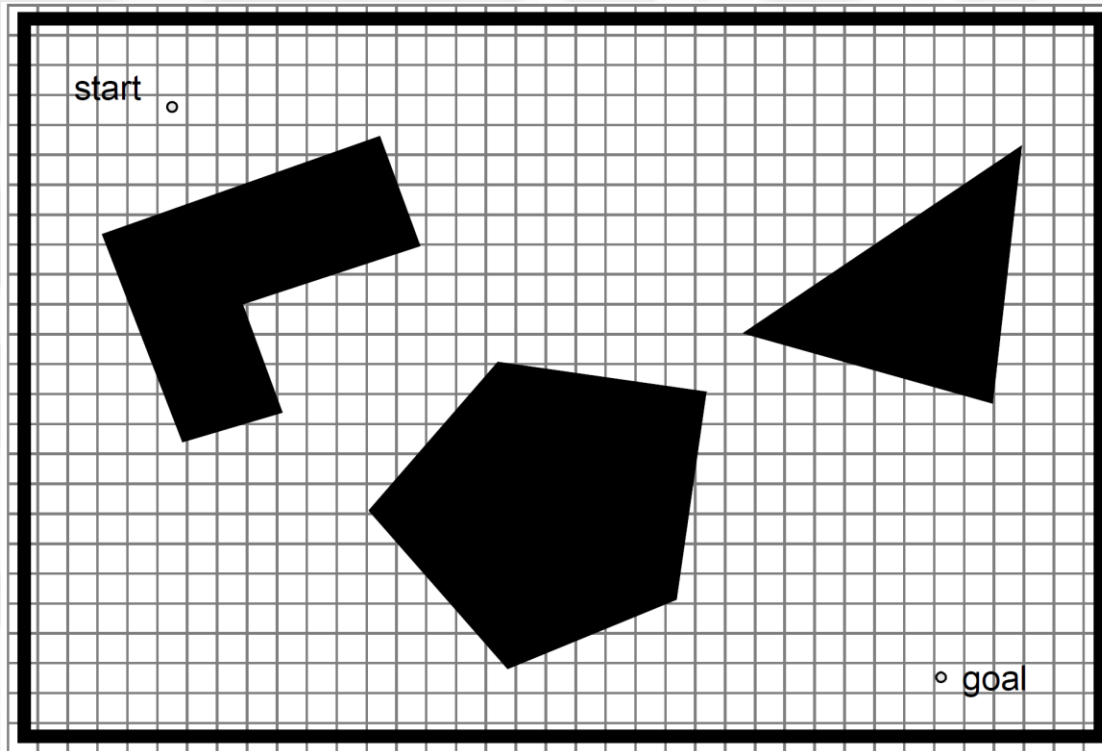
2.2 图的构建 (3)

精确单元分解地图:



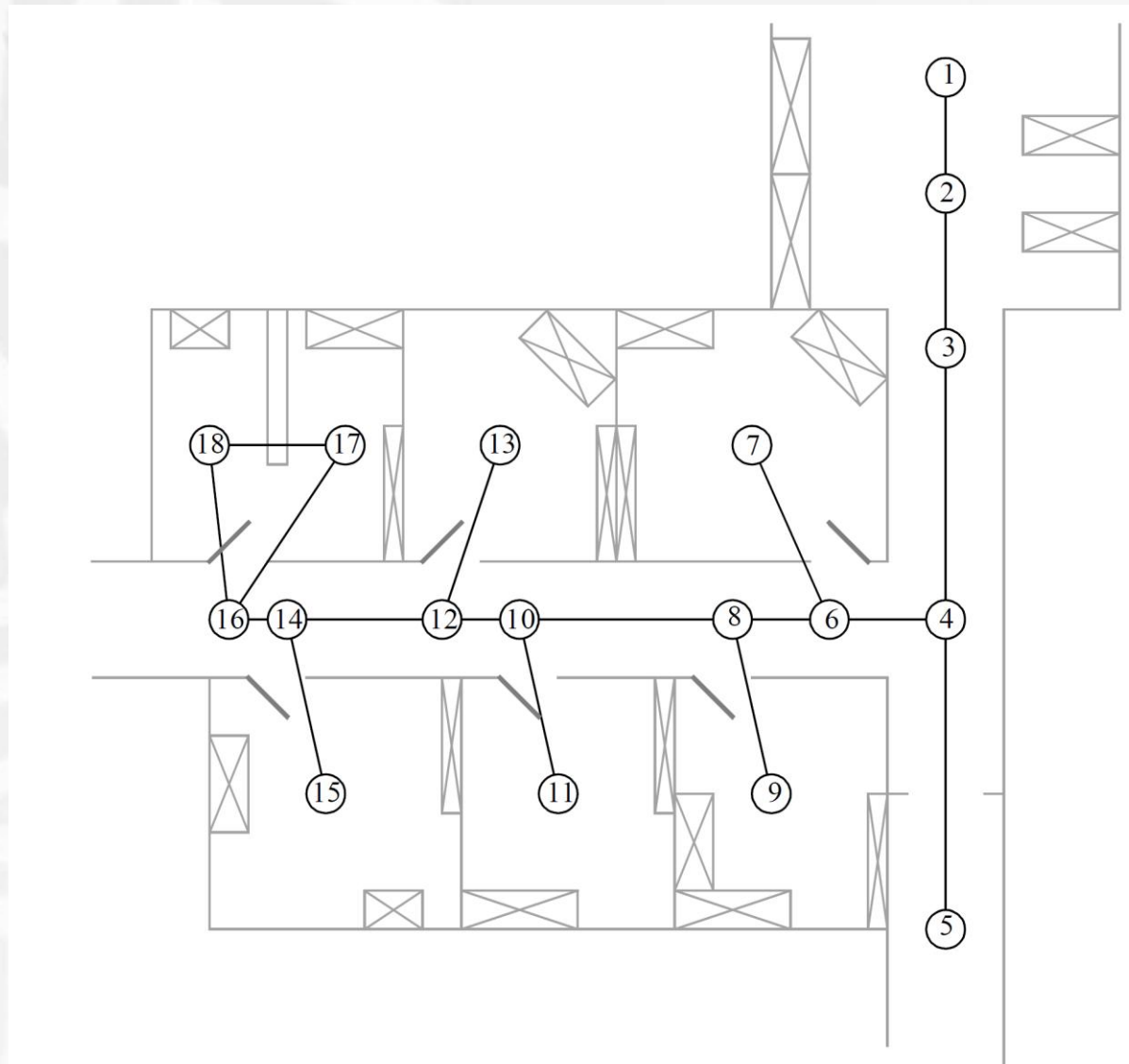
2.2 图的构建 (5)

栅格地图：



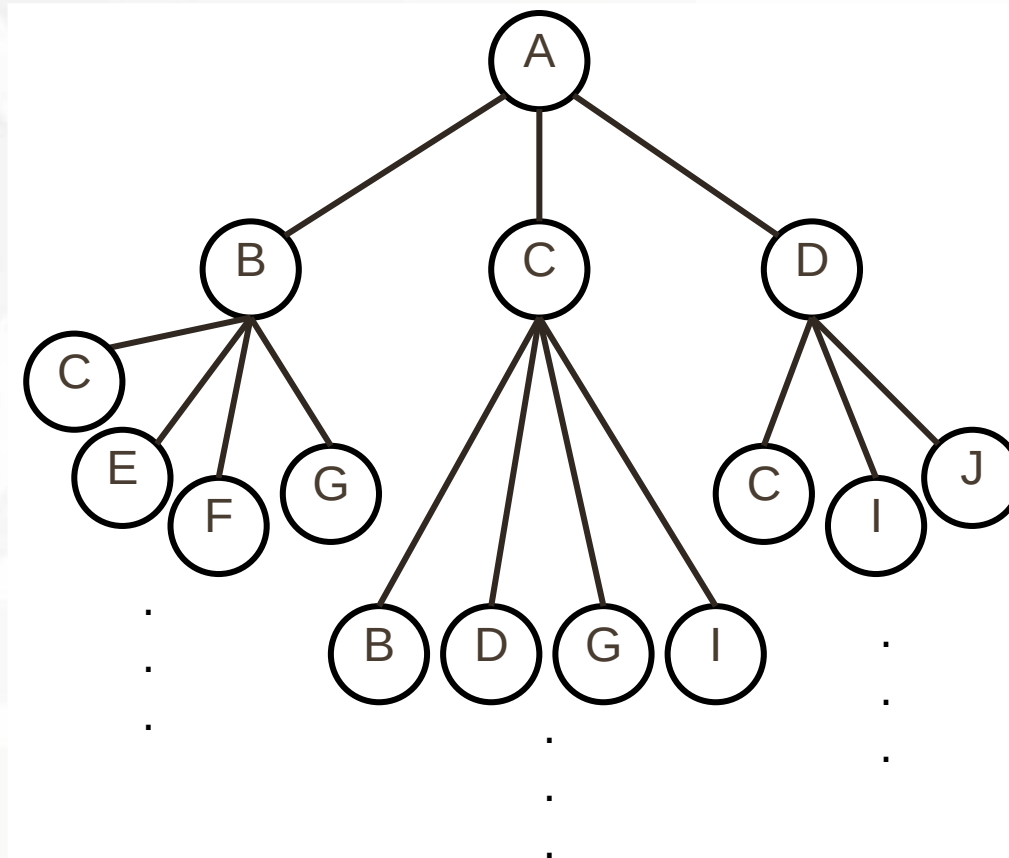
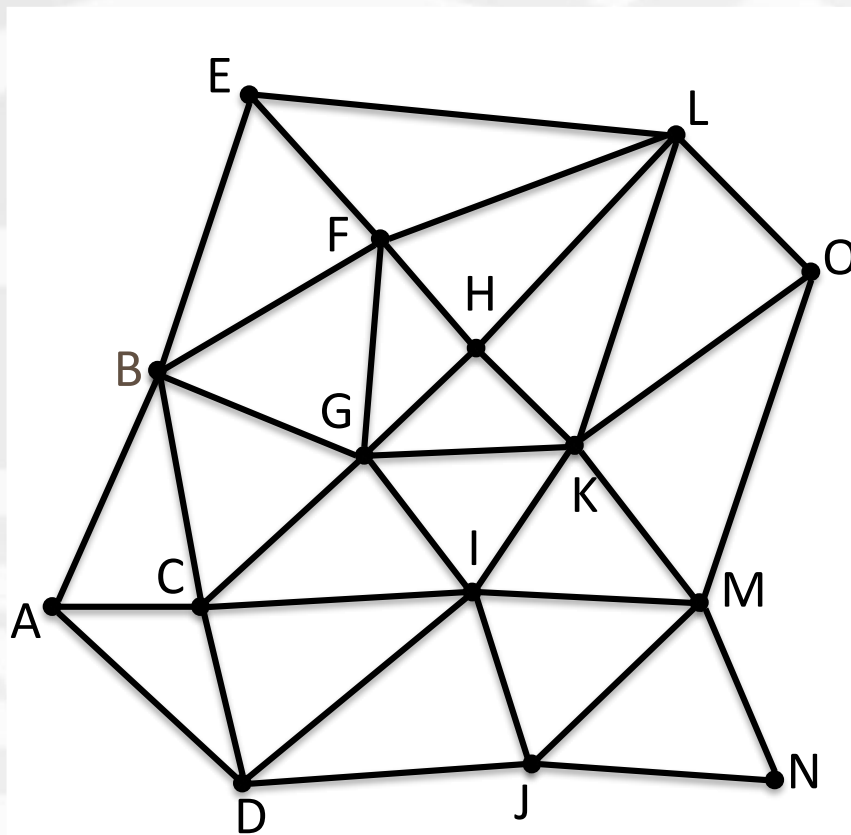
2.2 图的构建 (6)

拓扑地图:



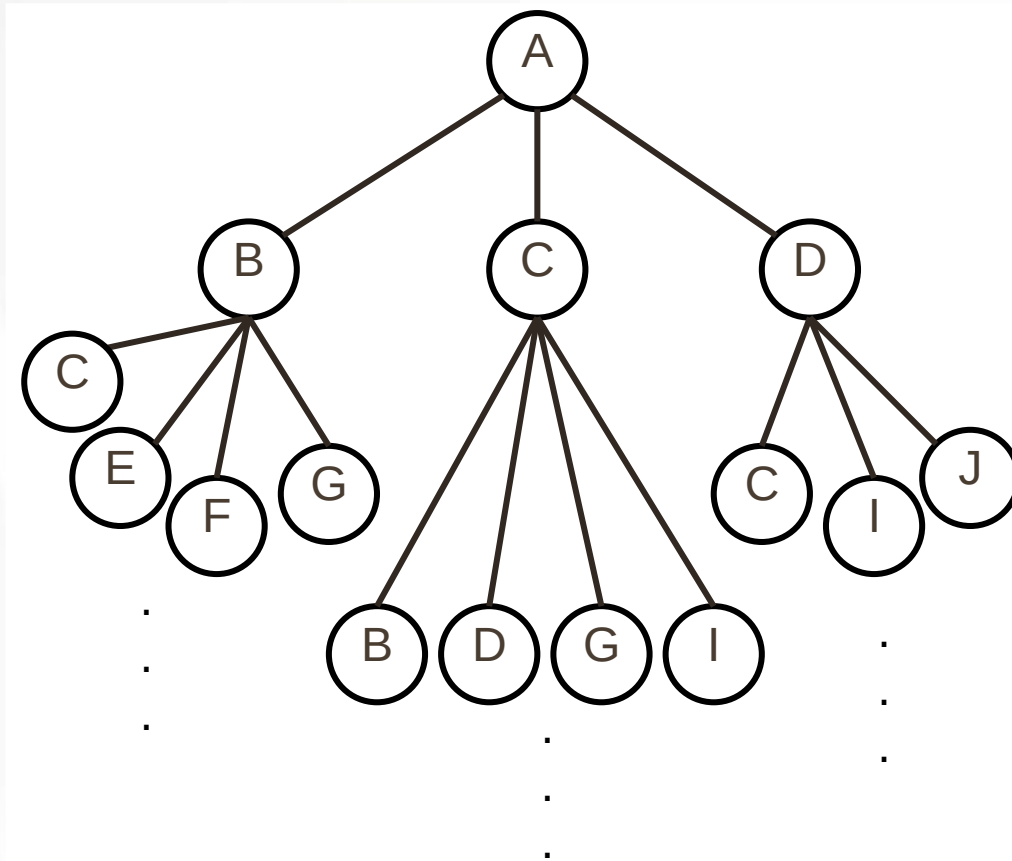
2.2 图的构建 (7) ——搜索树

- 基于构建的图，依照起点及其对应连接关系构建搜索树，实现最优路径的选取



2.2 图的构建 (7) ——搜索树

- ❑ 将起点设为搜索树的根节点
- ❑ 寻求与该节点相连的所有节点作为子节点
- ❑ 每个节点与一个规划路径相关联（搜索树上从根节点到该节点的连线）
- ❑ 对于大部分搜索算法，我们希望尽量避免构建全部搜索树
- ❑ 利用计算机已有的搜索算法可以有效地对树进行遍历



2.2 图的构建 (7) ——搜索树

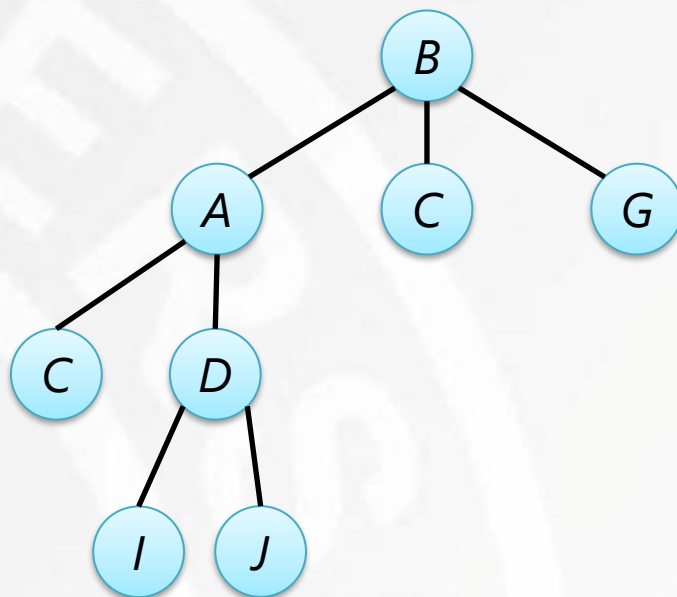
- 一般来说, 从起点开始生长树结构, 直至找到一个解决方案 (到达目标的路径)。
- Expanding a node refers to adding children to the tree, pushing them onto the *open set*
- Try to expand as few tree nodes as possible
- **Open set** maintains a list of frontier (unexpanded) plans
 - Keeps track of what nodes to expand next
 - Often stored as a priority queue
 - For each node in the open list, we know of at least one path to it from the start
- **Closed set:** 用于跟踪已扩展的节点。
 - 对于Closed set中的每个节点, 已经找到了从起点到该节点的成本最低路径。

2.3 图搜索算法 (2)

深度优先搜索:

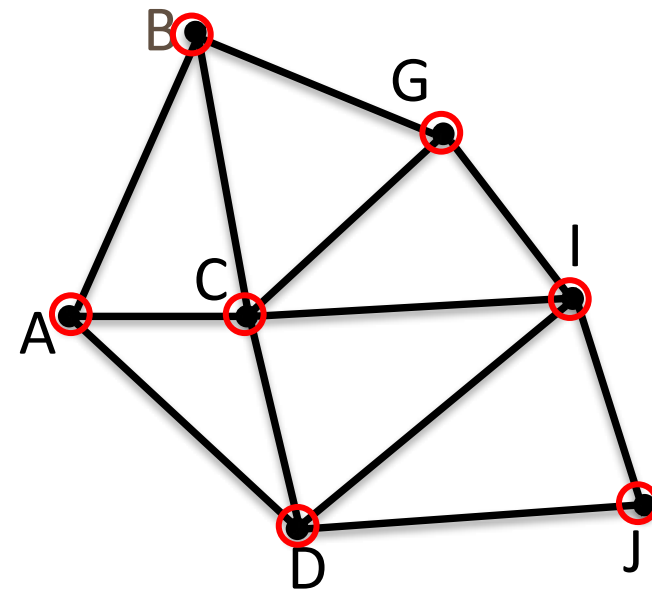
FORWARD SEARCH

```
1  $Q.Insert(x_I)$  and mark  $x_I$  as visited
2 while  $Q$  not empty do
3    $x \leftarrow Q.GetFirst()$ 
4   if  $x \in X_G$ 
5     return SUCCESS
6   forall  $u \in U(x)$ 
7      $x' \leftarrow f(x, u)$ 
8     if  $x'$  not visited
9       Mark  $x'$  as visited
10       $Q.Insert(x')$ 
11   else
12     Resolve duplicate  $x'$ 
13 return FAILURE
```



DFS queue will be stack:

- push ($Q.Insert$) onto the front
- pop ($Q.GetFirst$) from the front



Open List (Q):

{B}
{A, C, G}
{D, C, G}
{I, J, C, G}
{J, C, G}

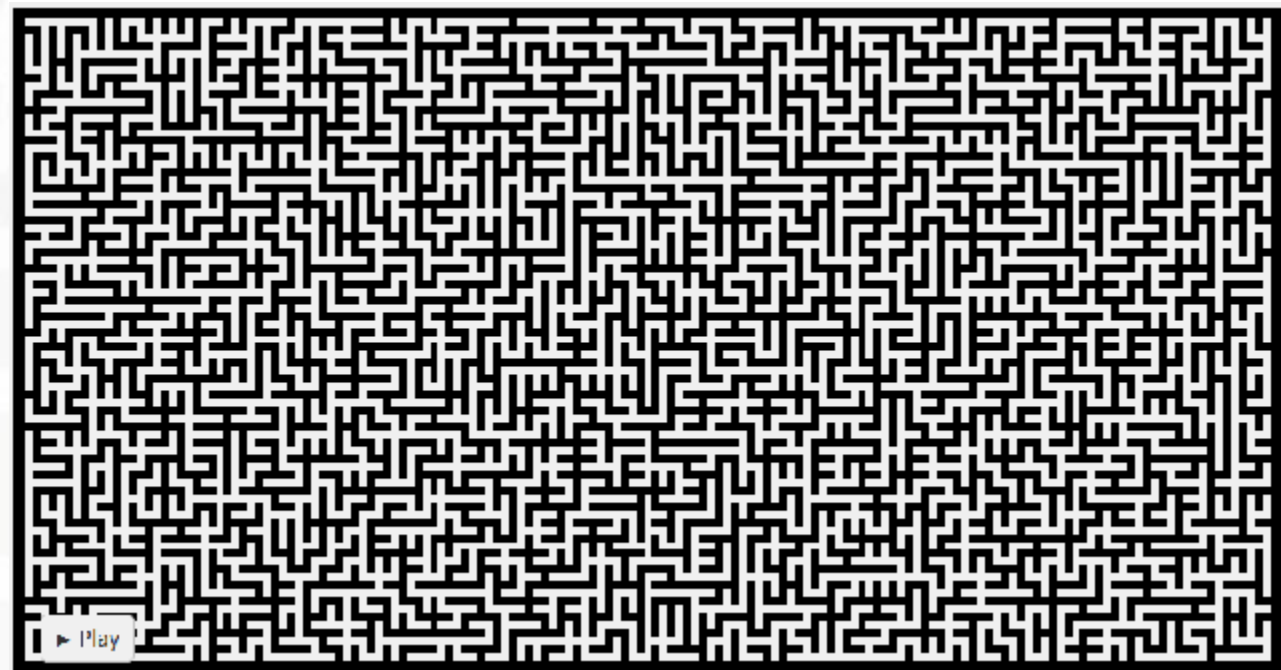
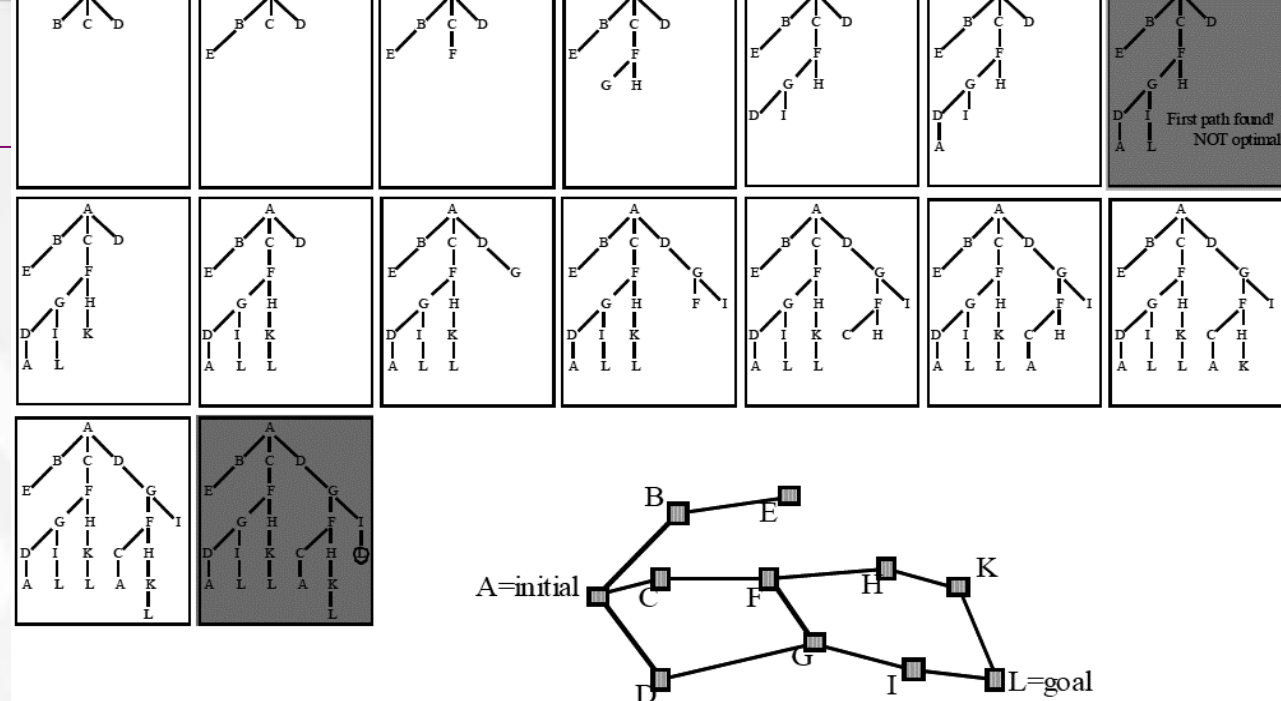
Visited:

{B}
{B, A, C, G}
{B, A, C, G, D}
{B, A, C, G, D, I, J}
{B, A, C, G, D, I, J}

2.3 图搜索算法 (1)

深度优先搜索 (DFS) :

- 以深度做为优先级进行搜索
- 遍历每一个分支，直到找到终点
- 算法简单，易于实现
- 具有完备性
- 不具备最优性，不能保证最短路径
 - ✓ 仅考虑到了图中**节点**间的连接属性，未考虑**边**（即连接代价）
- 无方向性，效率低下

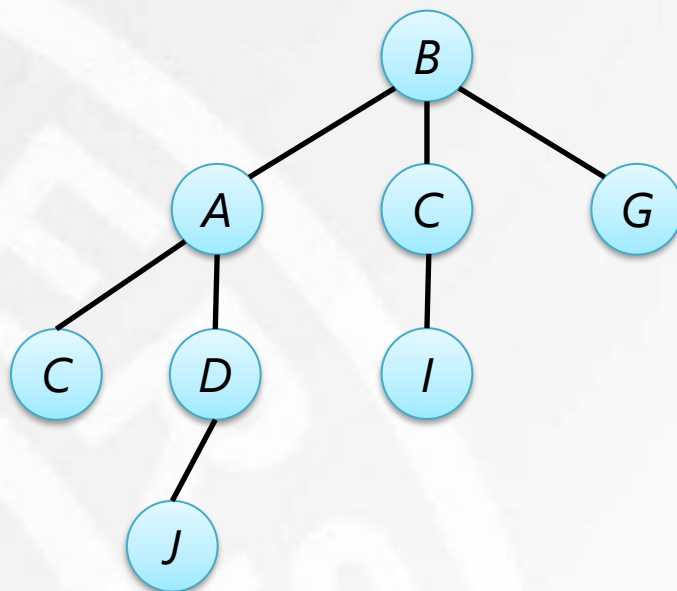


2.3 图搜索算法 (2)

宽度优先搜索:

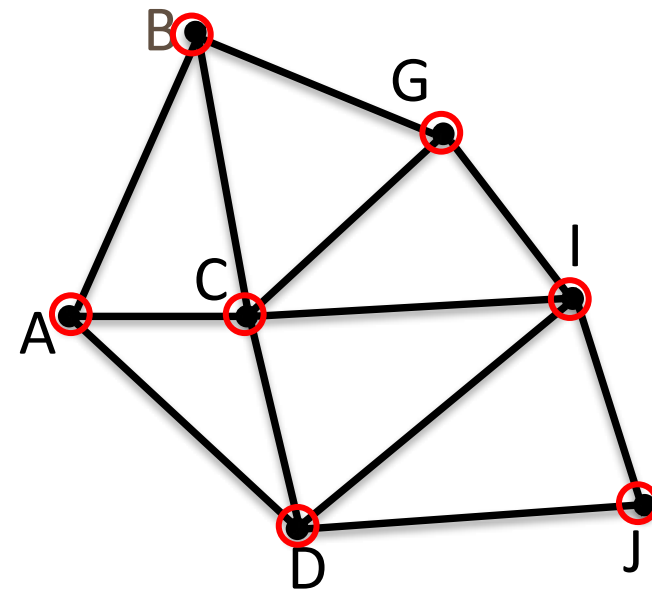
FORWARD SEARCH

```
1  $Q.Insert(x_I)$  and mark  $x_I$  as visited
2 while  $Q$  not empty do
3    $x \leftarrow Q.GetFirst()$ 
4   if  $x \in X_G$ 
5     return SUCCESS
6   forall  $u \in U(x)$ 
7      $x' \leftarrow f(x, u)$ 
8     if  $x'$  not visited
9       Mark  $x'$  as visited
10       $Q.Insert(x')$ 
11   else
12     Resolve duplicate  $x'$ 
13 return FAILURE
```



BFS queue will be FIFO:

- push ($Q.Insert$) onto the end
- pop ($Q.GetFirst$) from the front



Open List (Q):

{B}
{A, C, G}
{C, G, D}
{G, D, I}
{D, I}
{I, J}

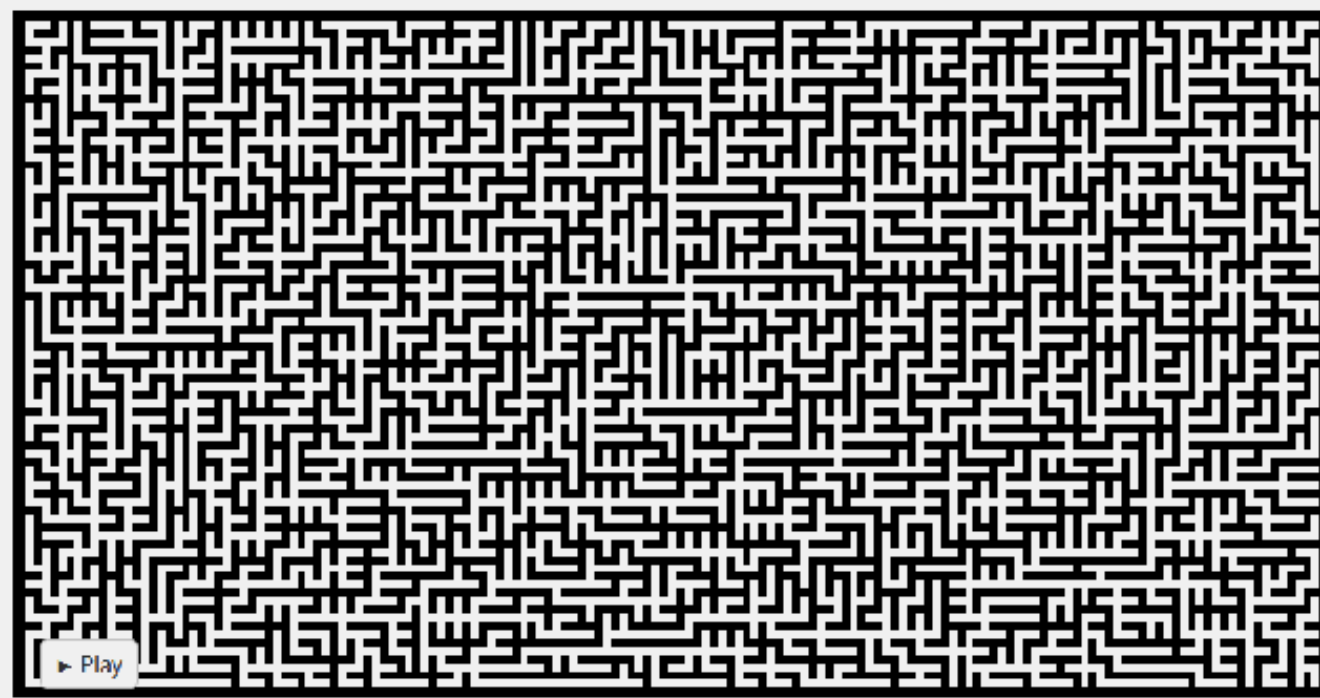
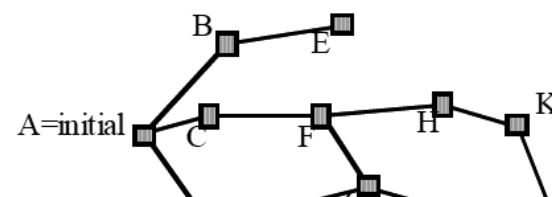
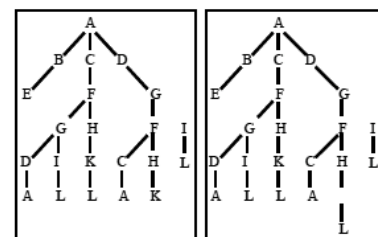
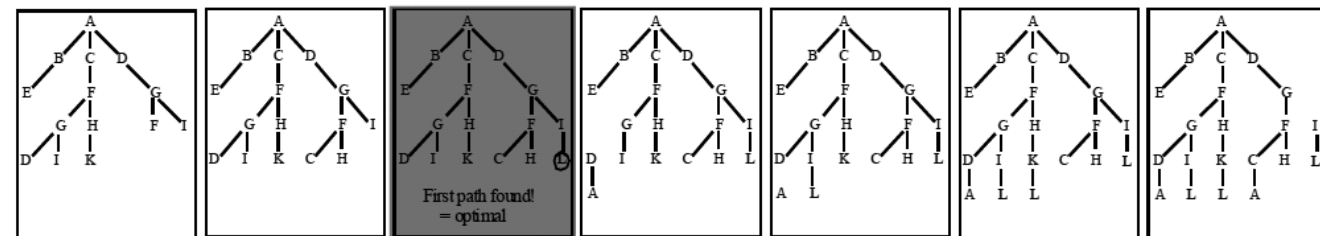
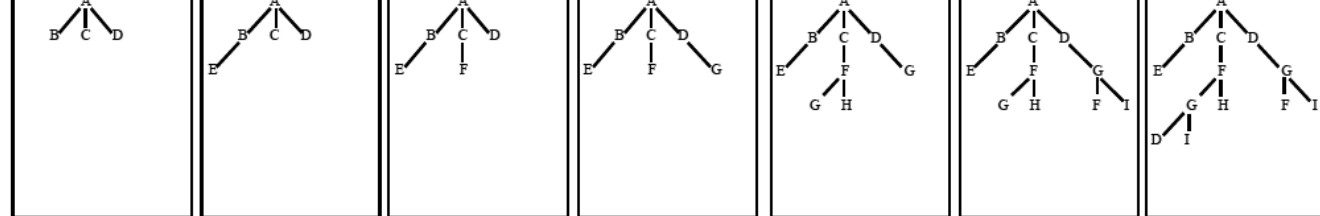
Visited:

{B}
{B, A, C, G}
{B, A, C, G, D}
{B, A, C, G, D, I}
{B, A, C, G, D, I, J}

2.2 图搜索算法 (2)

宽度优先搜索（泛洪搜索）：

- 以广度做为优先级进行搜索
- 遍历起点周围邻近的点，逐步的向外扩散，直到找到终点
- 算法简单，易于实现
- 具有完备性
- 不具备最优性，不能保证最短路径
 - ✓ 仅考虑到了图中**节点**间的连接属性，未考虑**边**（即连接代价）
- 无方向性，效率低下

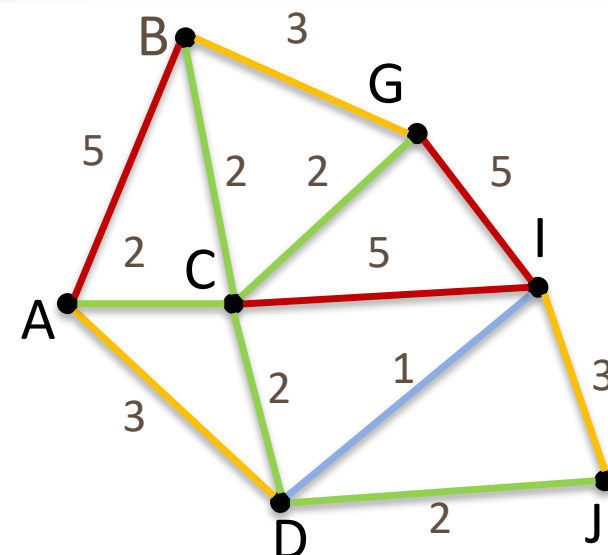
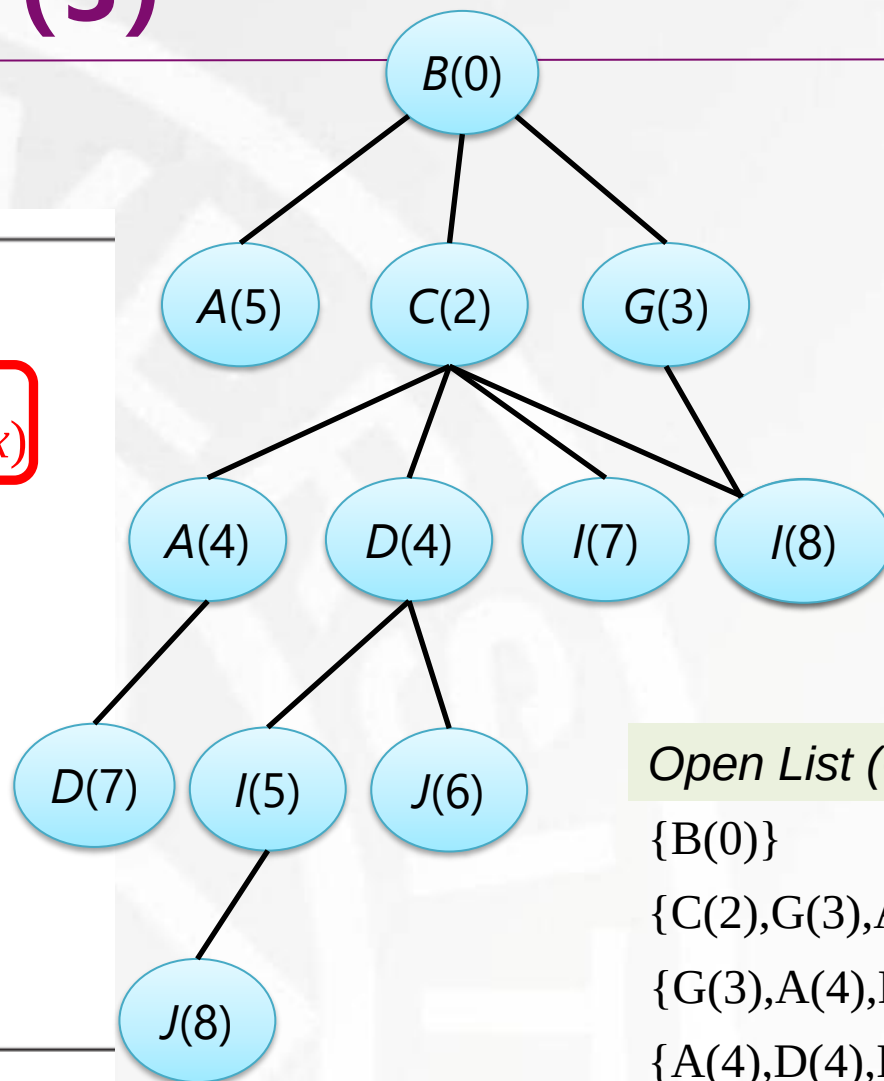


2.2 图搜索方法 (3)

Dijkstra算法:

FORWARD SEARCH

```
1  $Q.Insert(x_I)$ 
2 while  $Q$  not empty do
3    $x \leftarrow Q.GetFirst(), R.Insert(x)$ 
4   if  $x \in X_G$ 
5     return SUCCESS
6   forall  $u \in U(x)$ 
7      $x' \leftarrow f(x, u)$ 
8     if  $x \notin Q$  or  $R$ 
9       Mark  $x'$  as visited
10       $Q.Insert(x')$ 
11   else the least
12     Resolve duplicate  $x'$ 
13 return FAILURE
```



Open List (Q):

{B(0)}
{C(2), G(3), A(5)}
{G(3), A(4), D(4), I(7)}
{A(4), D(4), I(7)}
{D(4), I(7)}
{I(5), J(6)}
{J(6)}

Close List (R):

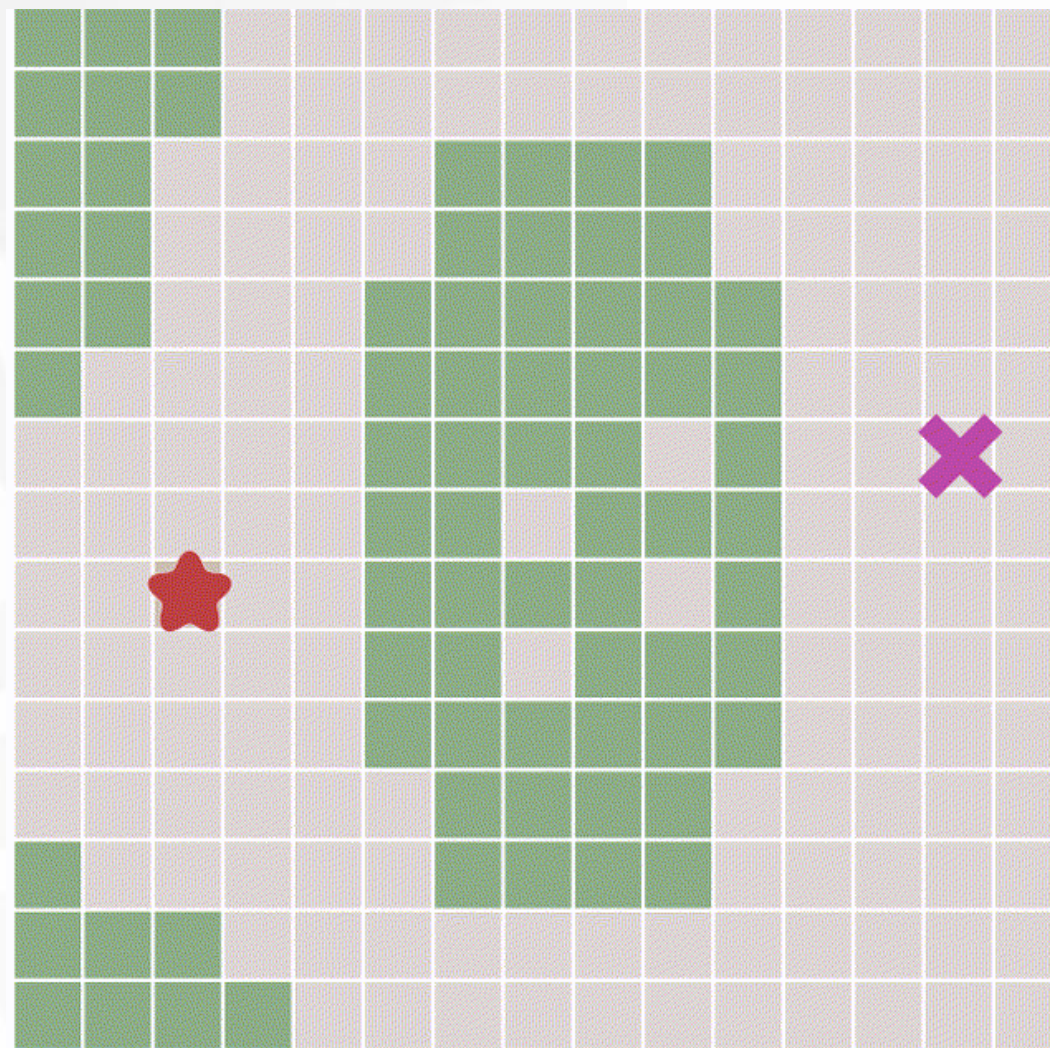
{B(0)}
{B(0)}
{B(0), C(2)}
{B(0), C(2), G(3)}
{B(0), C(2), G(3), A(4)}
{B(0), C(2), G(3), A(4), D(4)}
{B(0), C(2), G(3), A(4), D(4), I(5)}

- 优先弹出距离起点最小的节点
- Open列表中保存距离起点最小的节点

2.2 图搜索方法 (3)

Dijkstra算法(1959):

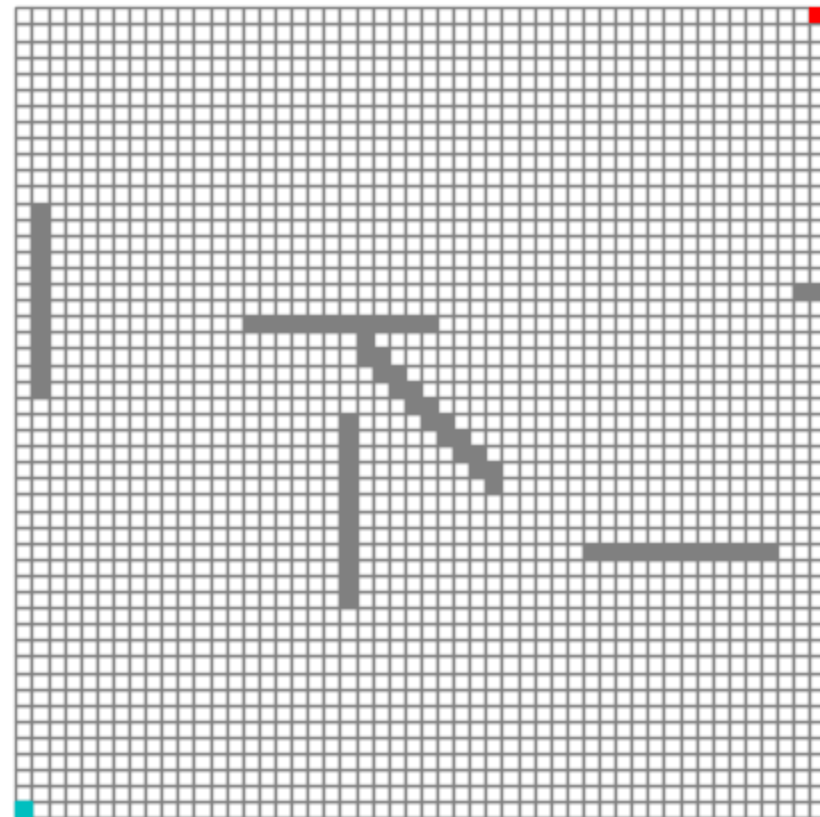
- 基于宽度优先算法：完备性
- 计算每一个节点距离起点的总移动代价。
- 一个优先队列结构。对于所有待遍历的节点，放入优先队列中会按照代价进行排序。
- 每次从优先队列中选出代价最小的作为下一个遍历的节点
- 保证路径规划的最优性
- 无法解决效率问题
 - 地图信息利用不充分



2.2 图搜索方法 (4)

A*算法:

- 在Dijkstra算法基础上, 结合目标进行搜索
- 一种启发式算法: 目标牵引
- 评估函数:
 - 从出发点到当前节点的路径代价——Dijkstra算法
 - 当前节点到目标节点的路径代价估计——目标牵引
- 算法具有完备性和可采纳性(admissible)——理论证明
 - 完备性: 对于一个路径规划算法, 在最优路径存在的情况下, 如果它一定能够找到一条从源节点s到目标节点的最优路径。
- 最为经典的启发式搜索策略: 后续算法的基础



2.2 图搜索方法 (4)

A*算法:

- 开放列表(Open List): 将路径规划过程中待检测的节点存放于Open List中
- 关闭列表(Close List): 已检测过的格子则存放于Close List中。
- 父节点 (parent) : 在路径规划中用于回溯的节点, 可考虑为双向链表结构中的父结点指针。
- 路径评价函数:

$$F(n) = G(n) + H(n)$$

□ $G(n)$ 代表的是从初始位置A沿着已生成的路径到指定位置的移动开销。

□ $H(n)$ (启发函数)当前节点到目标节点的路径代价估计 (**不考虑障碍**) 。

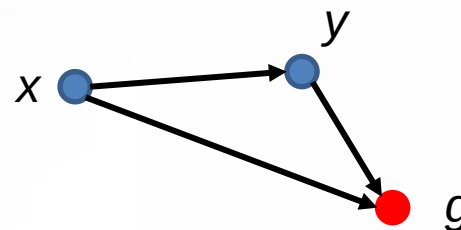
■ 可采纳性admissible :

$$H(x) \leq d(x, goal)$$

距离目标的实际代价

■ 一致性consistent:

$$H(x) \leq d(x, y) + H(y)$$



2.2 图搜索方法 (4)

- 可采纳性admissible :

$$H(x) \leq d(x, goal)$$

- 一致性consistent:

$$H(x) \leq d(x, goal) + H(y)$$

- 启发函数 (Heuristics Function) :

- 曼哈顿距离: $H(n) = |x_g - x_n| + |y_g - y_n|$

- 欧几里得距离: $H(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2}$

- 切比雪夫距离: $H(n) = \max(|x_g - x_n|, |y_g - y_n|)$

- Zero: Dijkstra算法

2.2 图搜索方法 (4)

A*算法流程:

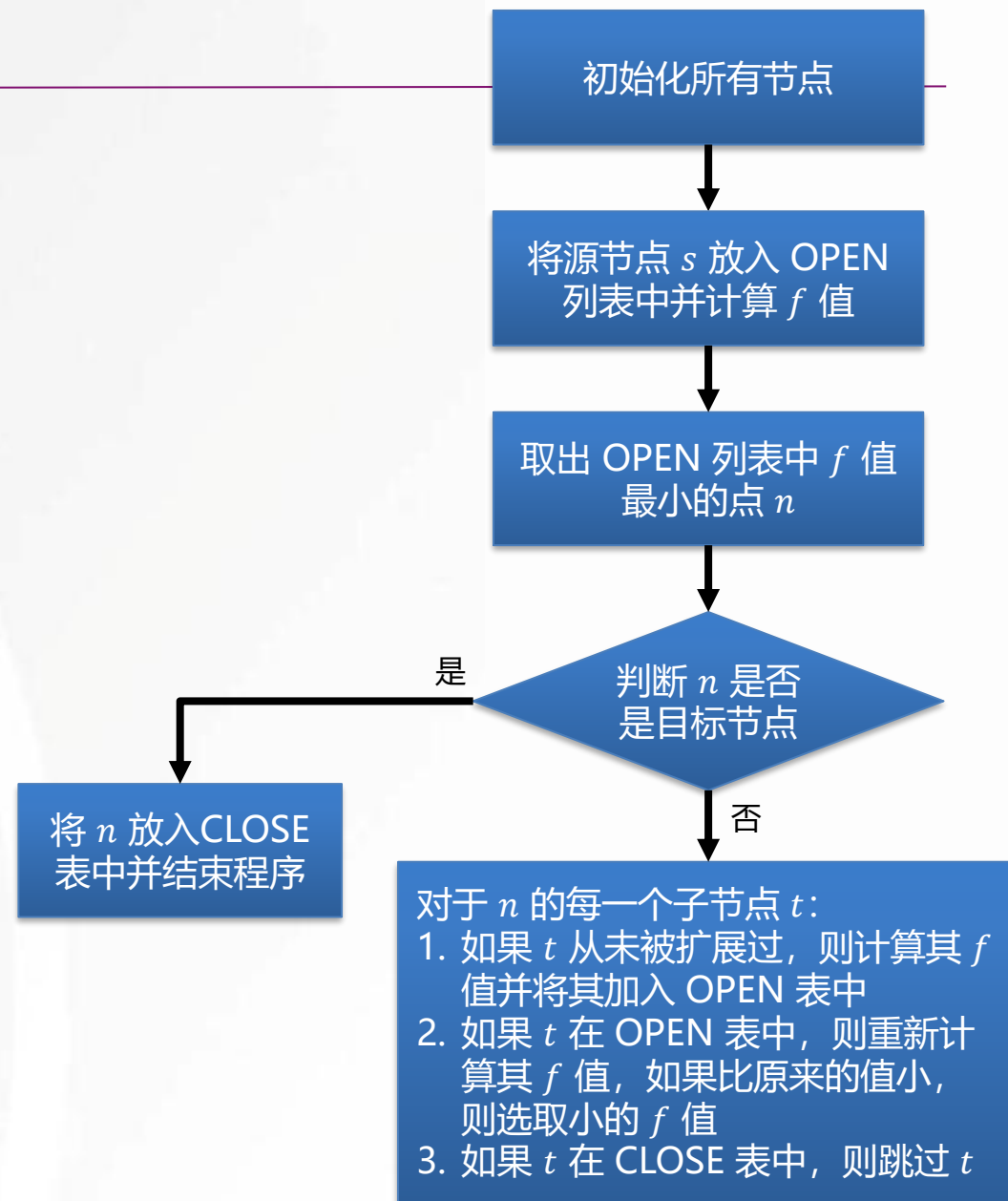
Step1: 将源节点 s 放入OPEN列表中并计算 F 值

Step2: 取出OPEN列表中 F 值最小的点 n , 若点 n 为目标节点, 则结束程序, 完成规划

Step3: 寻找周围可达节点, 并将 n 设为其父节点, 对于 n 的每一个子节点 t :

- 如果 t 从未被扩展过, 则计算其 F 值并将其加入OPEN表中
- 如果 t 在OPEN表中, 则重新计算其 F 值, 如果比原来的值小, 则选取小的 F 值
- 如果 t 在CLOSE表中, 则跳过 t

Step4: 从 Open List 中移除 n , 放入Close列表, 返回

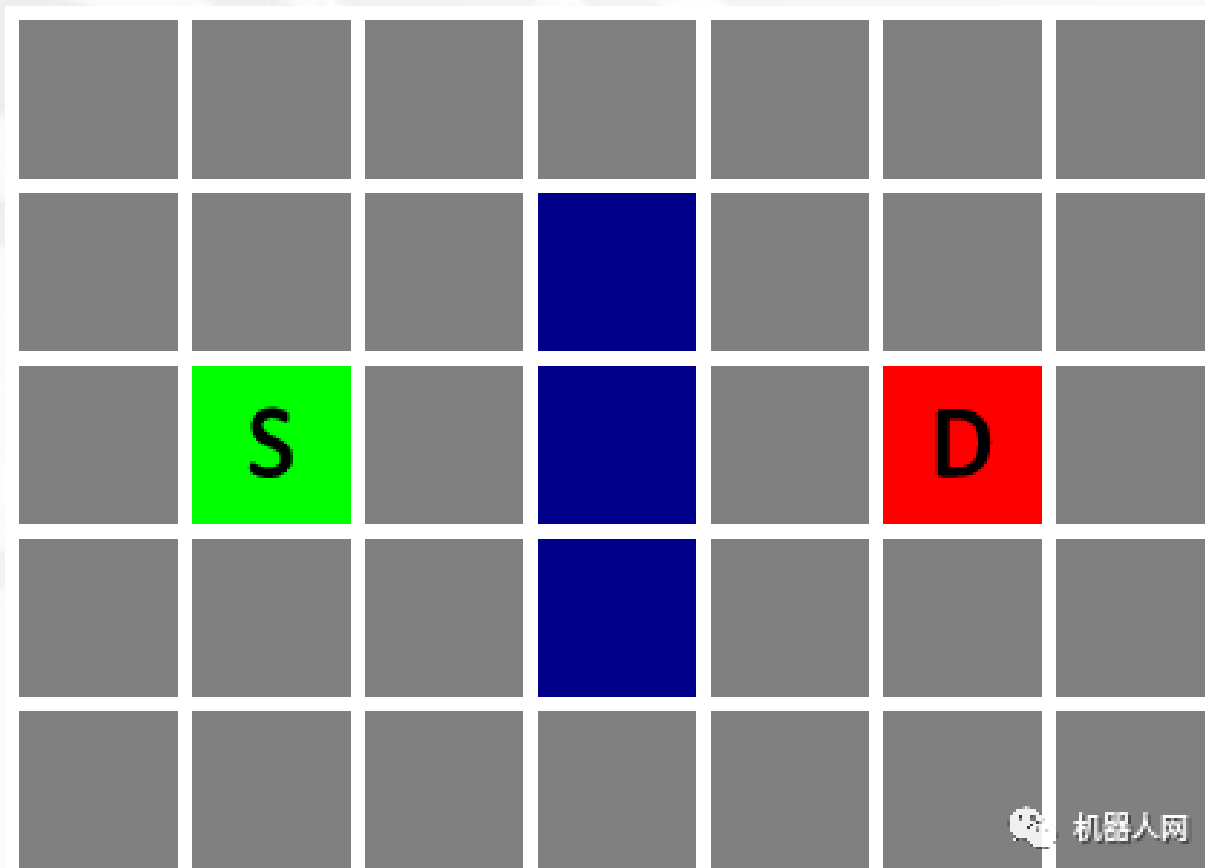


2.2 图搜索方法 (4)

A*算法例题:

Step1: 将源节点s放入OPEN列表中并计算 F 值

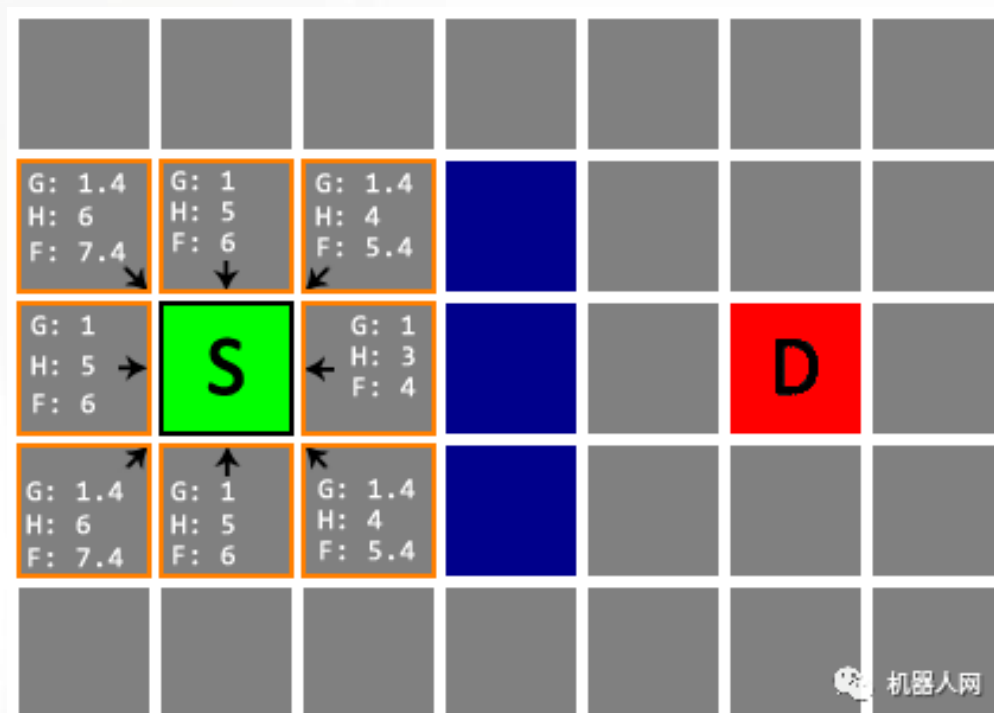
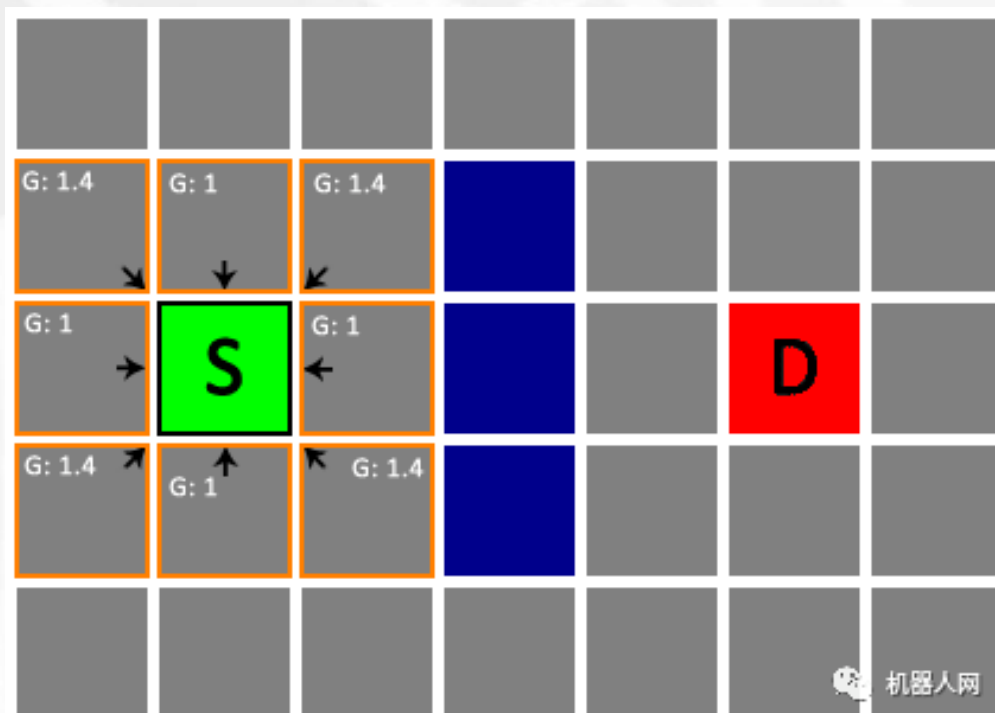
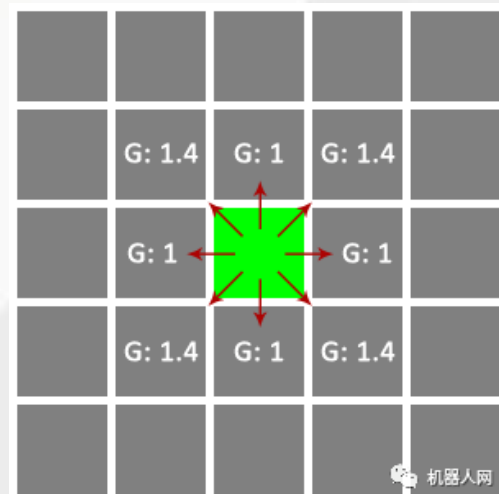
Step2: 取出OPEN列表中 F 值最小的点 n , 若点 n 为目标节点, 则结束程序, 完成规划



2.2 图搜索方法 (4)

Step3: 寻找周围可达节点, 并将 n 设为其父节点, 对于 n 的每一个子节点 t :

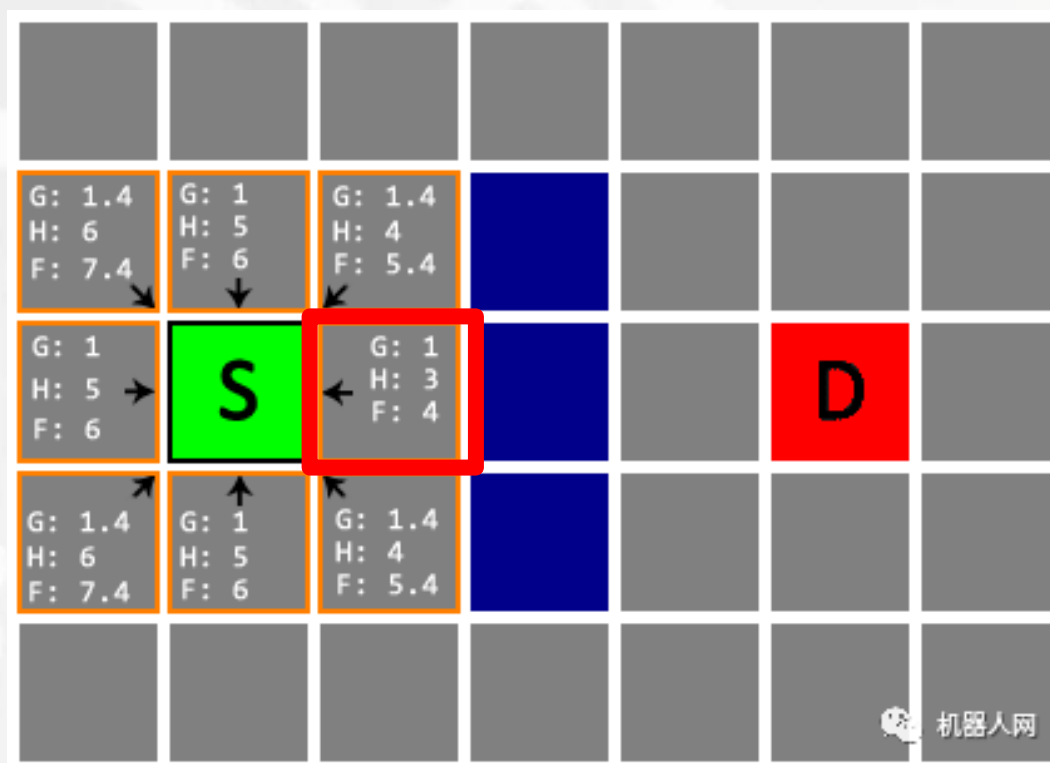
- 如果 t 从未被扩展过, 则计算其 F 值并将其加入OPEN表中
- 如果 t 在OPEN表中, 则重新计算其 F 值, 如果比原来的值小, 则选取小的 F 值
- 如果 t 在CLOSE表中, 则跳过 t



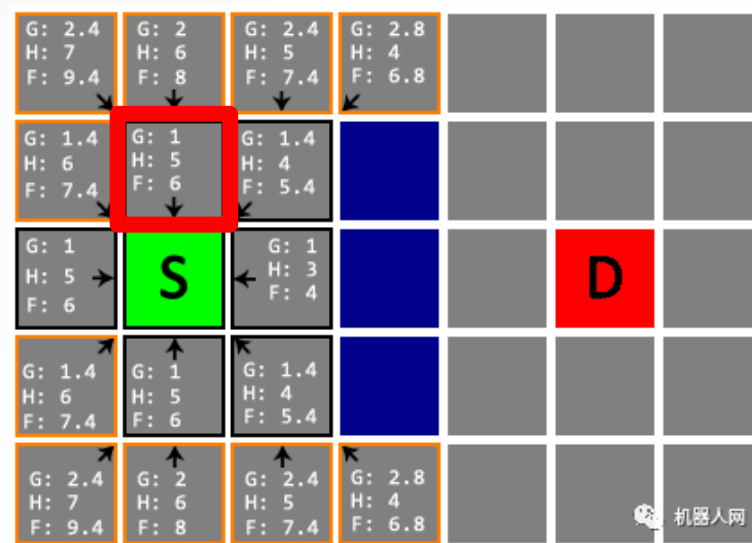
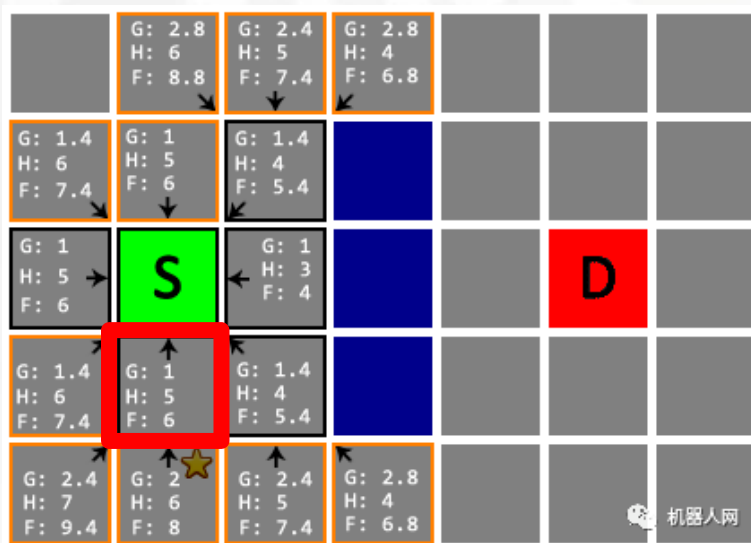
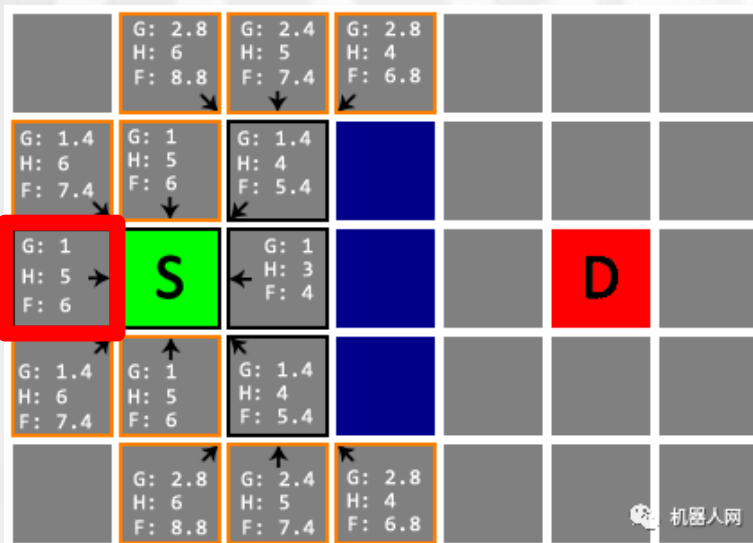
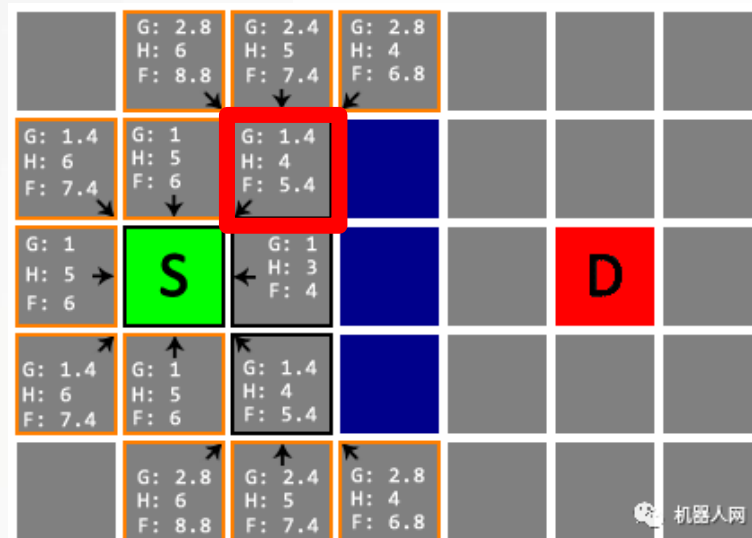
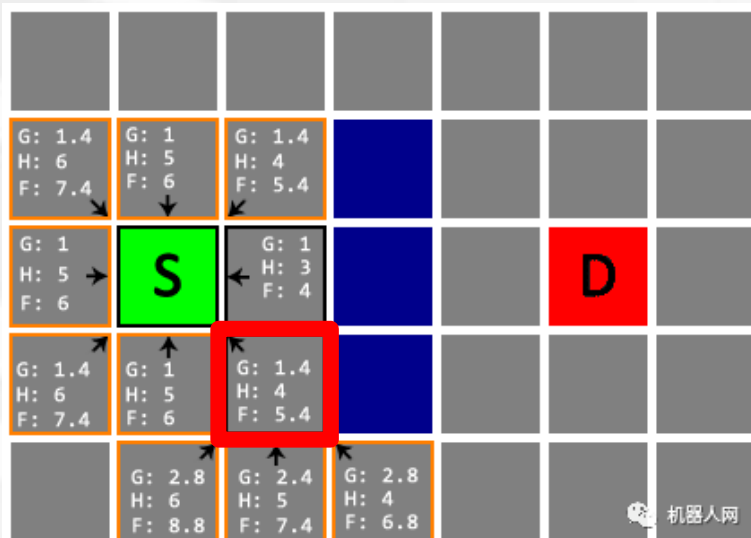
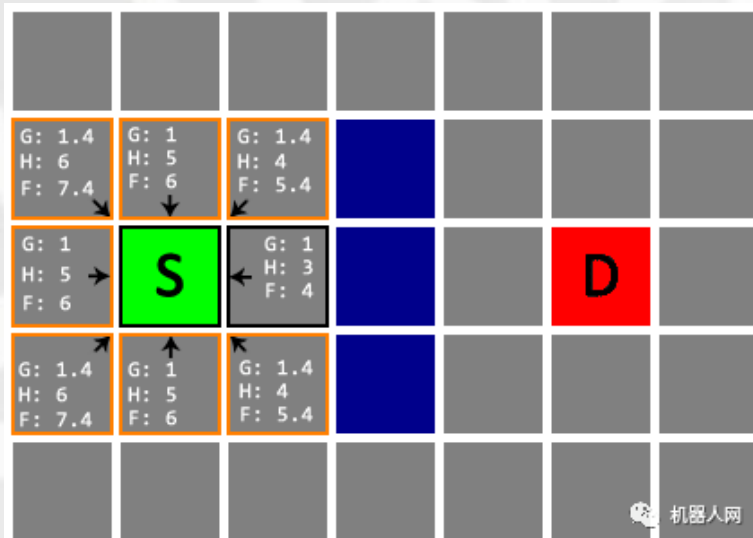
2.2 图搜索方法 (4)

Step 4: 当前节点从 Open List 中移除，放入 Closed List，重返回Step2。

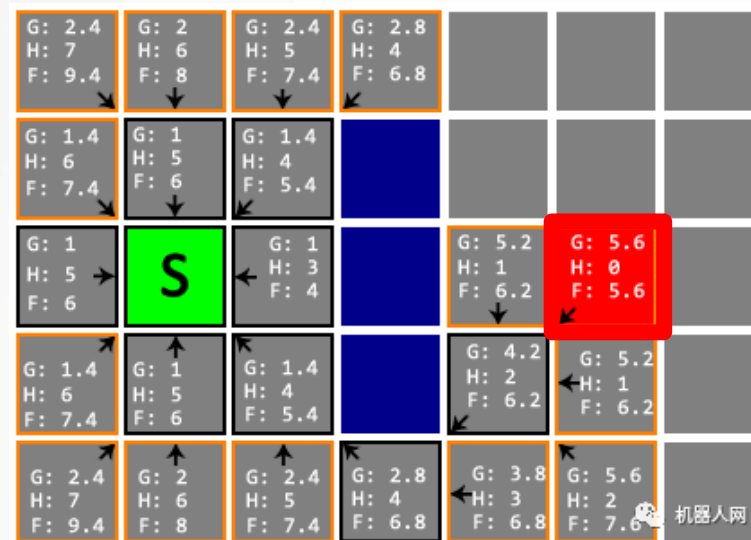
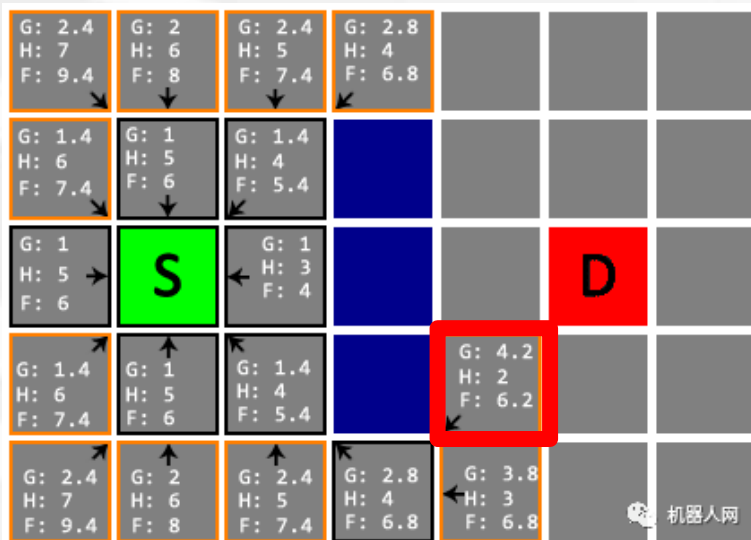
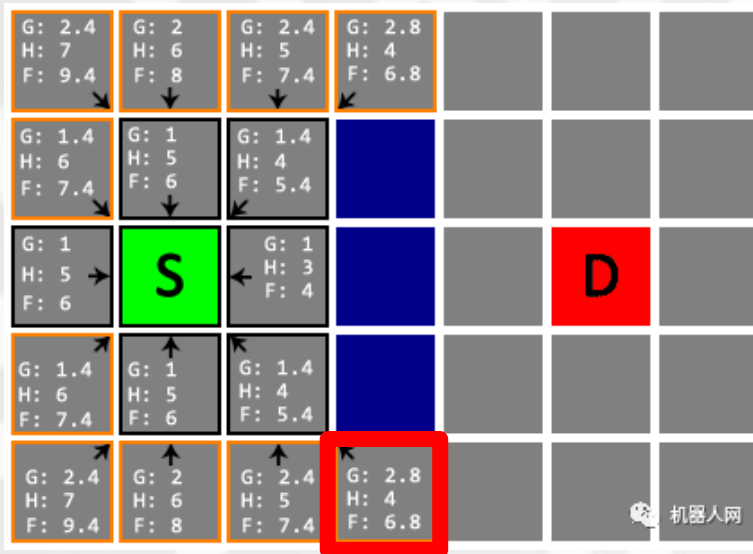
Step 5: 取出OPEN列表中 F 值最小的点 n ，若点 n 为目标节点，则结束程序，完成规划



2.2 图搜索方法(4)

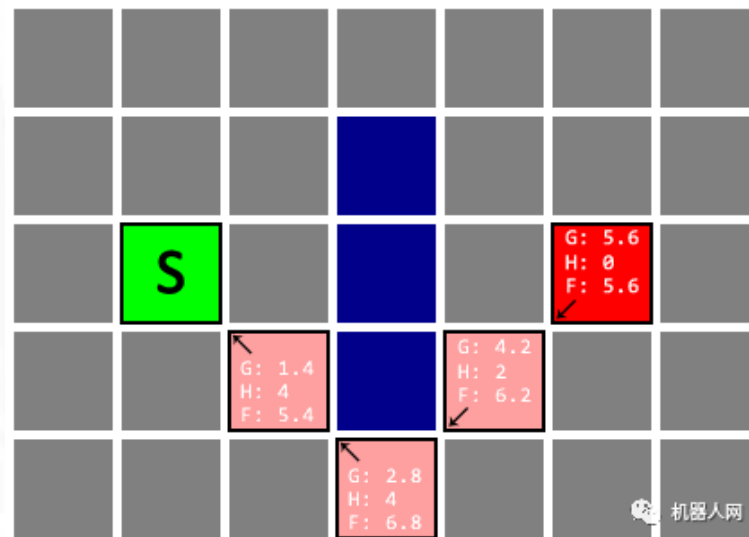


2.2 图搜索方法(4)



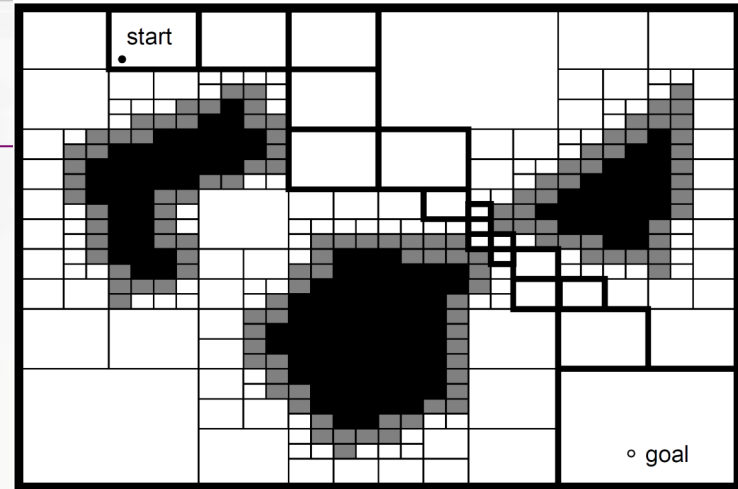
从终点开始，依次向父节点移动，直到回到起点，途经即最佳路线。

<https://qiao.github.io/PathFinding.js/visual/>



2.2 图搜索方法(4)

- 由于启发函数的存在，算法效率明显高于Dijkstra算法
- 最佳路线可能有多条
- 启发式函数的估算方法优劣是影响算法效率的重要因素
- 除了网格地图，A* 算法也能处理其他复杂甚至不规则多边形镶嵌的地图。
- A* 算法并不保证得到的路线是平滑的。
- A* 算法距离最短，非运行时间最短：转向成本
 - ✓ 增加额外的 G 值
- 搜索效率如何提升：
 - ✓ Open List数据结构：从中取出 F 值最小的节点，需要遍历整个 Open List，对其排序。
 - ✓ 多分辨率地图
 - ✓ 加大启发式函数的权重——权重A*算法 (Weighted A*)



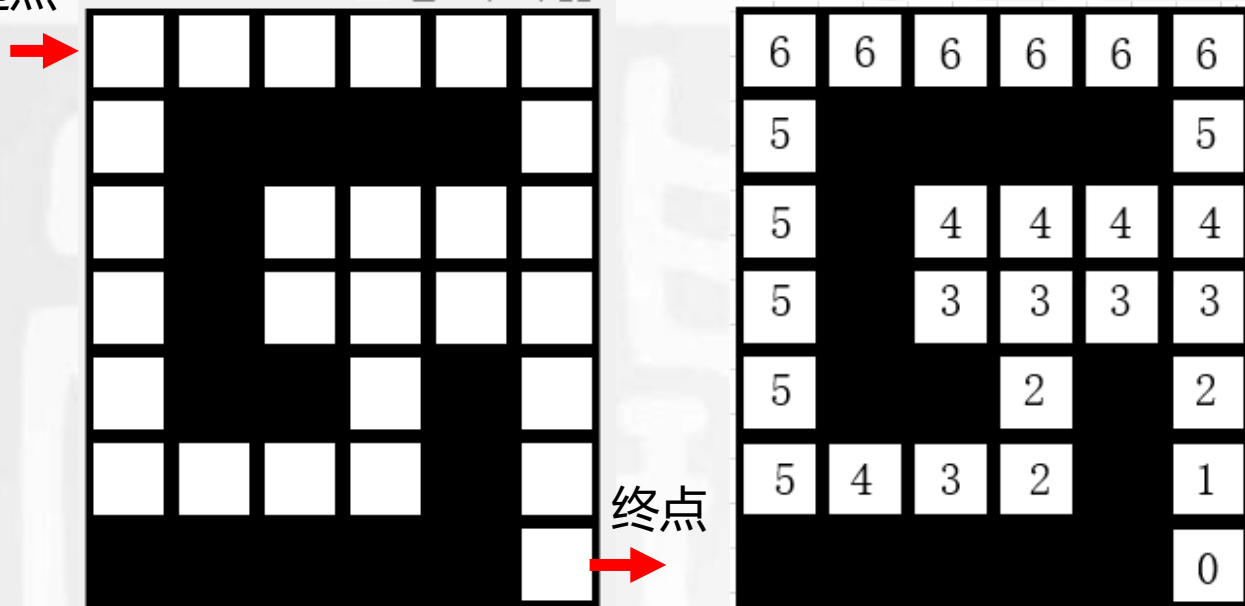
2.2 图搜索方法 (5)

权重A*算法 (Weighted A*)

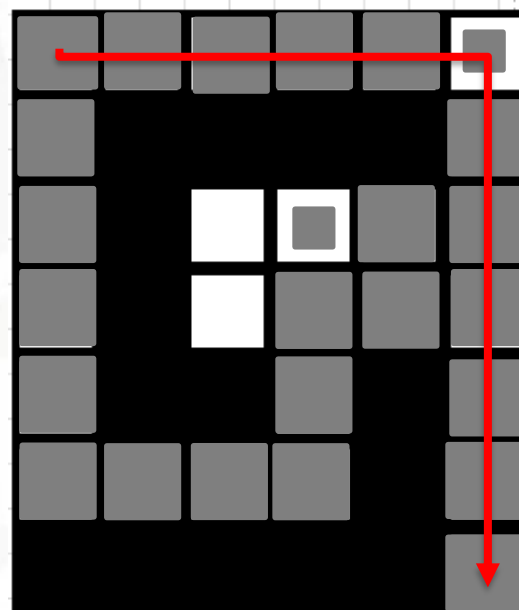
- A*算法限制了迪杰斯特拉算法在扩展节点过程中的**方向**，从而实现了算法计算效率的提升；
- 当启发式信息的权重在评估函数中的比重增加时，对于扩展节点的限制作用会更加明显
- 膨胀系数 ϵ ：启发函数的权重，大于等于1
- 路径评价函数：

$$F(n) = G(n) + \epsilon H(n)$$

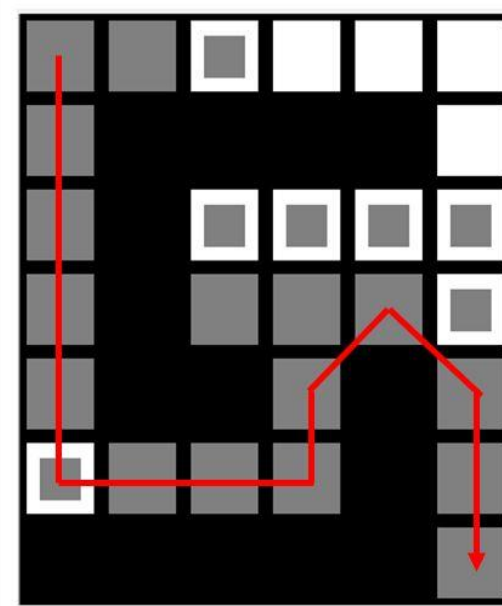
起点



H采用切比雪夫距离



A*算法
23个节点



膨胀系数 $\epsilon = 2.5$
16个节点

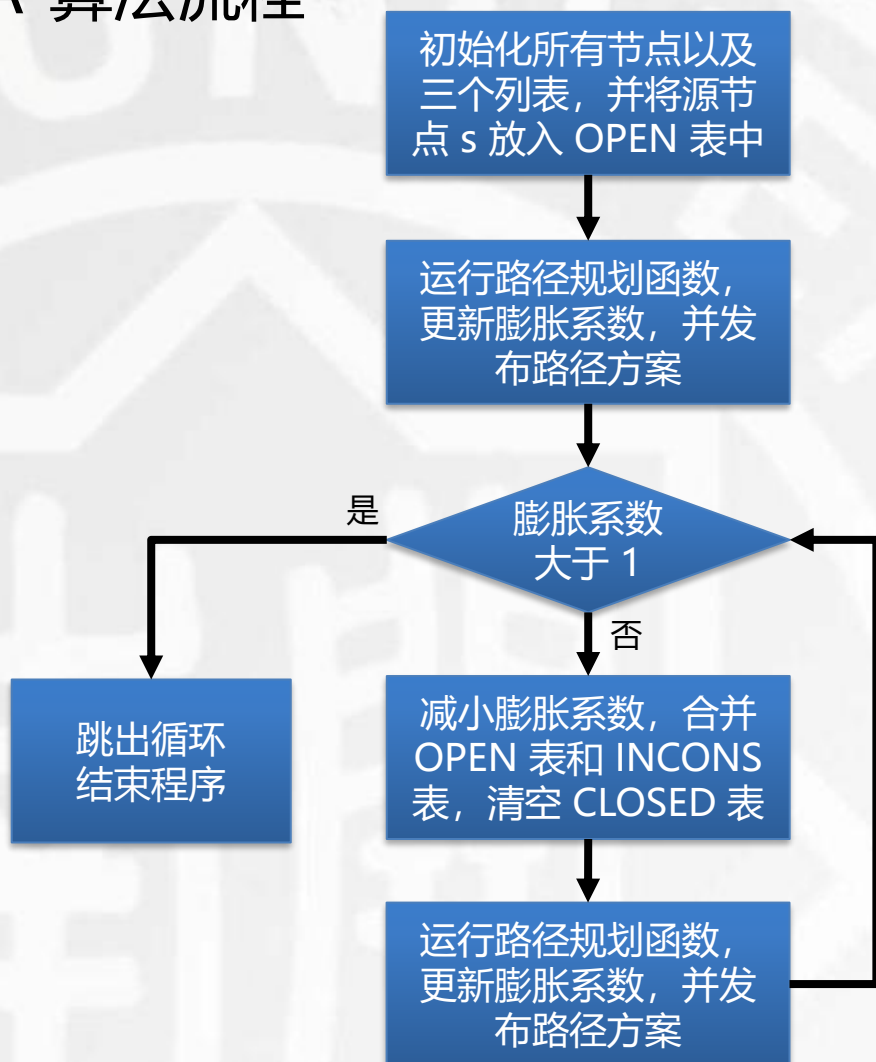
2.2 图搜索方法 (6)

ARA*算法:

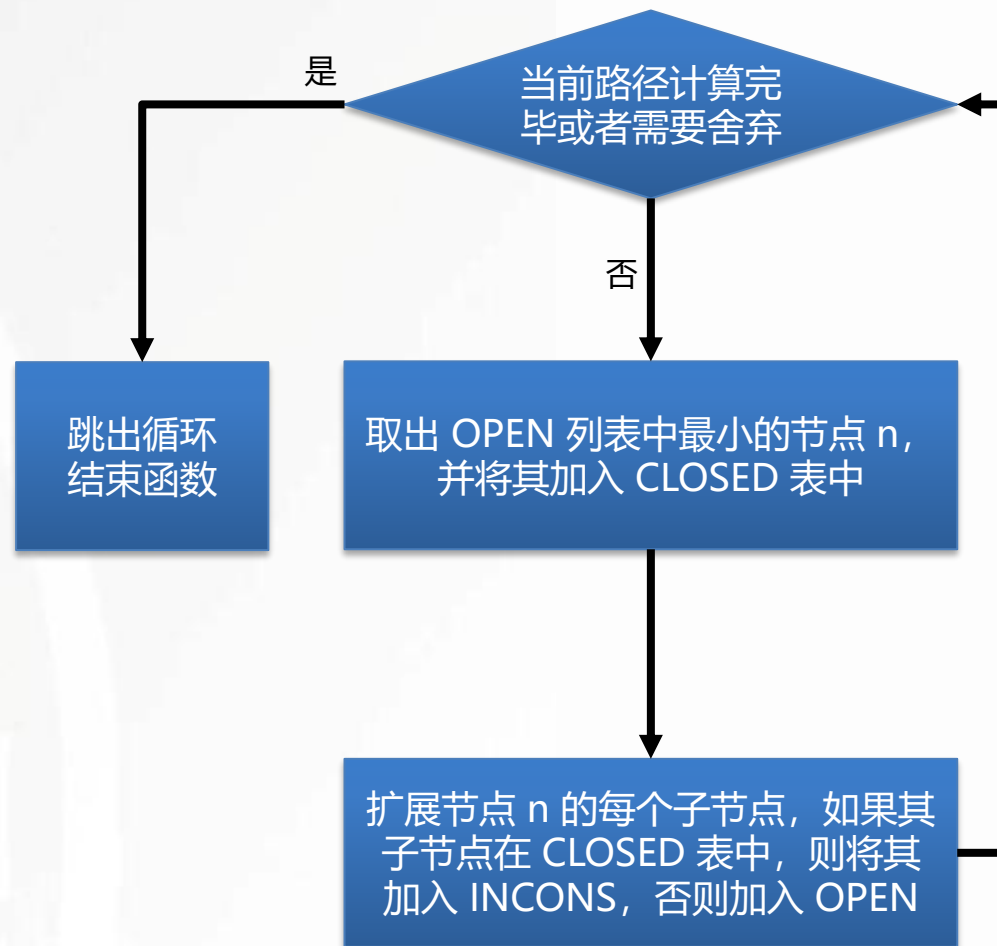
- 权重A*算法通过加大启发函数的权重（膨胀系数 ϵ ），提升算法效率
 - 不满足A*算法中的可采纳性(admissible)
 - 只能得到局部最优解，无法获得全局最优解
- A*可以得到全局最优解，但算法效率低，很难保证实时性
- ARA*算法：任意时间法（Anytime algorithm） + 权重A*（Weighted A*） 兼顾效率与性能
- 任意时间法（Anytime algorithm）：
 - 由Shlomo Zilberstein以及Stuart Russell提出
 - 普适性算法：一种策略、思路
 - 在很短的时间内先得出一个次优的解法，然后再对这个解法进行不断的优化。
 - ✓ 膨胀系数 ϵ 先取一个较大值，提升算法速度，然后不断降低，直至为1，或到达给定时间
- 如何降低迭代过程中的重复计算：
 - 引入INCONS表：防止已经进入CLOSED表的节点不再进入OPEN表中进行二次扩展

2.2 图搜索方法 (6)

ARA*算法流程



ARA*算法主函数流程图



ARA*算法路径规划部分流程图

2.2 图搜索方法 (6)

ARA*算法流程

procedure fvalue(s)

01 return $g(s) + \epsilon * h(s)$;

procedure ImprovePath()

02 while(fvalue(s_{goal}) > $\min_{s \in OPEN}(fvalue(s))$)

03 remove s with the smallest fvalue(s) from $OPEN$;

04 $CLOSED = CLOSED \cup \{s\}$;

05 for each successor s' of s

06 if s' was not visited before then

07 $g(s') = \infty$;

08 if $g(s') > g(s) + c(s, s')$

09 $g(s') = g(s) + c(s, s')$;

10 if $s' \notin CLOSED$

11 insert s' into $OPEN$ with fvalue(s');

12 else

13 insert s' into $INCONS$;

procedure Main()

01' $g(s_{goal}) = \infty$; $g(s_{start}) = 0$;

02' $OPEN = CLOSED = INCONS = \emptyset$;

03' insert s_{start} into $OPEN$ with fvalue(s_{start});

04' ImprovePath();

05' $\epsilon' = \min(\epsilon, g(s_{goal}) / \min_{s \in OPEN \cup INCONS}(g(s) + h(s)))$;

06' publish current ϵ' -suboptimal solution;

07' while $\epsilon' > 1$

08' decrease ϵ ;

09' Move states from $INCONS$ into $OPEN$;

10' Update the priorities for all $s \in OPEN$ according to fvalue(s);

11' $CLOSED = \emptyset$;

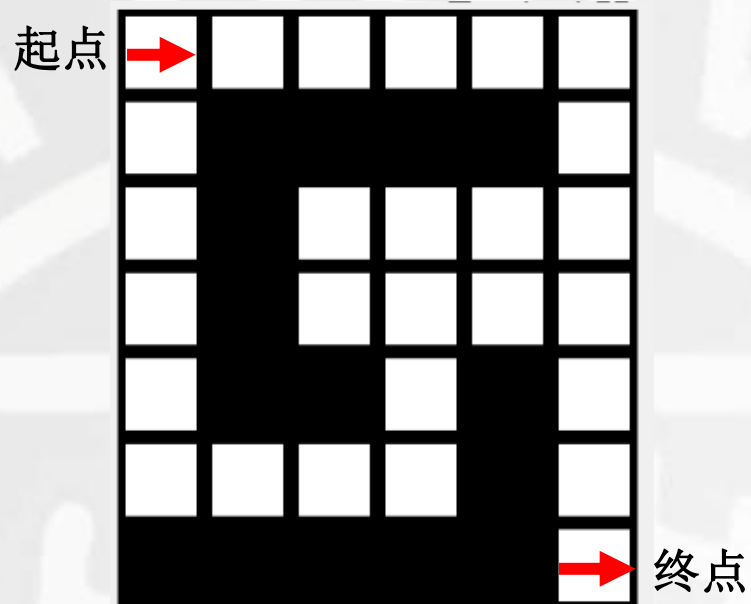
12' ImprovePath();

13' $\epsilon' = \min(\epsilon, g(s_{goal}) / \min_{s \in OPEN \cup INCONS}(g(s) + h(s)))$;

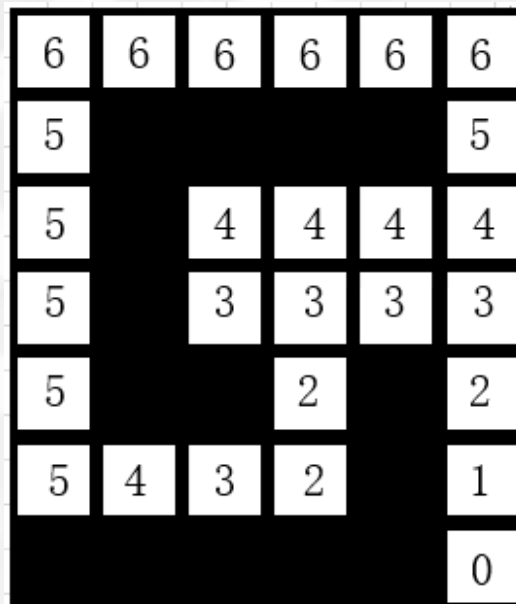
14' publish current ϵ' -suboptimal solution;

2.2 图搜索方法 (6)

ARA*算法示例

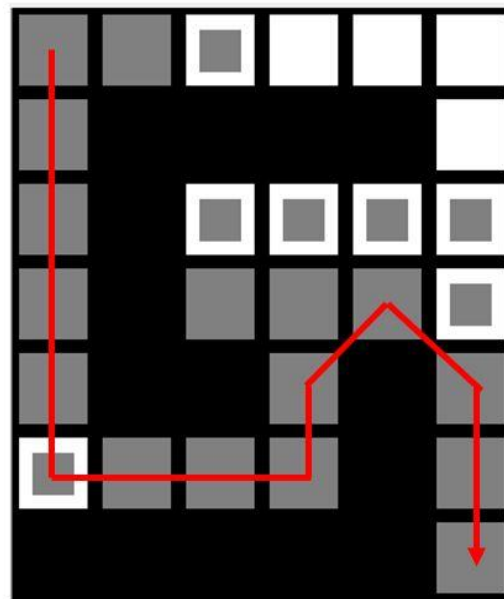


H采用切比雪夫距离

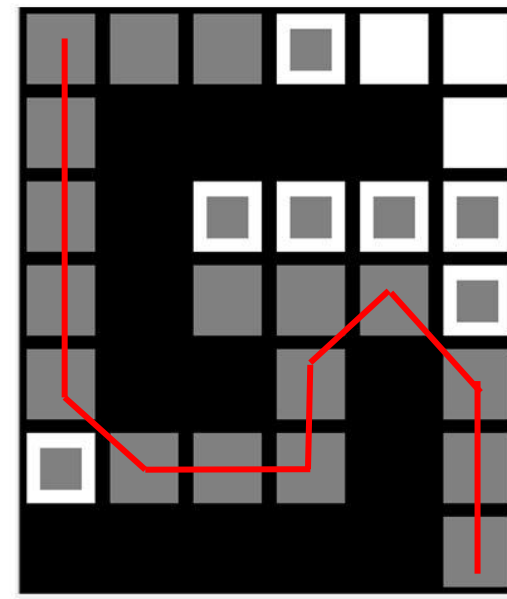


ARA*算法:

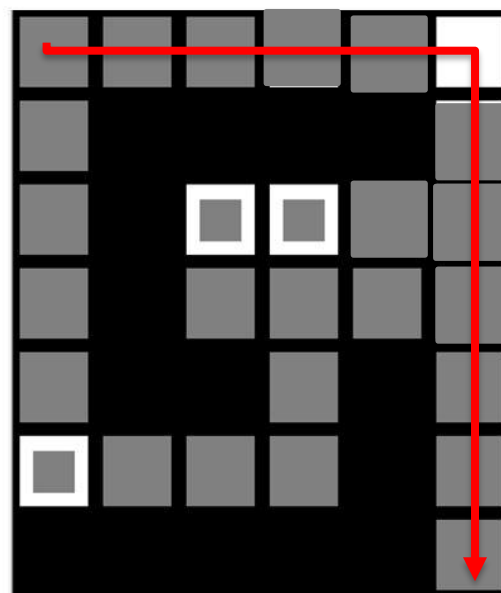
- 相比于A*算法, 二者得出的最优路径是相同的
- ARA*算法得到最优路径和A*算法的时间几乎没有损失
- ARA*算法能更好地适用于对时间有苛刻限制的情况
- 与A*算法一样, 仅限于静态环境下使用



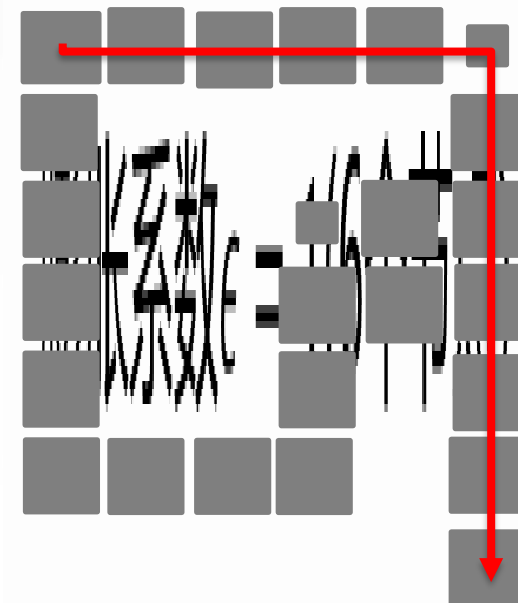
膨胀系数 $\epsilon = 2.5$ (16个节点)



膨胀系数 $\epsilon = 1.5$ (1个节点)



膨胀系数 $\epsilon = 1$ (6个节点)



A*算法23个节点

03

基于图搜索的动态路径规划

Graph Search-Based Dynamic Path Planning

3.1 动态路径规划算法

■ 动态环境 VS. 静态环境：

- 机器人实际运行过程当中，环境地图可能会改变。
- 多个机器人同时运行时，其他机器人——动态障碍物
- 动态环境是一般性环境，更符合实际

■ 动态路径规划算法

- 如何进一步降低算法的复杂度，提高算法的实时性
 - ✓ 避免地图变化后重复计算
 - ✓ 降低地图变化后拓展节点的数量

3.2 D*算法

■ 什么是D*算法

- D*算法又叫动态A*算法 (Dynamic A*)
- 用于**未知环境**或**动态环境**的路径规划算法。
- 将**目标点作为规划的起点**，将机器人**当前位置作为规划的终点**
- 基于**Dijkstra算法**进行路径搜索
- D*算法可以通过利用变化前的地图中的已知信息进行动态规划，从而大大提高了算法的计算效率。

3.2.1 D*算法的相关概念定义

■什么是D*算法

- $b(X) = Y$: 后向指针, 意思是 X 的下一个节点是 Y
- $c(X, Y)$: Y 到 X 的路径的代价
- $t(X)$: 节点标签, 用以区分OPEN, CLOSED, NEW
- $o(G, X)$: X 到目标点 G 的**实际**最短路径代价
- $h(G, X)$: X 到目标点 G 的路径代价**估计, 可变**。
- $k(G, X)$: OPEN表中节点 X 的 h 函数的值 (**历史值**) ——D*算法的核心
 - ✓ LOWER: $k(G, X) \geq h(G, X)$, 用以传播代价的减小
 - ✓ RAISE: $k(G, X) < h(G, X)$, 用以传播代价的增加

3.2.2 D*算法的算法描述

- PROCESS-STATE + MODIFY-COST两个函数构成算法运行的主体
 - PS函数是算法的主体运行部分,
 - MC函数是用来检测环境变化并修改地图的函数
- GET-KMIN(): 用以返回OPEN中的所有节点最小的 k_{min}
- MIN-STATE(): 用以返回OPEN表中具有最小值 k_{min} 所对应的节点
- DELETE(): 用以移除OPEN表中的节点并将其tag置为CLOSED
- INSERT(X, h_{new}):用以给节点 X 的 h 函数和 k 函数赋值。
 - 当 X 是NEW时, 令 $k = h_{new}$
 - 当 X 是OPEN时, 令 $k = \min\{h, h_{new}\}$
 - 当 X 是CLOSED时, 令 $k = \min\{h, h_{new}\}, h = h_{new}, t(X) = OPEN$

3.2.2 D*算法流程

■ PROCESS-STATE+MODIFY-COST两个函数构成算法运行的主体

□ PS函数是算法的主体运行部分

- ✓ $k(G, X) < h(G, X)$, 用以传播代价的增加, 拓展节点, 缩短距离
- ✓ $k(G, X) = h(G, X)$, 用以传播代价的不变, 拓展节点, 保存信息
- ✓ $k(G, X) > h(G, X)$, 用以传播代价的减小, 拓展节点, 找到最优

□ MC函数是用来检测环境变化并修改地图的函数

■ GET-KMIN(): 返回OPEN中的所有节点中最小的K值

■ MIN-STATE(): 返回OPEN表中 k_{min} 所对应的节点

■ DELETE(): 移除OPEN表中的节点并将其tag置为CLOSED

■ INSERT(X, h_{new}):用以给节点X的h函数和k函数赋值。

□ 当X是NEW时, 令 $k = h_{new}$

□ 当X是OPEN时, 令 $k = \min\{h, h_{new}\}$

□ 当X是CLOSED时, 令 $k = \min\{h, h_{new}\}, h = h_{new}, t(X) = OPEN$

Function: PROCESS-STATE ()

```
L1   $X = MIN-STATE()$ 
L2  if  $X = NULL$  then return -1
L3   $k_{old} = GET-KMIN(); DELETE(X)$ 
L4  if  $k_{old} < h(X)$  then
L5    for each neighbor  $Y$  of  $X$ :
L6      if  $h(Y) \leq k_{old}$  and  $h(X) > h(Y) + c(Y, X)$  then
L7         $b(X) = Y; h(X) = h(Y) + c(Y, X)$ 
L8  if  $k_{old} = h(X)$  then
L9    for each neighbor  $Y$  of  $X$ :
L10   if  $t(Y) = NEW$  or
L11     ( $b(Y) = X$  and  $h(Y) \neq h(X) + c(X, Y)$ ) or
L12     ( $b(Y) \neq X$  and  $h(Y) > h(X) + c(X, Y)$ ) then
L13      $b(Y) = X; INSERT(Y, h(X) + c(X, Y))$ 
L14 else
L15   for each neighbor  $Y$  of  $X$ :
L16     if  $t(Y) = NEW$  or
L17       ( $b(Y) = X$  and  $h(Y) \neq h(X) + c(X, Y)$ ) then
L18        $b(Y) = X; INSERT(Y, h(X) + c(X, Y))$ 
L19     else
L20       if  $b(Y) \neq X$  and  $h(Y) > h(X) + c(X, Y)$  then
L21          $INSERT(X, h(X))$ 
L22       else
L23         if  $b(Y) \neq X$  and  $h(X) > h(Y) + c(Y, X)$  and
L24            $t(Y) = CLOSED$  and  $h(Y) > k_{old}$  then
L25            $INSERT(Y, h(Y))$ 
L26 return  $GET-KMIN()$ 
```

3.2.2 D*算法流程

■ MODIFY-COST函数

□ 检测环境变化并修改地图的函数

Function: MODIFY-COST($X, Y, cval$)

L1 $c(X, Y) = cval$

L2 if $t(X) = CLOSED$ then $INSERT(X, h(X))$

L3 return $GET - KMIN()$

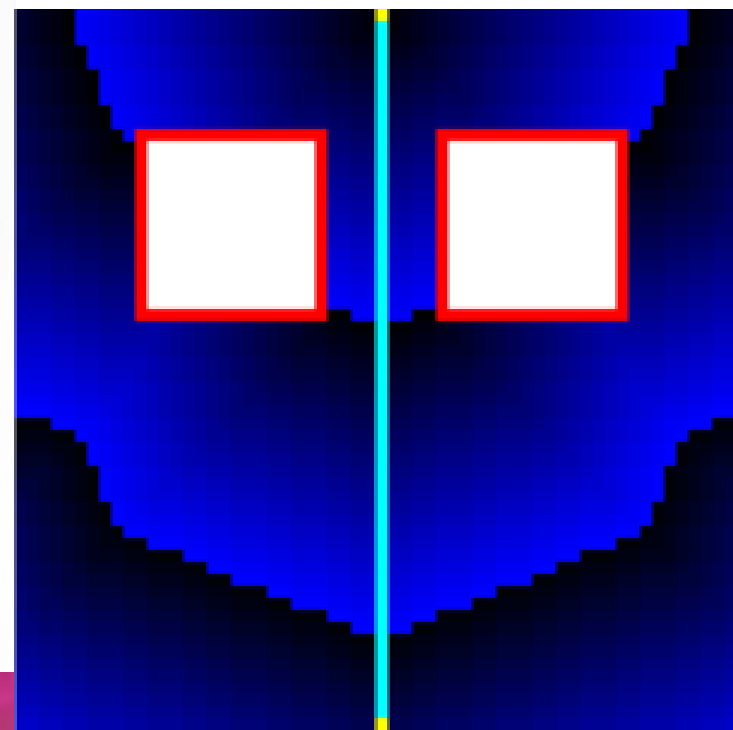
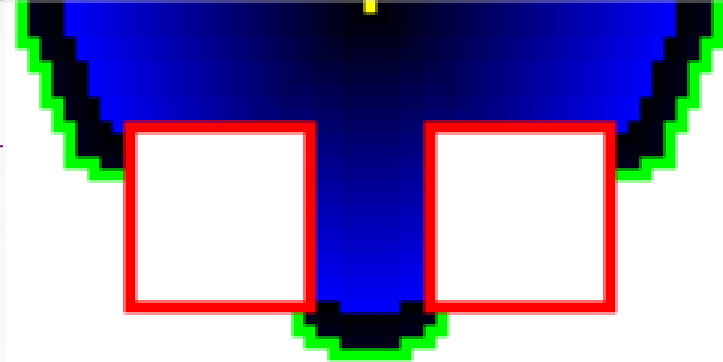
3.2.3 D*算法思路

- 算法起始，初始化地图上的所有节点的tag为NEW，k函数和h函数为0；
- 将目标点G放入OPEN表中，从目标节点开始进行路径规划。直到机器人当前所在的节点X被从OPEN列表中移除，规划完毕。
- 机器人沿着当前已经规划好的路径序列前进，直到到达目标节点或者检测到地图和实际环境不匹配。
- 当检测到地图和环境有出入的时候，调用MC函数，修改地图上的路径代价信息，并通过OPEN表将地图变化的信息传播出去，直到机器人所在的当前节点的路径调整完毕后，机器人沿着新的路径继续前进，直到到达终点。
- 算法一开始的路径规划是迪杰斯特拉算法，并且是从终点开始的迪杰斯特拉算法。当第一次迪杰斯特拉算法过后，保留原始的节点状态，机器人开始运行，并在检查到路径代价错误时自动调整地图。

3.2.3 D*算法思路

扩张:

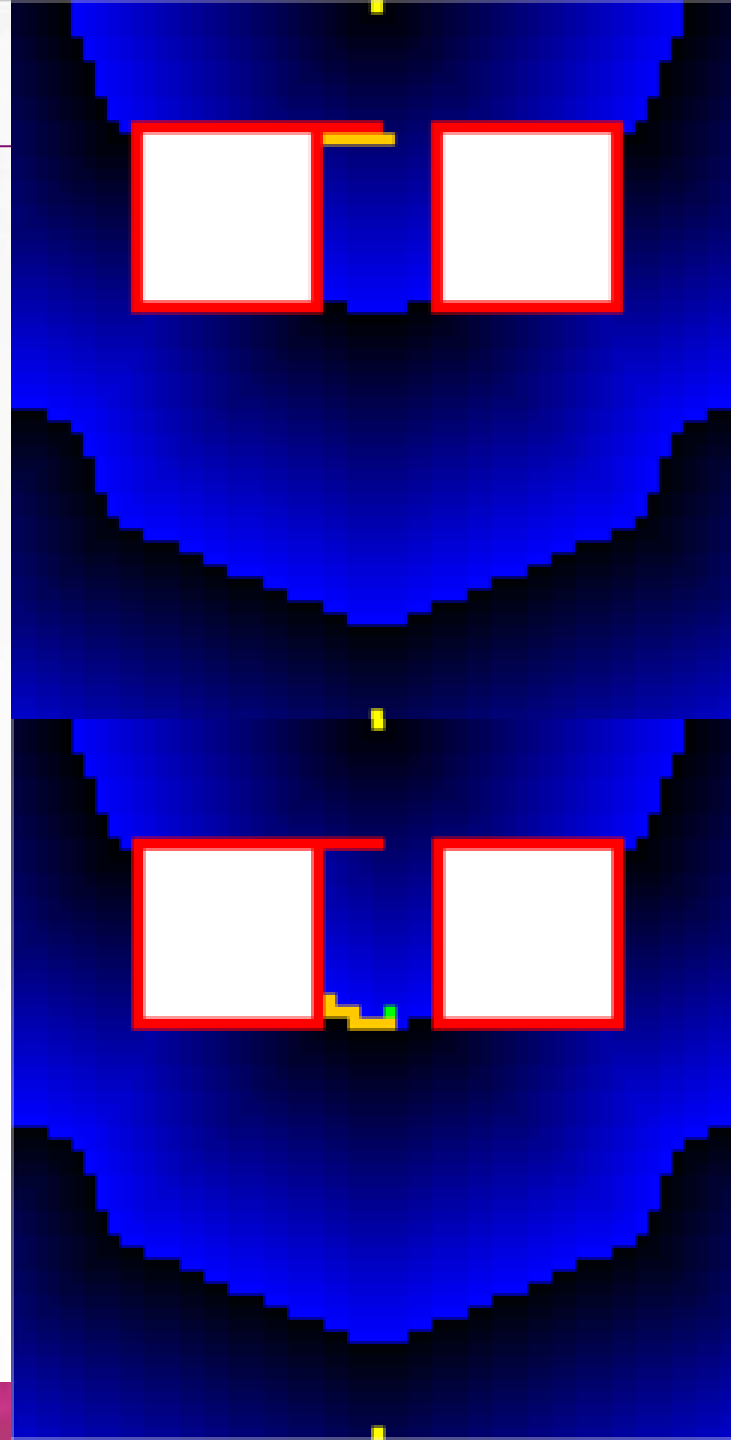
- 从目标点开始，利用迪杰斯特拉算法（宽度优先）扩张
- 每个扩张节点都有一个反向指针，它指向指向目标的下一个节点，每个节点都知道目标的确切成本
- 当开始节点是下一个要展开的节点时，算法完成
- 只需遵循反向指针就可以找到目标的路径



3.2.3 D*算法思路

动态障碍物处理：

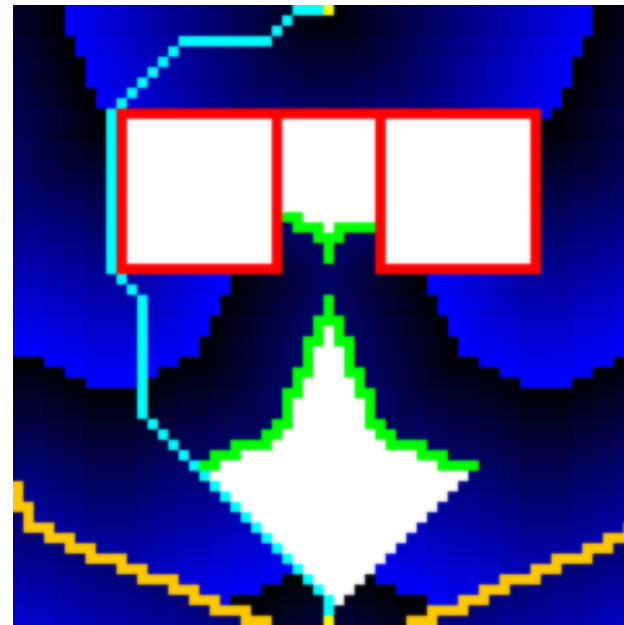
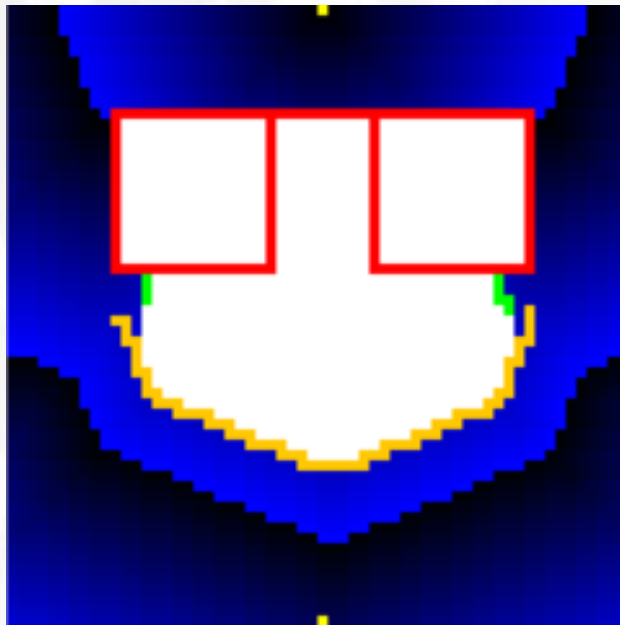
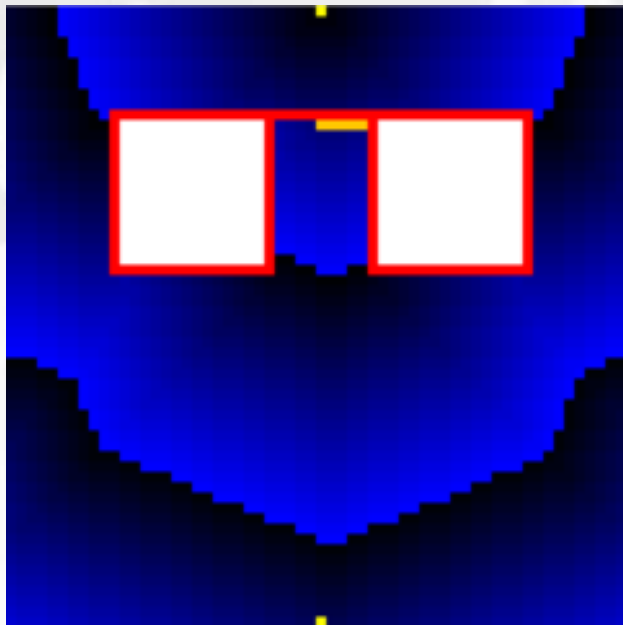
- 当在指定的路径上检测到障碍物时，所有受影响的点将再次被放到OPEN列表中，标记为RAISE（节点成本增加）。
- 利用记录的指针，重新评估后继节点，更新节点成本，传递RAISE状态，形成波。
- 当RAISED节点可以减少时，其的反向指针（backpointer）会更新，并将LOWER状态传递给它的邻居。



3.2.3 D*算法思路

死锁情况处理：

- 没有一个点可以通过邻居找到一条新的路线到达目的地。因此，继续传播节点的成本增加。只有在通道外才能找到点，这些点可以通过可行的路线到达目的地。这就是两个较低的波是如何发展的，它们扩张成具有新路线信息的不可到达的标记点。



3.3 Focussed D*算法

■ Focussed D*算法:

- Focussed D*算法是D*算法的改进版，是由原作者Anthony Stentz自己改进的一个算法。
- 在原D*算法中，每次路径规划采用的是**迪杰斯特拉算法**，导致计算效率上有所损失，Focussed D*用了启发函数**A*算法**，从而限定了每次函数遍历的方向，能更快地找到机器人所在的当前位置。

3.4 LPA*算法

■ LPA*算法（全称Lifelong Planning A*算法）

- 是一种参考人工智能的**增量式搜索**进行改进的A*算法
- 它被用来处理动态环境下从给定起始点到给定目标点的最短路径问题

■ 基本概念：

- $succ(s)$: s 节点的后继节点。
- $pred(s)$: s 节点的前置节点。
- $c(s, s_0)$: 边缘代价函数，从 s 节点到 s_0 节点的路径代价
- $g^*(s)$: 节点 s 到起始点的实际最短距离(对应于A*算法里的 $g(n)$)

$$g^*(s) = \begin{cases} 0, & x = Start \\ \min_{s' \in pred(s)} (c(s', s) + g^*(s')), & otherwise \end{cases}$$

- $rhs(s)$: 其中 $g(s)$ 是对 $g^*(s)$ 的估计

$$rhs^*(s) = \begin{cases} 0, & x = Start \\ \min_{s' \in pred(s)} (c(s', s) + g(s')), & otherwise \end{cases}$$

3.4 LPA*算法

$$g^*(s) = \begin{cases} 0, & x = Start \\ \min_{s' \in pred(s)} (c(s', s) + g^*(s')), & otherwise \end{cases}$$
$$rhs^*(s) = \begin{cases} 0, & x = Start \\ \min_{s' \in pred(s)} (c(s', s) + g(s')), & otherwise \end{cases}$$

- 局部一致(locally consistent): $g(s) = rhs(s)$
- 如果所有的节点都是局部一致, 则所有节点的g函数值都等于从起始点到当前点的路径代价, 即当所有节点的 $g(s) = rhs(s)$ 时, $g^*(s) = g(s)$ 。
- 局部不一致(locally inconsistent): $g(s) \neq rhs(s)$
- 过一致(Overconsistent): $g(s) > rhs(s)$
 - ✓ 边缘代价函数值 $c(s', s)$ 变低, 代表网格上障碍物被清除或搜索到一条更短的“捷径”
- 欠一致(Subconsistent): $g(s) < rhs(s)$
 - ✓ 边缘代价函数c值变高, 代表着自由网格被障碍物所占据, 这时节点点的信息需要被重置, 这时候就需要重新搜索计算最短路径。

3.4 LPA*算法

- $key(s) = [k_1(s); k_2(s)]$: 代价函数, 类似于A*算法中的F函数

$$k_1(s) = \min[g(s), rhs(s)] + h(s),$$

$$k_2(s) = \min[g(s), rhs(s)]$$

- 路径搜索时与A*算法类似, 取 $key(s)$ 值小的进行搜索

- $key(s)$ 大小的比较:

if $k_1(s) \leq k_1(s')$ or $(k_1(s) = k_1(s') \text{ and } k_2(s) \leq k_2(s'))$ then

$$key(s) \leq key(s')$$

procedure CalculateKey(s)

{01} return $[\min(g(s), rhs(s)) + h(s); \min(g(s), rhs(s))];$

procedure Initialize()

{02} $U = \emptyset;$

{03} for all $s \in S$ $rhs(s) = g(s) = \infty;$

{04} $rhs(s_{start}) = 0;$

{05} $U.Insert(s_{start}, [h(s_{start}); 0]);$

procedure UpdateVertex(u)

{06} if $(u \neq s_{start})$ $rhs(u) = \min_{s' \in pred(u)} (g(s') + c(s', u));$

{07} if $(u \in U)$ $U.Remove(u);$

{08} if $(g(u) \neq rhs(u))$ $U.Insert(u, CalculateKey(u));$

procedure ComputeShortestPath()

{09} while $(U.TopKey() < CalculateKey(s_{goal}) \text{ OR } rhs(s_{goal}) \neq g(s_{goal}))$

{10} $u = U.Pop();$

{11} if $(g(u) > rhs(u))$

{12} $g(u) = rhs(u);$

{13} for all $s \in succ(u)$ $UpdateVertex(s);$

{14} else

{15} $g(u) = \infty;$

{16} for all $s \in succ(u) \cup \{u\}$ $UpdateVertex(s);$

procedure Main()

{17} $Initialize();$

{18} forever

{19} $ComputeShortestPath();$

{20} Wait for changes in edge costs;

{21} for all directed edges (u, v) with changed edge costs

{22} Update the edge cost $c(u, v);$

{23} $UpdateVertex(v);$

3.4 LPA*算法

- 第一次搜索（类似于A*）：从A3出发，到达F0

Start distances / heuristics

	0	1	2	3
A	3/5	2/5	1/5	0/5
B		2/4		1/4
C		3/3		2/3
D		4/2		3/3
E		5/1		4/3
F	6/0	6/1	5/2	5/3

Iteration #1

	0	1	2	3
A	∞	∞	∞	∞ [5;0]
B		∞		∞
C		∞		∞
D		∞		∞
E		∞		∞
F	∞	∞	∞	∞

Iteration #2

	0	1	2	3
A	∞	∞	∞ [6;1]	0
B		∞		∞ [5;1]
C		∞		∞
D		∞		∞
E		∞		∞
F	∞	∞	∞	∞

<https://blog.csdn.net/a380331382>

3.4 LPA*算法

- 第一次搜索（类似于A*）：从A3出发，到达F0

Iteration #3

	0	1	2	3
A	∞	∞	∞ [6;1]	0
B		∞		1
C		∞		∞ [5;2]
D		∞		∞
E		∞		∞
F	∞	∞	∞	∞

Iteration #4

	0	1	2	3
A	∞	∞	∞ [6;1]	0
B		∞		1
C		∞		2
D		∞		∞ [6;3]
E		∞		∞
F	∞	∞	∞	∞

Iteration #5

	0	1	2	3
A	∞	∞ [7;2]	1	0
B		∞ [6;2]		1
C		∞		2
D		∞		∞ [6;3]
E		∞		∞
F	∞	∞	∞	∞

<https://blog.csdn.net/a350331382>

3.4 LPA*算法

- 第一次搜索（类似于A*）：从A3出发，到达F0

Iteration #6				
	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		∞ [6;3]		2
D		∞		∞ [6;3]
E		∞		∞
F	∞	∞	∞	∞

Iteration #7				
	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D		∞ [6;4]		∞ [6;3]
E		∞		∞
F	∞	∞	∞	∞

Iteration #8				
	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D		∞ [6;4]		3
E		∞		∞ [7;4]
F	∞	∞	∞	∞

3.4 LPA*算法

- 第一次搜索（类似于A*）：从A3出发，到达F0

Iteration #9

	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D		4		3
E		∞ [6;5]		∞ [7;4]
F	∞	∞	∞	∞

Iteration #10

	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D		4		3
E		5		∞ [7;4]
F	∞ [6;6]	∞ [7;6]	∞ [8;6]	∞

Shortest path

	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D		4		3
E		5		∞ [7;4]
F	6	∞ [7;6]	∞ [8;6]	∞

3.4 LPA*算法

- 第二次搜索（地图发生变化）首先从边缘代价函数发生变化的节点进行更新，并将其后代节点置于优先队列中，该节点的变动对父辈节点的状态并无影响。

Iteration #1

	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D				3
E		5 [6;5]		∞ [7;4]
F	6	∞ [7;6]	∞ [8;6]	∞

Iteration #2

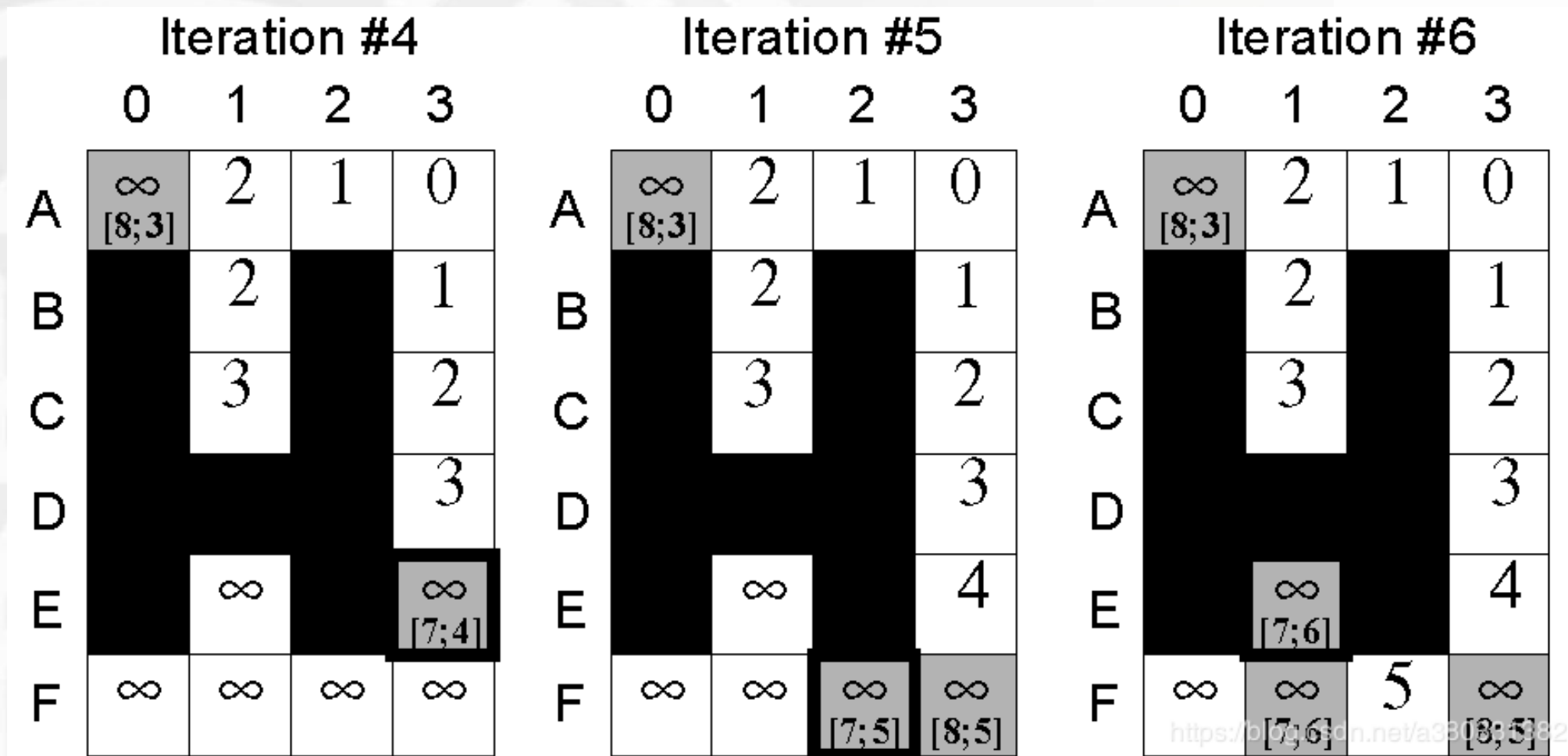
	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D				3
E		∞ [8;7]		∞ [7;4]
F	6 [6;6]	∞ [8;7]	∞	∞

Iteration #3

	0	1	2	3
A	∞ [8;3]	∞ [7;2]	1	0
B		2		1
C		3		2
D				3
E		∞		∞ [7;4]
F	∞	∞	∞	∞

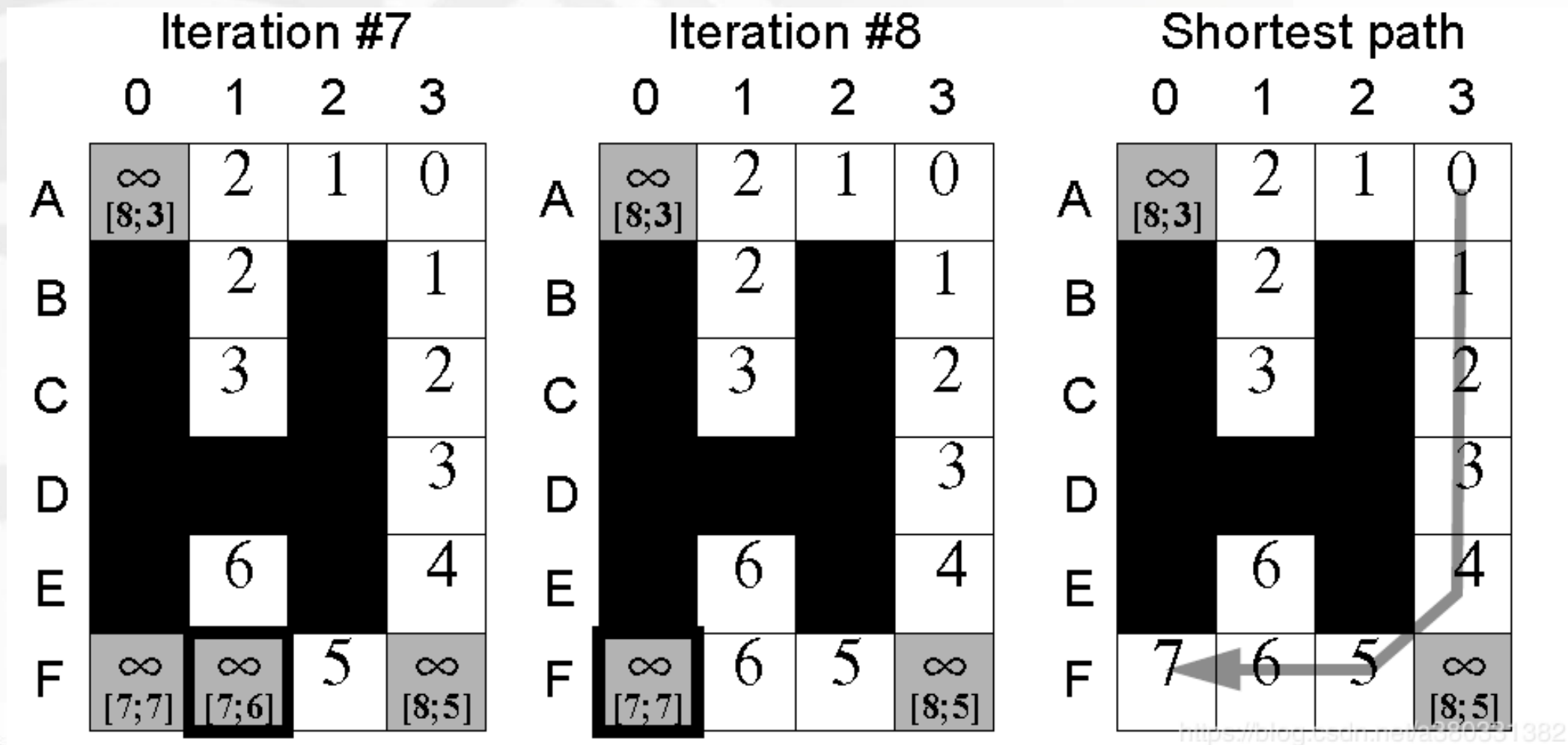
3.4 LPA*算法

■ 第二次搜索（地图发生变化）



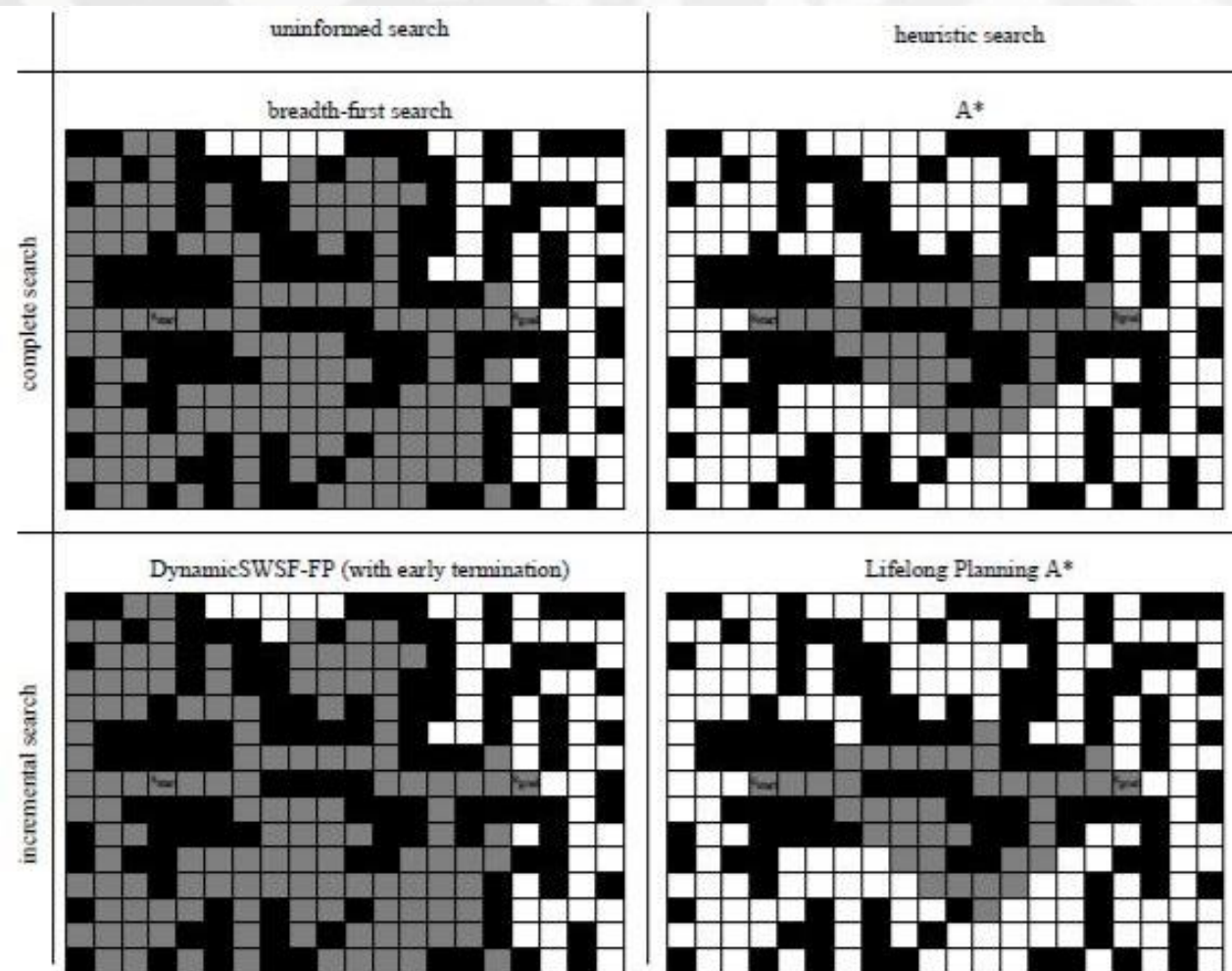
3.4 LPA*算法

■ 第二次搜索（地图发生变化）

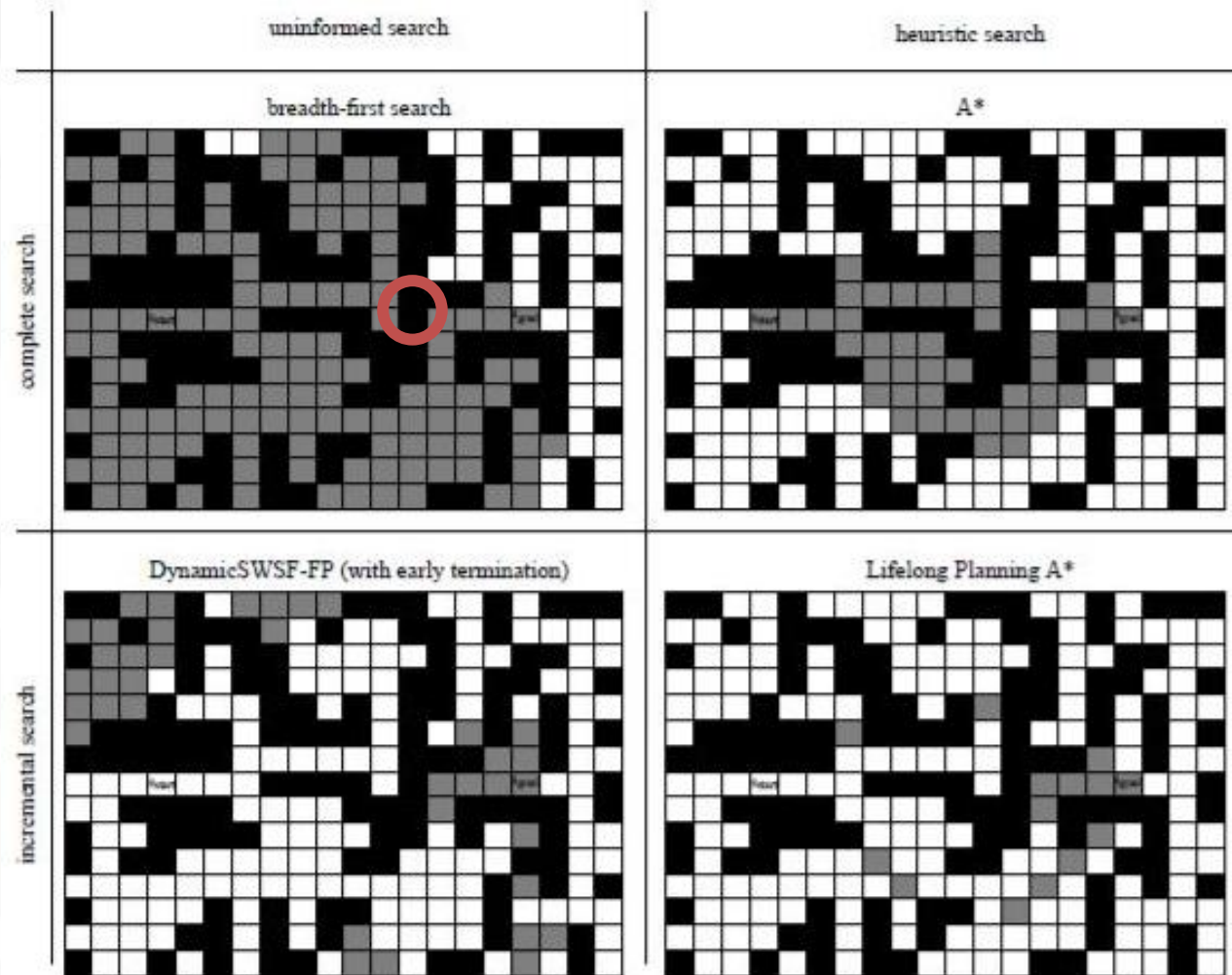


3.4 LPA*算法实验对比

静态地图



动态地图



3.4 LPA*算法总结

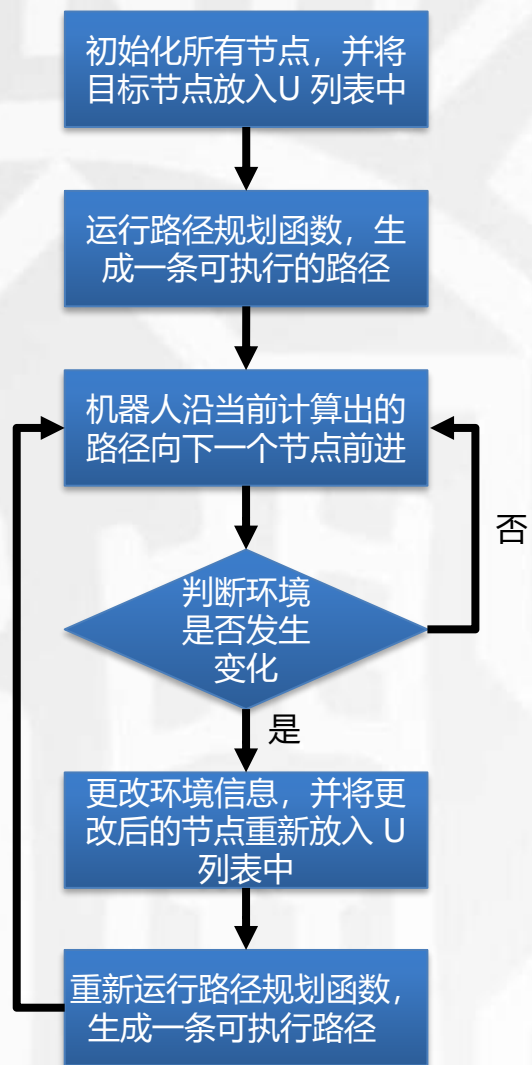
LPA*提出了一种解决**动态环境**下的最短路径搜索方法，但是它针对的是**起始点和目标点固定**的情况，如果在机器人按照搜索到的最短路径行走过程中，环境中某些节点发生变化，这时如果想获得新的路径，就得以机器人当前节点为起始点，重新用LPA*算法解算一次，这效率是很低的。

3.5 D*-Lite算法

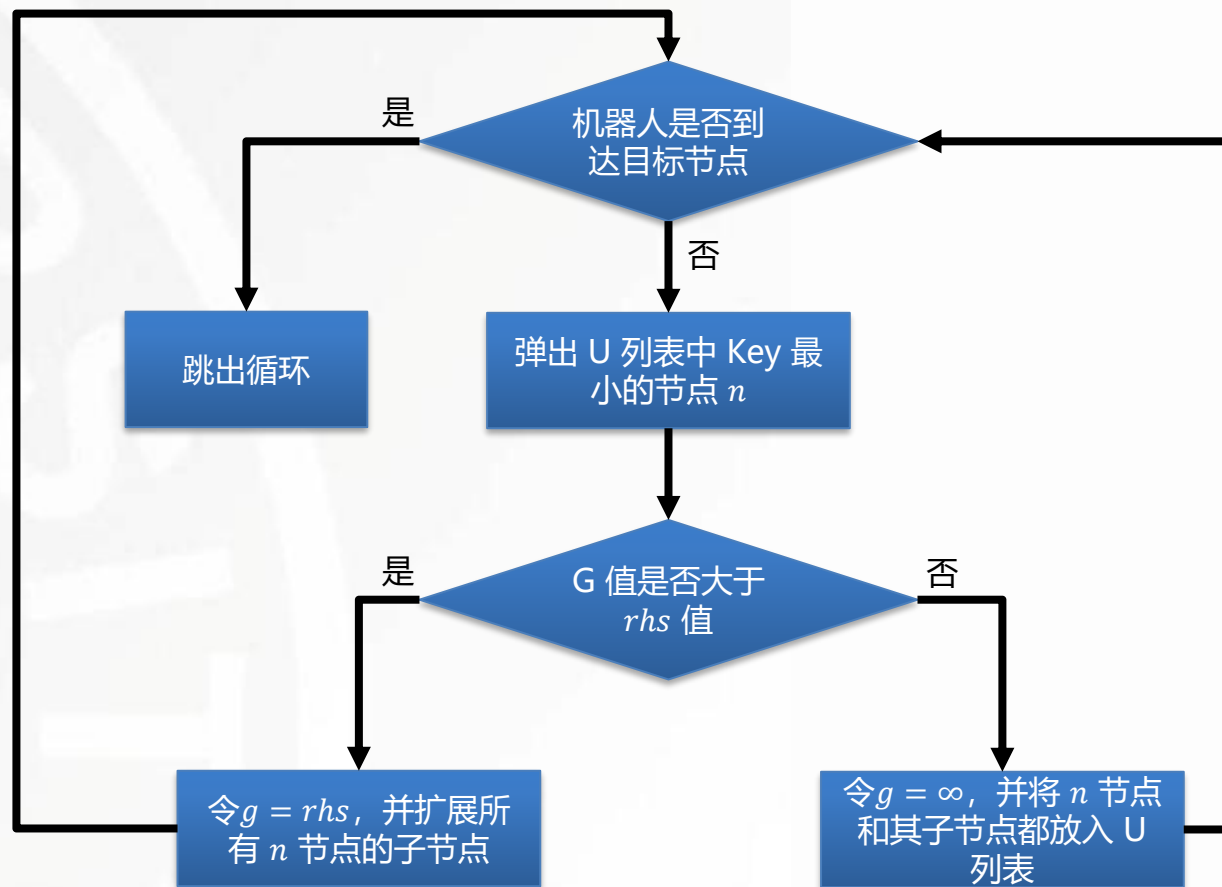
- D*-lite算法是LPA*算法的动态形式，即将目标点作为规划起点，将机器人当前位置作为规划终点
- LPA*算法只能适用于给定两点的动态地图导航，而D*-lite算法则可直接用于动态机器人的导航
- D*-lite算法从算法逻辑上并不是D*算法的改进版本，它被作者称为D* like 算法。相较于D*算法，代码量更小，逻辑更容易理解，更易于学习，且在实验中，D*-lite算法的效率至少和D*算法的效率是一样的。

3.5 D*-Lite算法流程

■ 主函数:



■ 路径规划函数:



3.5 D*-Lite算法流程

```
procedure CalcKey(s)
{01'} return [min(g(s), rhs(s)) + h(s_start, s); min(g(s), rhs(s))];

procedure Initialize()
{02'} U = ∅;
{03'} for all s ∈ S rhs(s) = g(s) = ∞;
{04'} rhs(s_goal) = 0;
{05'} U.Insert(s_goal, CalcKey(s_goal));

procedure UpdateVertex(u)
{06'} if (u ≠ s_goal) rhs(u) = min_{s' ∈ Succ(u)} (c(u, s') + g(s'));
{07'} if (u ∈ U) U.Remove(u);
{08'} if (g(u) ≠ rhs(u)) U.Insert(u, CalcKey(u));

procedure ComputeShortestPath()
{09'} while (U.TopKey() < CalcKey(s_start) OR rhs(s_start) ≠ g(s_start))
{10'}   u = U.Pop();
{11'}   if (g(u) > rhs(u))
{12'}     g(u) = rhs(u);
{13'}     for all s ∈ Pred(u) UpdateVertex(s);
{14'}   else
{15'}     g(u) = ∞;
{16'}     for all s ∈ Pred(u) ∪ {u} UpdateVertex(s);
```

与LPA*相同

D*-Lite

```
procedure Main()
{17'} Initialize();
{18'} ComputeShortestPath();
{19'} while (s_start ≠ s_goal)
{20'}   /* if (g(s_start) = ∞) then there is no known path */
{21'}   s_start = arg min_{s' ∈ Succ(s_start)} (c(s_start, s') + g(s'));
{22'}   Move to s_start;
{23'}   Scan graph for changed edge costs;
{24'}   if any edge costs changed
{25'}     for all directed edges (u, v) with changed edge costs
{26'}       Update the edge cost c(u, v);
{27'}       UpdateVertex(u);
{28'}     for all s ∈ U
{29'}       U.Update(s, CalcKey(s));
{30'}   ComputeShortestPath();
```

由于每一次地图中路径变化都会导致h函数的变化，算法不得不对U列表中的节点重新进行排序，这样会浪费很多计算时间。

```
procedure Main()
{17} Initialize();
{18} forever
{19}   ComputeShortestPath();
{20}   Wait for changes in edge costs;
{21}   for all directed edges (u, v) with changed edge costs
{22}     Update the edge cost c(u, v);
{23}     UpdateVertex(v);
```

LPA*

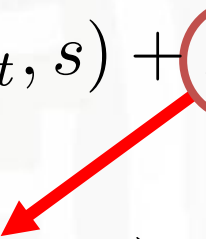
3.5 D*-Lite算法第二种版本

- D*-lite算法第二种版本可以省去每次环境变化后的重排列表，但必须首先满足以下条件：

$$h(s, s'') \leq h(s, s') + h(s', s'')$$

- 在满足条件的基础上更改算法的key函数：

$$k(s) = [\min(g(s), rhs(s)) + h(s_{start}, s) + k_m \quad \min(g(s), rhs(s))]$$

$$k_m = k_{m0} + h(s_{last}, s_{start})$$


3.5 D*-Lite算法第二种版本

```
procedure CalcKey(s)
{01"} return  $[\min(g(s), rhs(s)) + h(s_{start}, s) + k_m \min(g(s), rhs(s))];$ 
```

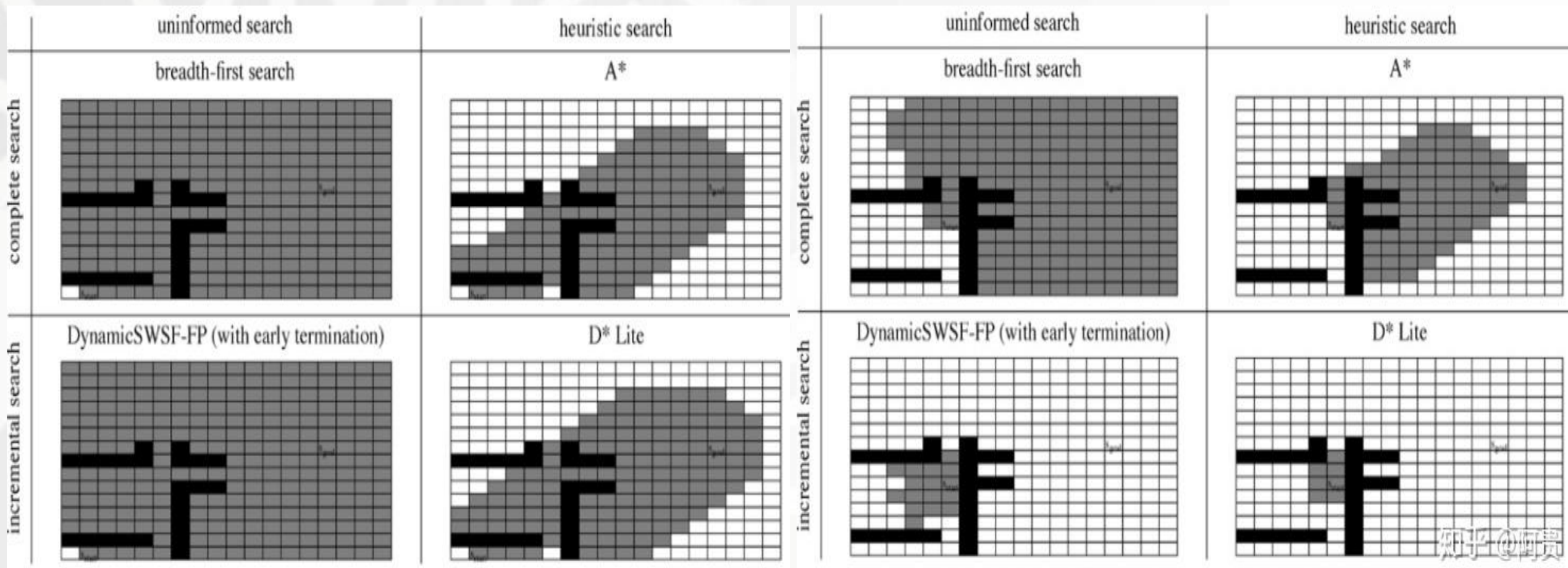
```
procedure Initialize()
{02"}  $U = \emptyset;$ 
{03"}  $k_m = 0;$ 
{04"} for all  $s \in S$   $rhs(s) = g(s) = \infty;$ 
{05"}  $rhs(s_{goal}) = 0;$ 
{06"}  $U.Insert(s_{goal}, CalcKey(s_{goal}));$ 
```

```
procedure UpdateVertex(u)
{07"} if ( $u \neq s_{goal}$ )  $rhs(u) = \min_{s' \in Succ(u)} (c(u, s') + g(s'));$ 
{08"} if ( $u \in U$ )  $U.Remove(u);$ 
{09"} if ( $g(u) \neq rhs(u)$ )  $U.Insert(u, CalcKey(u));$ 
```

```
procedure ComputeShortestPath()
{10"} while ( $U.TopKey() < CalcKey(s_{start})$  OR  $rhs(s_{start}) \neq g(s_{start})$ )
{11"}  $k_{old} = U.TopKey();$ 
{12"}  $u = U.Pop();$ 
{13"} if ( $k_{old} < CalcKey(u)$ )
{14"}  $U.Insert(u, CalcKey(u));$ 
{15"} else if ( $g(u) > rhs(u)$ )
{16"}  $g(u) = rhs(u);$ 
{17"} for all  $s \in Pred(u)$   $UpdateVertex(s);$ 
{18"} else
{19"}  $g(u) = \infty;$ 
{20"} for all  $s \in Pred(u) \cup \{u\}$   $UpdateVertex(s);$ 
```

```
procedure Main()
{21"}  $s_{last} = s_{start};$ 
{22"}  $Initialize();$ 
{23"}  $ComputeShortestPath();$ 
{24"} while ( $s_{start} \neq s_{goal}$ )
{25"} /* if ( $g(s_{start}) = \infty$ ) then there is no known path */
{26"}  $s_{start} = \arg \min_{s' \in Succ(s_{start})} (c(s_{start}, s') + g(s'));$ 
{27"} Move to  $s_{start};$ 
{28"} Scan graph for changed edge costs;
{29"} if any edge costs changed
{30"}  $k_m = k_m + h(s_{last}, s_{start});$ 
{31"}  $s_{last} = s_{start};$ 
{32"} for all directed edges  $(u, v)$  with changed edge costs
{33"} Update the edge cost  $c(u, v);$ 
{34"}  $UpdateVertex(u);$ 
{35"}  $ComputeShortestPath();$ 
```


3.5 D*-Lite算法第二种版本



3.6 AD*算法

■ 什么是AD*算法

- ARA*算法适用于复杂环境
- D*以及D*-lite算法适用于动态环境
- 在现实应用中，问题都是既复杂又动态的
- AD*算法是结合ARA*算法以及D*-lite算法的一种既可用于动态环境又能对时间做出响应的图搜索路径规划算法。

■ AD*算法思路：

- $\epsilon=1$ 时，环境发生变化，增大 ϵ 重新规划。
- 当环境发生变化，根据D* Lite的规划方法得到新的路径。

```
if ( $g(s) > rhs(s)$ )  
    return [ $rhs(s) + \epsilon \cdot h(s_{start}, s); rhs(s)$ ];  
else  
    return [ $g(s) + h(s_{start}, s); g(s)$ ];
```

3.6 AD*算法伪代码

key(s)

01. if ($g(s) > rhs(s)$)
02. return [$rhs(s) + \epsilon \cdot h(s_{start}, s); rhs(s)$];
03. else
04. return [$g(s) + h(s_{start}, s); g(s)$];

UpdateState(s)

05. if s was not visited before
06. $g(s) = \infty$;
07. if ($s \neq s_{goal}$) $rhs(s) = \min_{s' \in Succ(s)} (c(s, s') + g(s'))$;
08. if ($s \in OPEN$) remove s from $OPEN$;
09. if ($g(s) \neq rhs(s)$)
10. if $s \notin CLOSED$
11. insert s into $OPEN$ with $key(s)$;
12. else
13. insert s into $INCONS$;

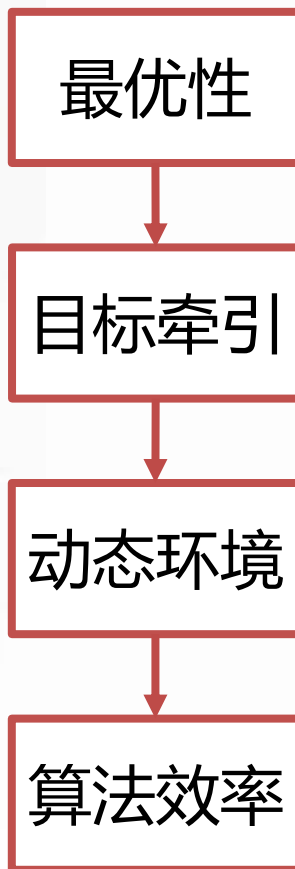
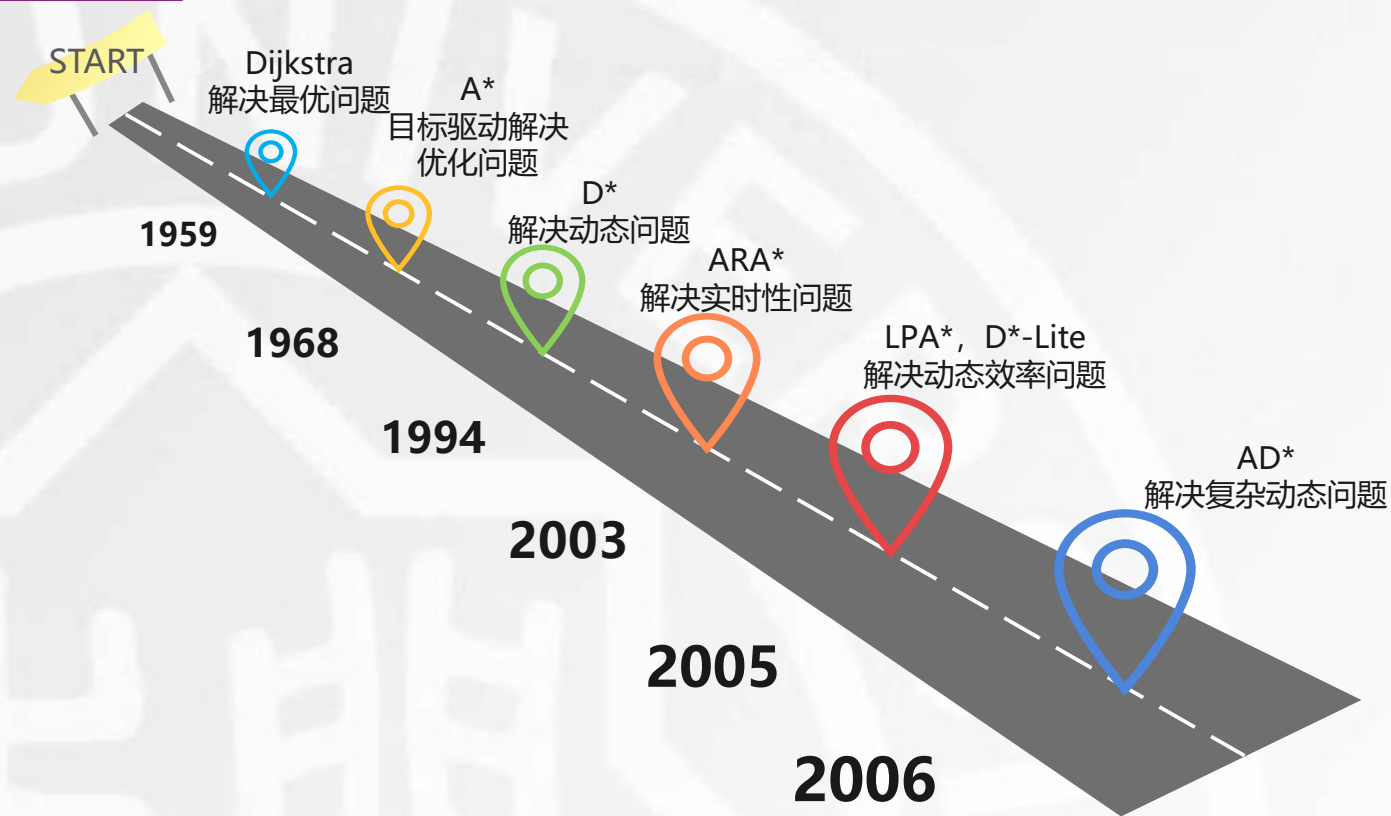
ComputeorImprovePath()

14. while ($\min_{s \in OPEN} (key(s)) < key(s_{start})$ OR $rhs(s_{start}) \neq g(s_{start})$)
15. remove state s with the minimum key from $OPEN$;
16. if ($g(s) > rhs(s)$)
17. $g(s) = rhs(s)$;
18. $CLOSED = CLOSED \cup \{s\}$;
19. for all $s' \in Pred(s)$ UpdateState(s');
20. else
21. $g(s) = \infty$;
22. for all $s' \in Pred(s) \cup \{s\}$ UpdateState(s');

Main()

01. $g(s_{start}) = rhs(s_{start}) = \infty; g(s_{goal}) = \infty$;
02. $rhs(s_{goal}) = 0; \epsilon = \epsilon_0$;
03. $OPEN = CLOSED = INCONS = \emptyset$;
04. insert s_{goal} into $OPEN$ with $key(s_{goal})$;
05. ComputeorImprovePath();
06. publish current ϵ -suboptimal solution;
07. forever
08. if changes in edge costs are detected
09. for all directed edges (u, v) with changed edge costs
10. Update the edge cost $c(u, v)$;
11. UpdateState(u);
12. if significant edge cost changes were observed
13. increase ϵ or replan from scratch;
14. else if $\epsilon > 1$
15. decrease ϵ ;
16. Move states from $INCONS$ into $OPEN$;
17. Update the priorities for all $s \in OPEN$ according to $key(s)$;
18. $CLOSED = \emptyset$;
19. ComputeorImprovePath();
20. publish current ϵ -suboptimal solution;
21. if $\epsilon = 1$
22. wait for changes in edge costs;

3.7 基于图搜索的动态路径规划总结



■ 对于大型复杂动态环境：分层思想

- 全局规划：A*、D*、D*-Lite、AD* 根据机器人运行速度，重规划周期小时、分、秒
- 局部规划：Weight A*、ARA*、LPA* 根据传感器检测信息，重规划周期百或十毫秒级

3.7 基于图搜索的动态路径规划算法对比



自动驾驶技术演进综述

从模块化自动驾驶→端到端→VLA (Vision-Language-Action Model)

报告人：李阳

- 2019.7-2022.6 南开大学人工智能学院 硕士
- 2022.7-2023.5 小鹏汽车，激光雷达感知算法工程师
- 2023.5至今 小米汽车，端到端规划模型算法工程师

报告内容：

- 自动驾驶技术的演进
- 激光雷达感知的瓶颈、优势及发展趋势
- 端到端模型发展现状



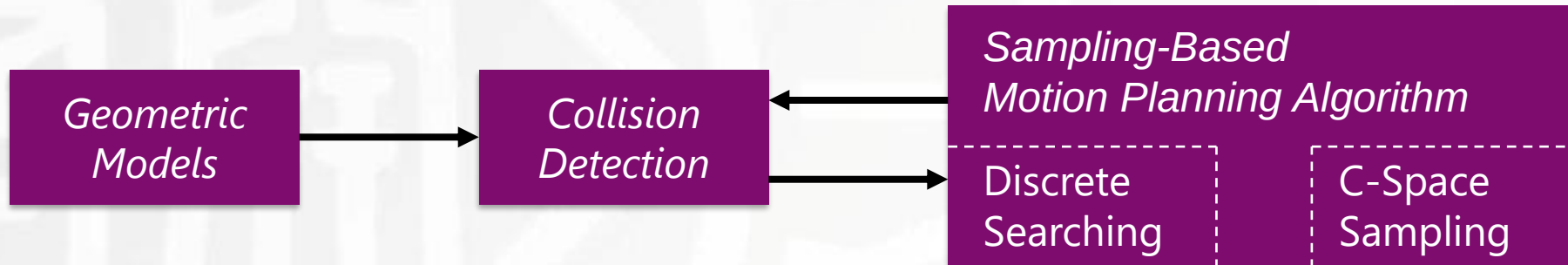
04

基于采样的路径规划算法

Simple-based Path Planning

4.1 Motivation for sampling-based

- The previous methods rely on an explicit representation of the obstacles in configuration space
- This may result in an excessive computational burden in:
 - ▣ high-dimensions
 - ▣ environments with a large number of obstacles
- Avoiding such a representation is the main underlying idea of sampling-based methods
- Instead of using an explicit representation of the environment, sampling-based methods rely on a collision-checking module



4.1.1 Sampling-based Motion Planning

■ The two most influential sampling-based motion planning algorithms to date are:

- Probabilistic Roadmaps (PRM, 1996)
- Rapidly-exploring random trees (RRT, 1998)

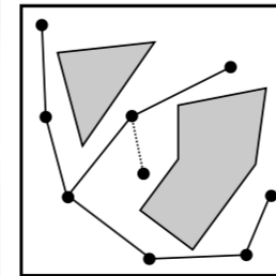
PRM:

1. Roadmap construction
2. Search

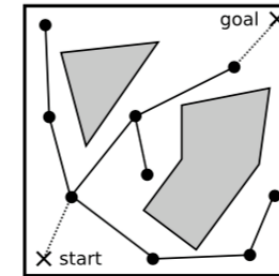
RRT:

1. Roadmap construction and search online

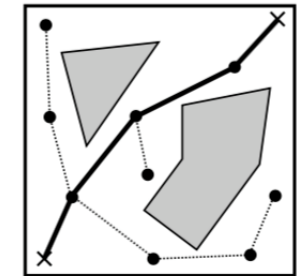
(1) PRM Algorithm



(b) A local planner is used to connect the new sample to nearby roadmap vertices.

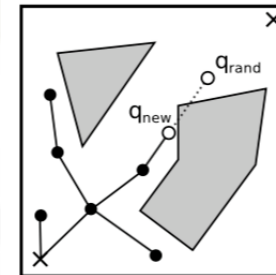


(c) The query phase: the start and goal configurations are added to the roadmap.

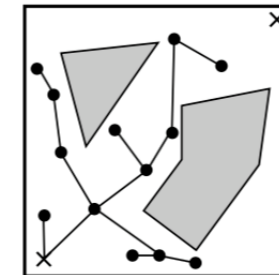


(d) A graph search algorithm is used to connect the start and goal through the roadmap.

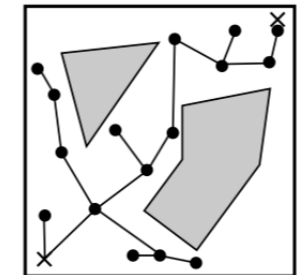
(2) RRT Algorithm



(b) The planner generates a configuration q_{rand} , and grows from the nearest node towards it to create q_{new} .



(c) The tree rapidly explores the free space.

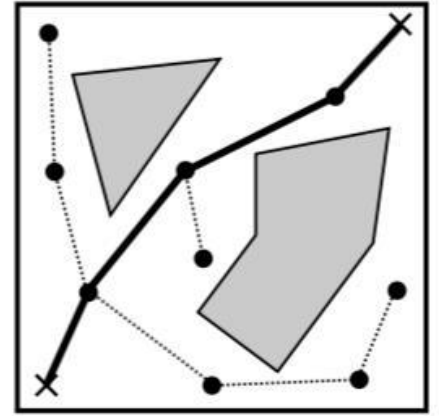


(d) The planner terminates when a node is close to the goal node. Common implementations will connect directly to the goal.

4.1.1 The general framework

■ Incremental Sampling and Searching:

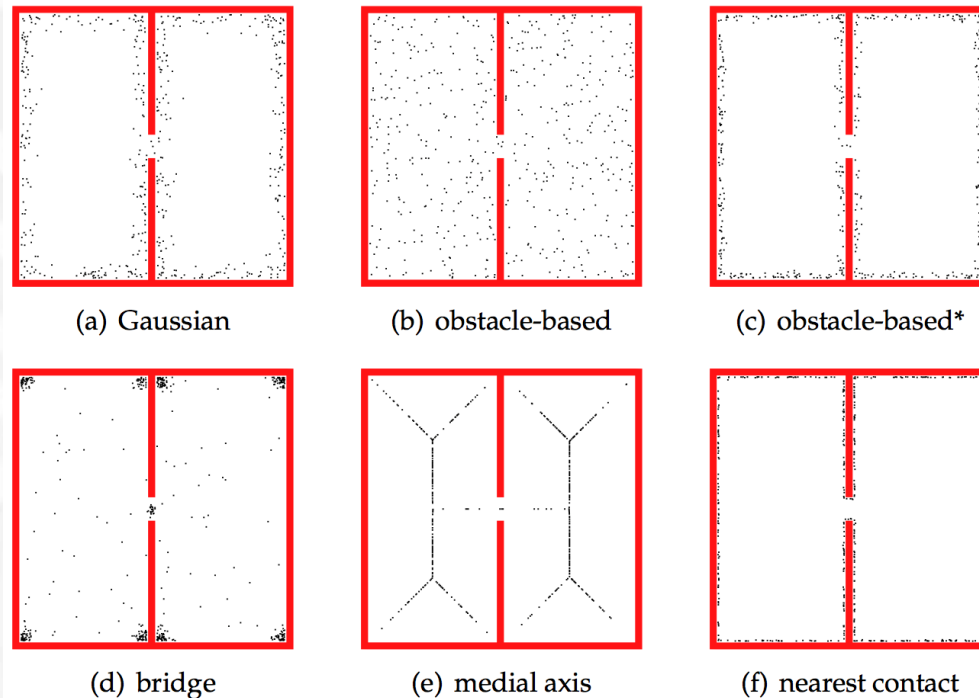
1. Initially, the graph is empty $G = (V, E)$
2. Then, a **random configuration** is generated and added to V
 - Need a Sampling algorithm
3. For every new vertex c , select a number of nodes from V 'near' c and try to connect c to each of them using a **local planner**
 - Need to define a distance function
4. If local path is **collision free**, a new edge is added to E
 - Need collision detection approach
5. Check for a solution (connection between origin and target)
6. Return to step 2



(d) A graph search algorithm is used to connect the start and goal through the roadmap.

4.1.2 Sampling algorithm

- Random uniform sampling is the easiest of all sampling methods in C-spaces
 - Combining samples from different dimensions can be combined to provide samples of the whole space
- Uniform sampling is typically used, however there are others based on different heuristics:

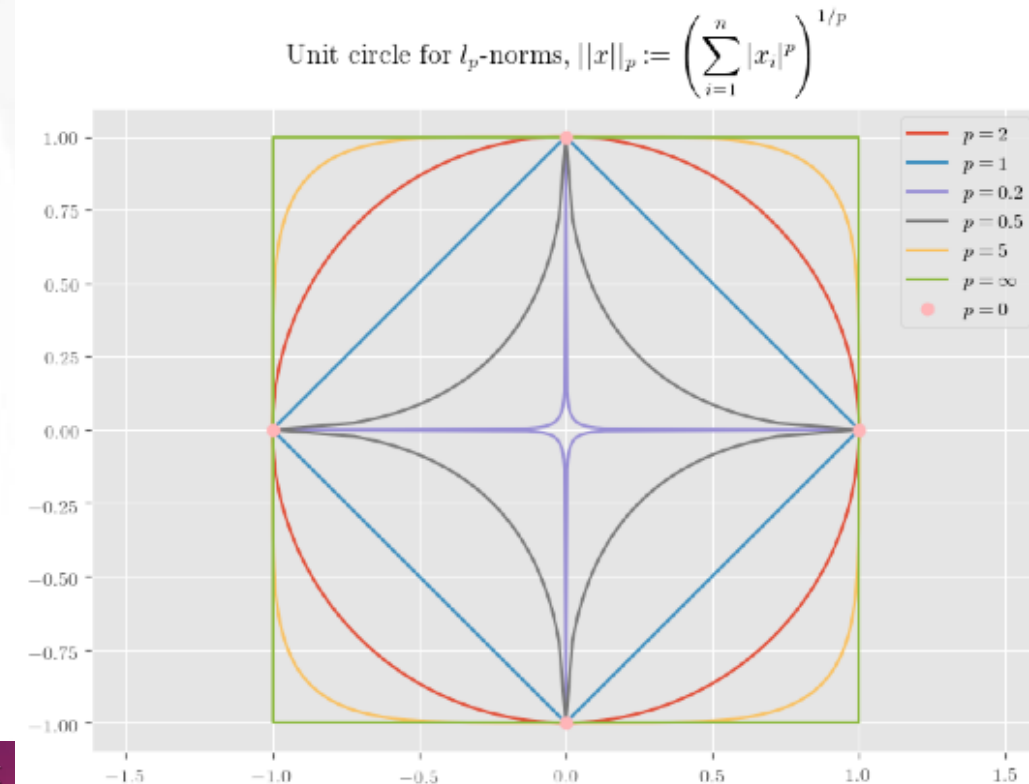


4.1.3 Metrics

- Sampling-based algorithms need to get a 'sense of proximity' among samples in configuration space
 - What does it mean to be the closest? This depends on your C-space
- In virtually all sampling-based algorithms performance depends on the choice of metric

- l_p -norm (l_p -metrics)

$$\rho(x, x') = \left(\sum_{i=1}^n |x_i - x'_i|^p \right)^{1/p}$$



4.1.3 Metrics

- In most robotic problems, the configuration space will include Euclidean points plus angles $(x, y, z, \phi, \theta, \psi)$, how should you combine these distances?
 - The simplest one, is to perform a weighted combination between rotational and translational distances

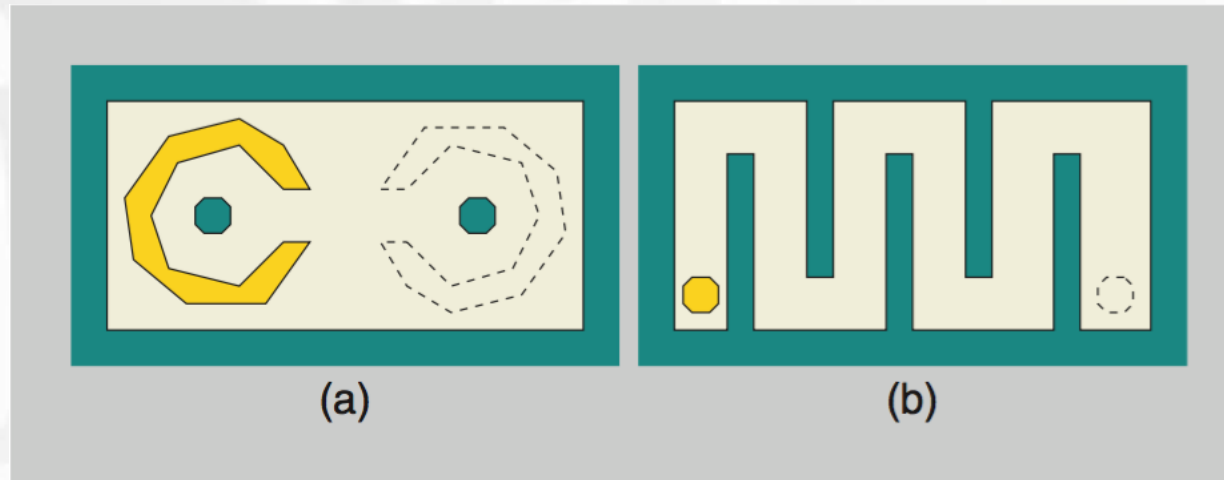
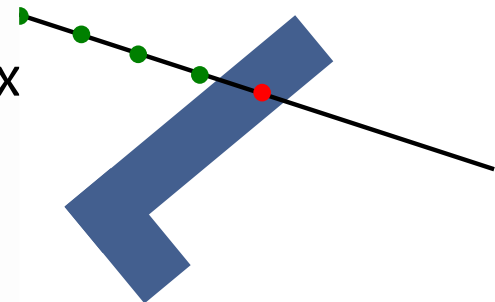


Figure 11. Rotation versus translation domination: (a) The task is to move the C shape to the right. Rotation dominates. Performance should improve if rotation is weighted heavily in the metric. (b) In this case, the translation dominates and should therefore be weighted more heavily if this fact is known in advance.

4.1.4 Collision Detection

- Once it has been decided where the samples will be placed, the next problem is to determine collisions
- In most planning applications, the majority of computation time is spent in collision checking
- A variety of collision detection algorithms exist,
 - ranging from theoretical algorithms that have excellent computational complexity
 - to heuristic, practical algorithms whose performance is tailored to a particular application
- Examples:
 - Two-phases methods: broad phase, narrow-phase
 - Hierarchical methods: decompose each body into a tree, each vertex represents a bounding region
 - Checking a path segment: by sampling the interval



4.1.4 Collision Detection

- If objects and robot can be modeled as convex, we can do linear-time collision detection

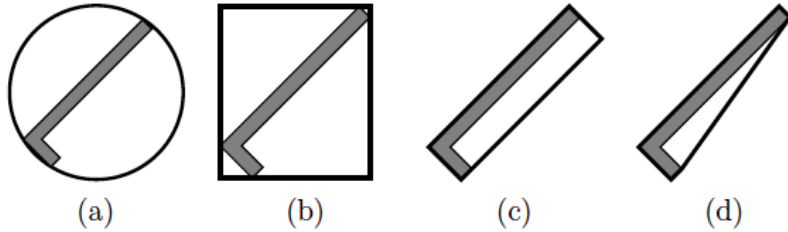
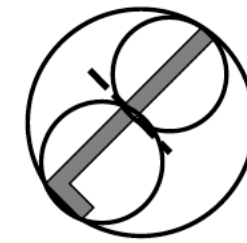


Figure 5.9: Four different kinds of bounding regions: (a) sphere, (b) axis-aligned bounding box (AABB), (c) oriented bounding box (OBB), and (d) convex hull. Each usually provides a tighter approximation than the previous one but is more expensive to test for overlapping pairs.

- New representations have been adopted in recent years that facilitate collision checking
 - At the cost of spending more time in building the representation



4.2 Probabilistic Road Maps (PRM)

■ Probabilistic Road(M)aps (PRMs) use this principle:

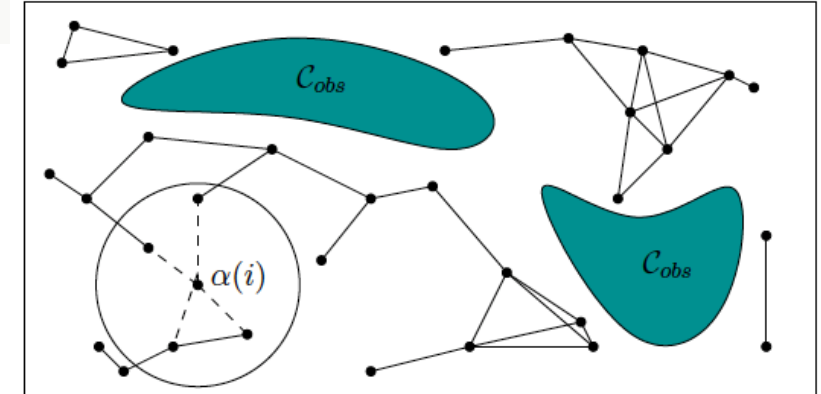
■ Three steps:

- Step 1: Build a roadmap by connecting nearby (sampled, free-space) configurations using simple planners to construct a graph of valid path segments

BUILD_ROADMAP

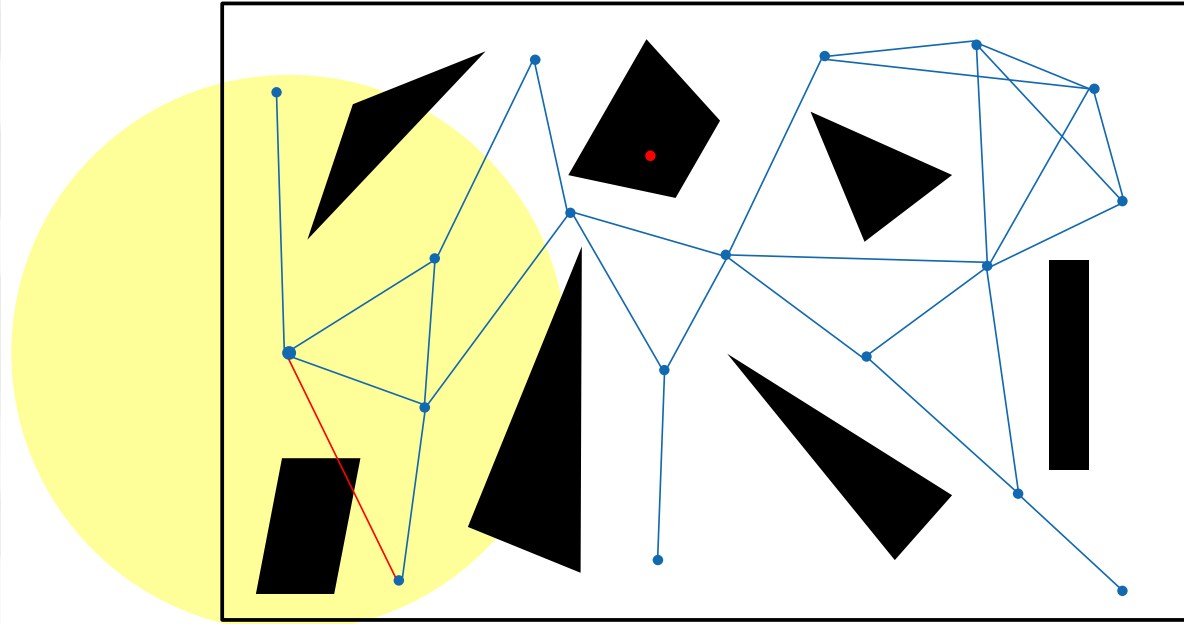
```
1   $\mathcal{G}.\text{init}(); i \leftarrow 0;$   
2  while  $i < N$   
3      if  $\alpha(i) \in \mathcal{C}_{\text{free}}$  then  
4           $\mathcal{G}.\text{add\_vertex}(\alpha(i)); i \leftarrow i + 1;$   
5          for each  $q \in \text{NEIGHBORHOOD}(\alpha(i), \mathcal{G})$   
6              if  $((\text{not } \mathcal{G}.\text{same\_component}(\alpha(i), q)) \text{ and } \text{CONNECT}(\alpha(i), q))$  then  
7                   $\mathcal{G}.\text{add\_edge}(\alpha(i), q);$ 
```

- Step 2: Collision detection and cut the edge
- Step 3: Query; Search the graph using a graph search technique (A*)

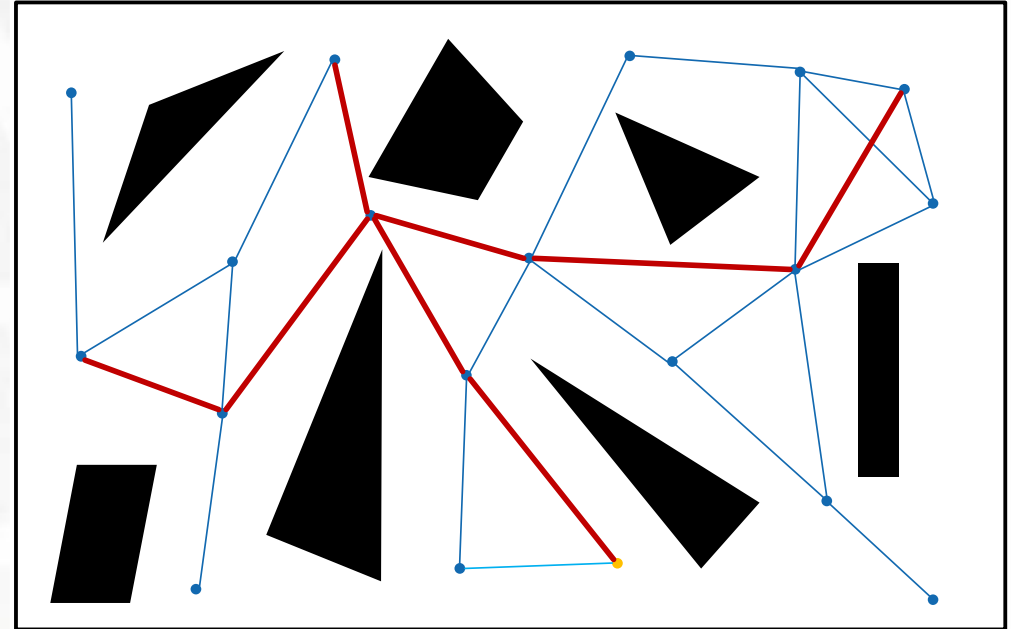


Kavraki, Lydia E., et al. "Probabilistic roadmaps for path planning in high-dimensional configuration spaces." IEEE Transactions on Robotics and Automation 12.4 (1996): 566-580.

4.2 Probabilistic Road Maps (PRM)



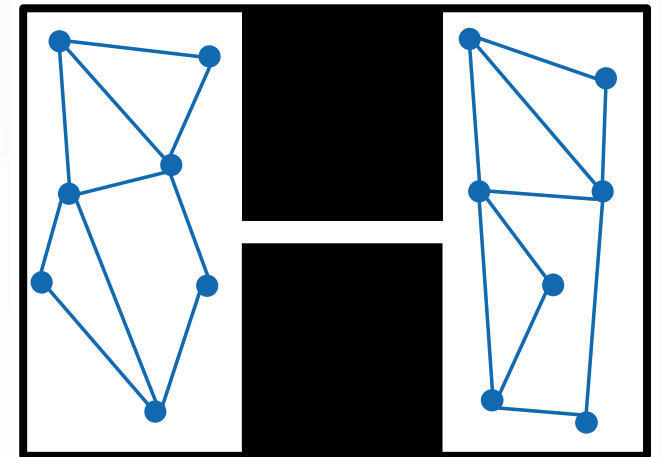
Step 2: Collision detection and cut the edge



Step 3: Query; Search the graph using a graph search technique

4.2 Probabilistic Road Maps (PRM)

- Conceptually very simple, and became rapidly popular due to PRM's ability to solve previously challenging problems (high-dimension planning, snake robots, piano-mover)
- A few limitations with the original method, most of which were largely subsequently solved in derivative variations
 - ❑ Requires holonomic motion
 - ❑ Can suffer from 'narrow passage problem'
 - ❑ Not suited for dynamic environments



4.2 Probabilistic Road Maps (PRM)

PRM Summary:

- A graph is constructed in the configuration space by connecting random configurations
- PRM is designed for **multiple-queries**
- The underlying assumption is that it is worth performing **substantial pre-computation** on a given environment to enable multiple planning queries
- However, often we are interested only in a **efficient single-query** without pre-computations
 - E.g. in dynamic environments, different robots model
- In this case we want to **combine exploration and search** into a single method
 - This motivates tree-based planning approaches

4.3 Tree-based Simple Planners

- Problem – How do we connect all these disjoint paths?
- Solution – Instead of sampling all our target states at once, we start from a start configuration, and **incrementally** grow outwards from there – forming a tree of connected, dynamically feasible local path
- Terminate when we reach a configuration in a goal **region**

- The tree-based simple is for **single-query** planning
- In roadmaps, when planning with differential constraints, it's not easy to encode control information (undirected edges)
- Tree-based, on the other hand, can include complex dynamics in their directed graphs
 - Control information can be encoded in each edge

4.3 Tree-based Simple Planners

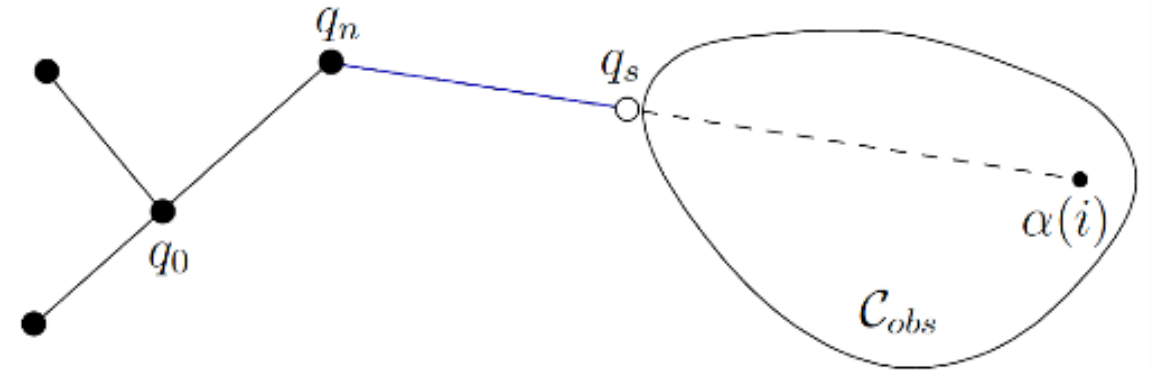
- There exist many types of sampling-based planners that create tree structures of the free-space
 - E.g. RRT (LaValle 1998), EST (Hsu, 2001), SBL (Sanchez, 2005), KPIECE, etc
- In general
 - Start with a root node
 - Employs an expansion heuristic (typically gives the name to the method)
 - Connect node to tree if collision free path
- Note: the methods often bias the expansion of the tree toward the goal state

4.3 Rapidly Exploring Dense Trees (RDT)

- RDTs are an incremental sampling and searching approach that gives good performance “without any parameter tuning”
- The idea is to incrementally construct a search tree that gradually improves the resolution
- A dense sequence of samples is used to incrementally build the tree
 - If the sequence is random, is called rapidly exploring random tree (RRT)

SIMPLE_RDT(q_0)

```
1   $\mathcal{G}.\text{init}(q_0);$   
2  for  $i = 1$  to  $k$  do  
3     $\mathcal{G}.\text{add\_vertex}(\alpha(i));$   
4     $q_n \leftarrow \text{NEAREST}(S(\mathcal{G}), \alpha(i));$   
5     $\mathcal{G}.\text{add\_edge}(q_n, \alpha(i));$ 
```



Connect nearest point with random point using a local planning that travels from q_{near} to q_{rand}

- ✓ No collision : add edge
- ✓ Collision: new vertex is q_s , as close as possible to C_{obs}

4.3 RRT

LAVALLE S M. R apidly-exploring random trees: a new tool path planning[R].
Ames, USA: Iowa State University, 1998: 293-308.

S. M. LaValle and J. J. Kuffner Jr., “Randomized kinodynamic planning,” IJRR,
20(5): 378–400, 2001



Steve LaValle

University of Oulu

Finland



James Kuffner

School of Computer Science

Carnegie Mellon University

4.3.1 RRT核心思想

- Let's consider a simple case, where the motion is unconstrained (holonomic)
- The main idea is to incrementally probe and explore the C-space
 - In the early iterations the RRT quickly reaches the unexplored parts
 - However, RRT is dense in the limit, which means that it will eventually get to any point in the space

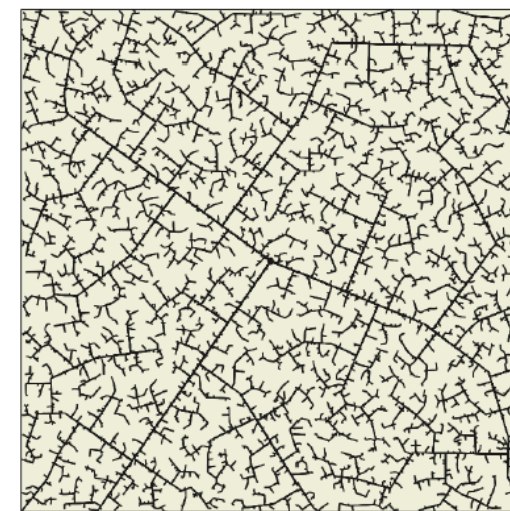
Input: q_{start} , q_{goal} , number n of nodes, stepsize α , β

Output: tree $T = (V, E)$

- 1: initialize $V = \{q_{\text{start}}\}$, $E = \emptyset$
- 2: **for** $i = 0 : n$ **do**
- 3: **if** $\text{rand}(0, 1) < \beta$ **then** $q_{\text{target}} \leftarrow q_{\text{goal}}$
- 4: **else** $q_{\text{target}} \leftarrow$ random sample from Q
- 5: $q_{\text{near}} \leftarrow$ nearest neighbor of q_{target} in V
- 6: $q_{\text{new}} \leftarrow q_{\text{near}} + \frac{\alpha}{|q_{\text{target}} - q_{\text{near}}|} (q_{\text{target}} - q_{\text{near}})$
- 7: **if** $q_{\text{new}} \in Q_{\text{free}}$ **then** $V \leftarrow V \cup \{q_{\text{new}}\}$, $E \leftarrow E \cup \{(q_{\text{near}}, q_{\text{new}})\}$
- 8: **end for**

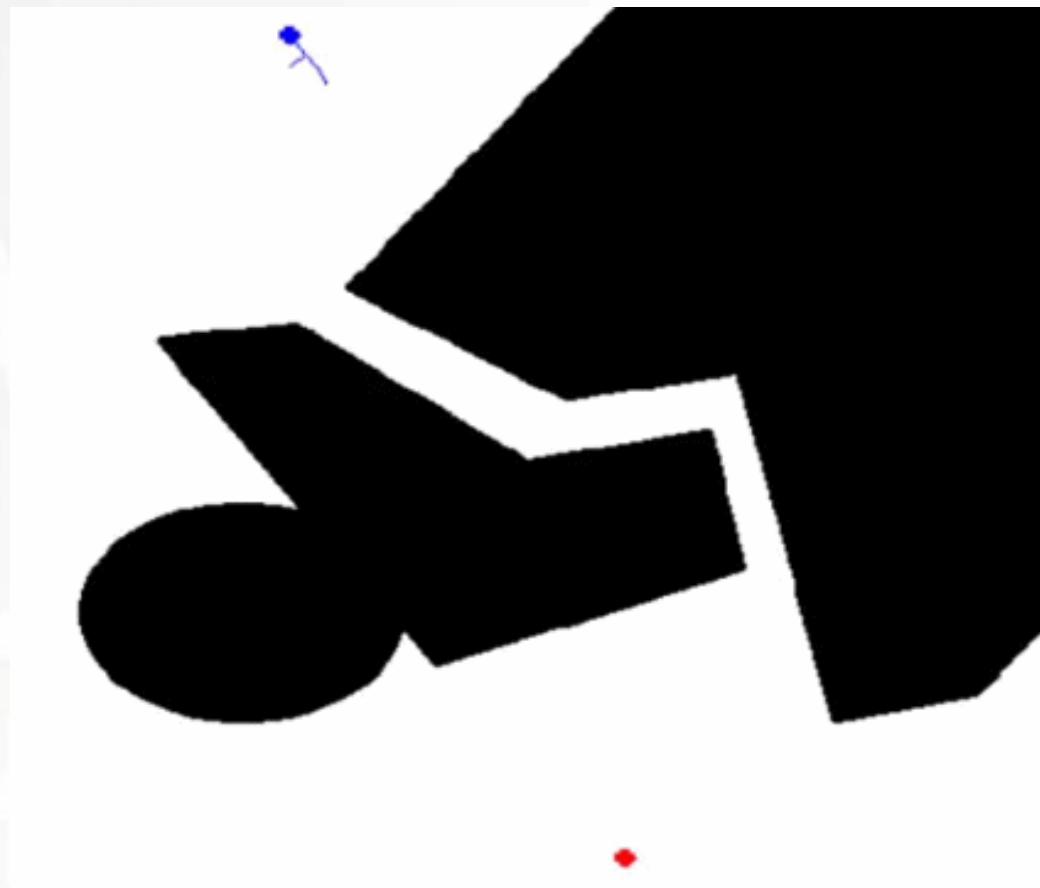
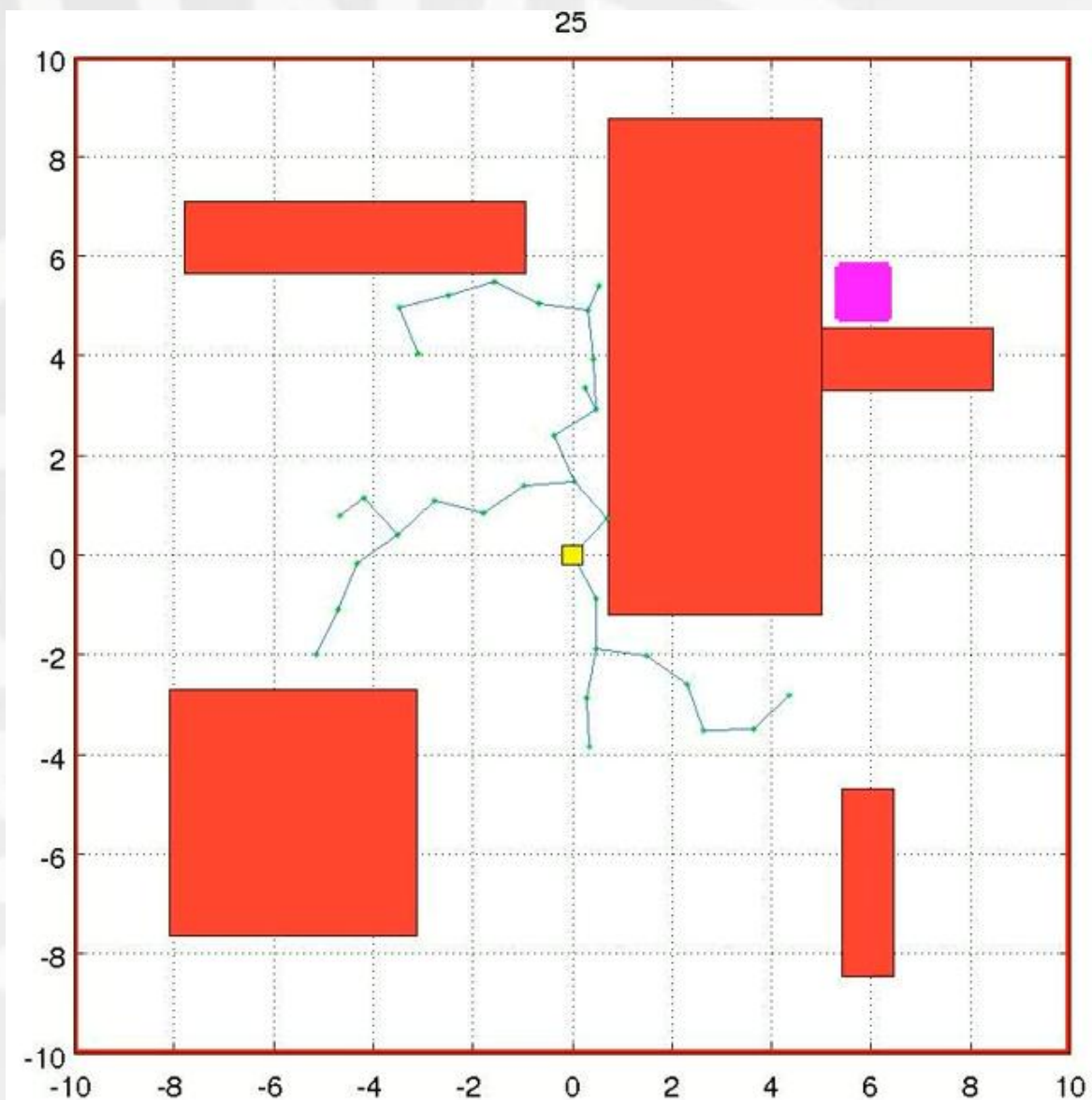


45 iterations

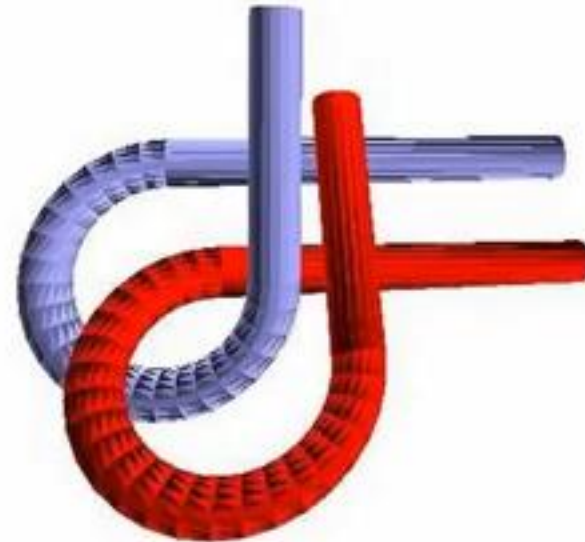
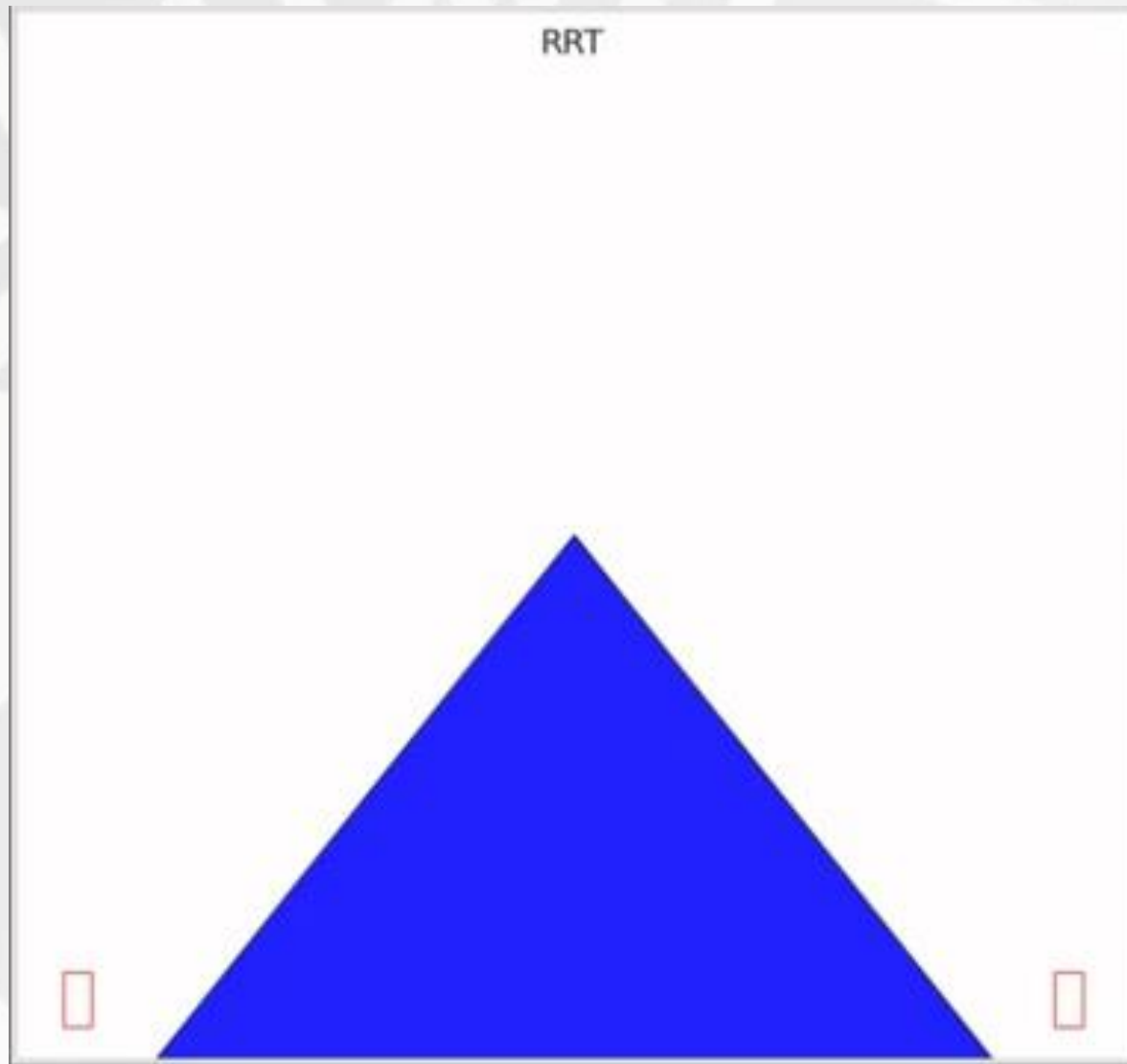


2345 iterations

4.3.2 RRT算法仿真实验



4.3.2 RRT算法仿真实验



4.3.3 RRT算法分析

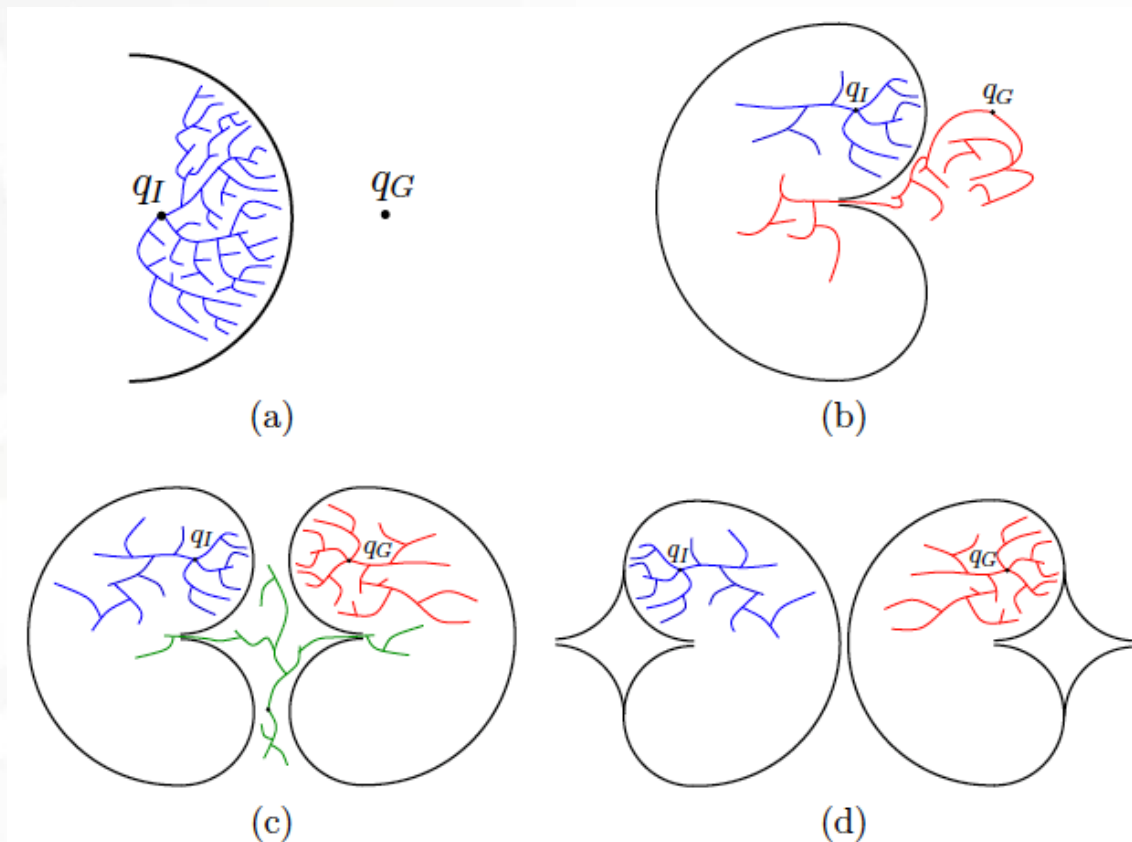
- RRT算法可以生成一条可行路径，算法具有完备性（概率上），而且RRT无需对空间建模就可以进行路径搜索，能够有效解决高维空间和复杂约束的规划问题。

- 如何提高效率？

- Bidirectional: two-trees
 - Multi-directional

- 算法只能生成一条可行路径，而不是最优的。

- 如何能够得到最优路径？



4.4 RRT*

S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” IJRR, 30(7): 846–894, 2011.



Sertac Karaman

*Department of
Aeronautics and
Astronautics*

*Massachusetts
Institute of Technology*



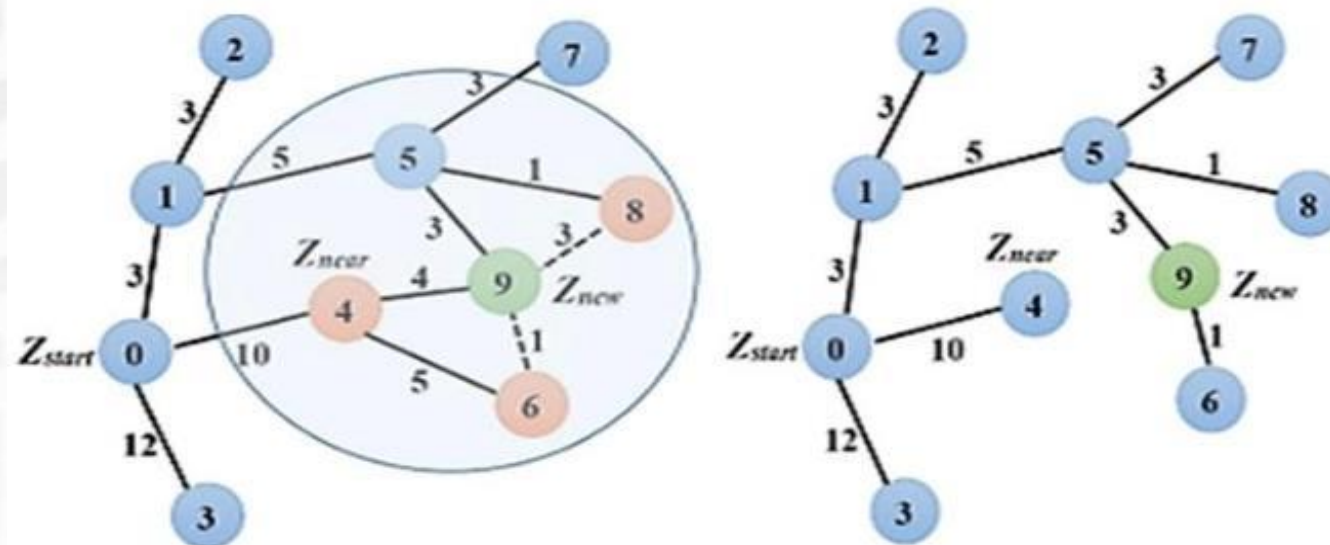
Emilio Frazzoli

*Department of
Aeronautics and
Astronautics*

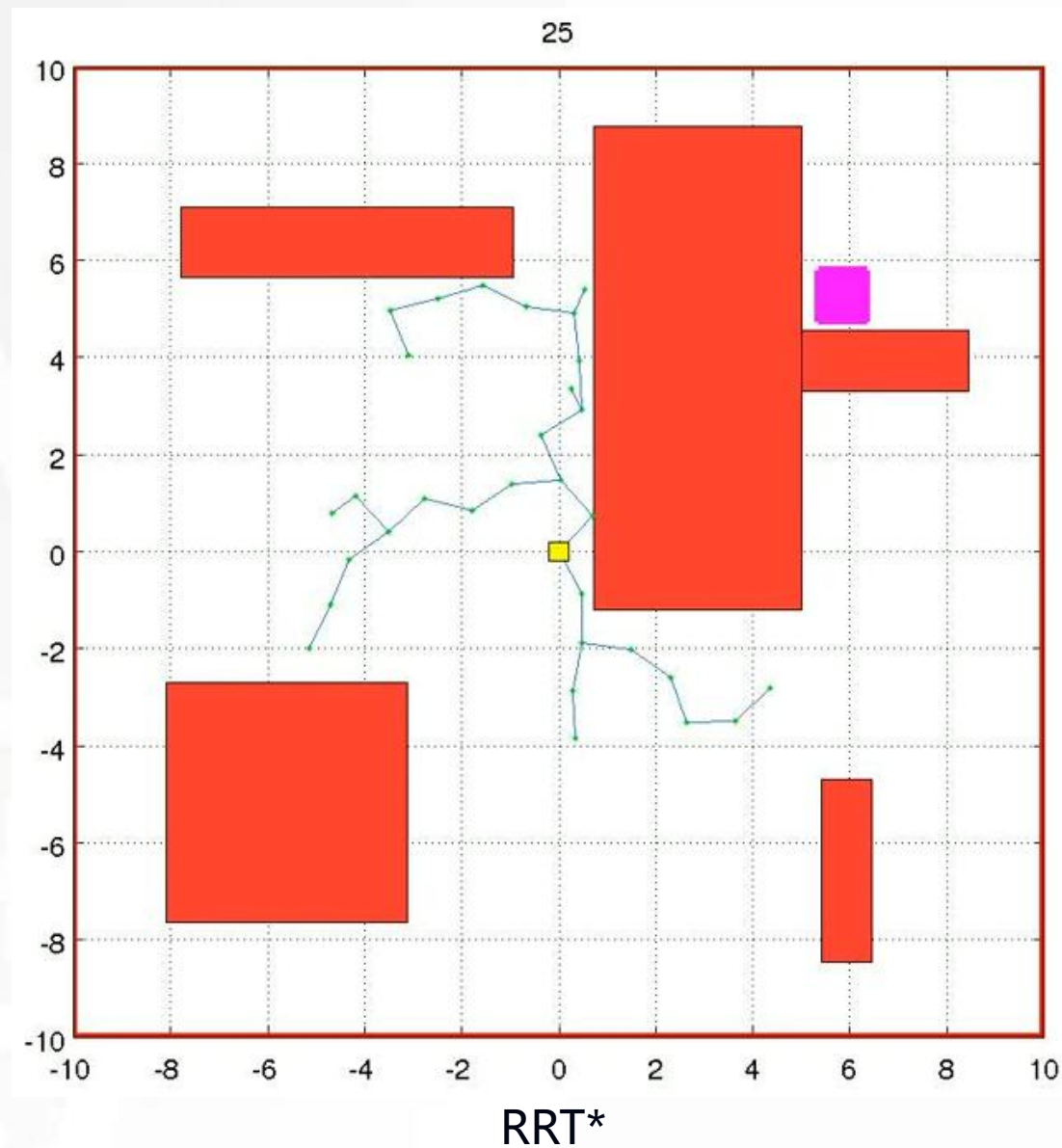
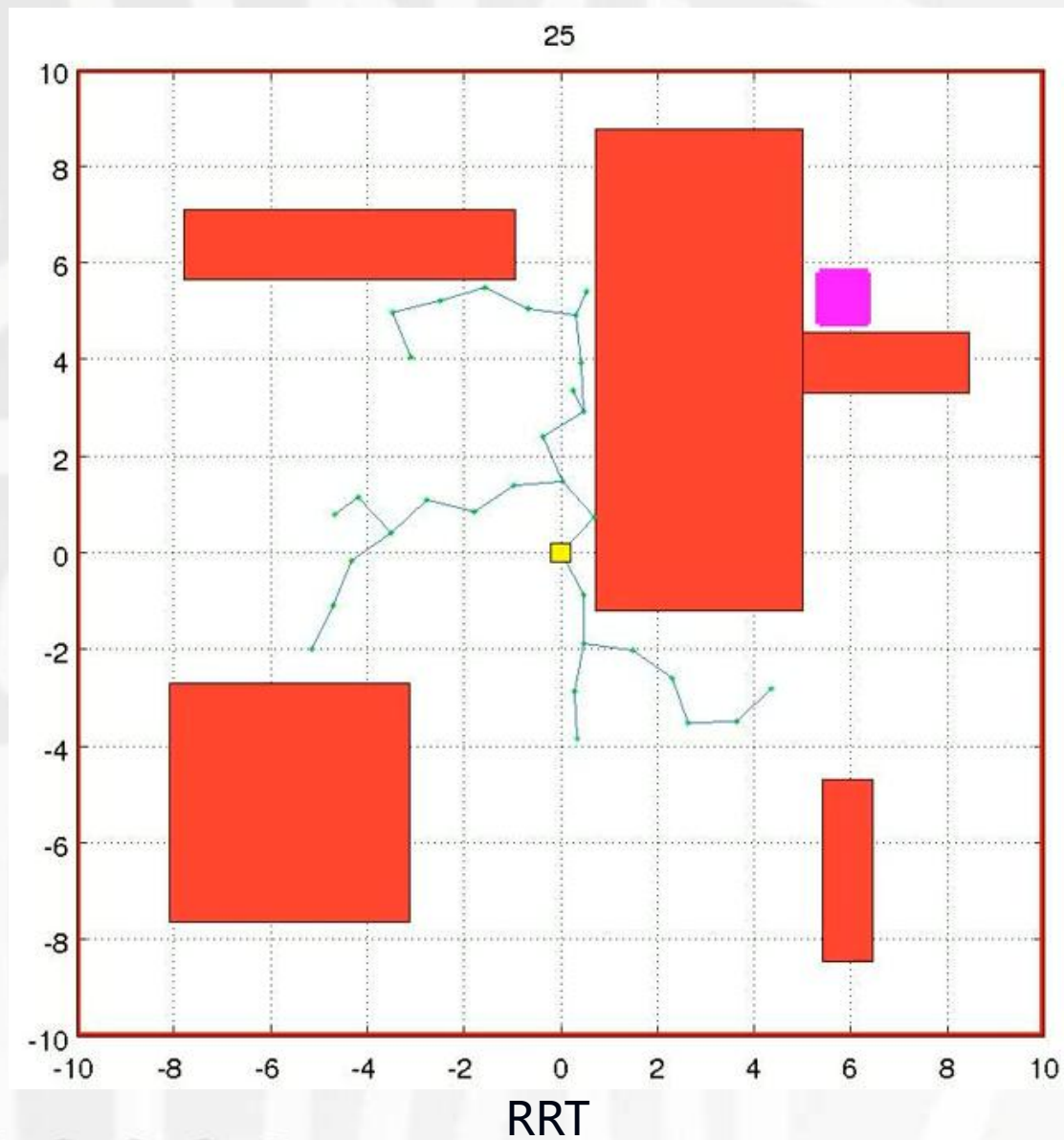
*Massachusetts Institute
of Technology*

4.4.1 RRT*核心思想

- RRT* inherits all the properties of RRT and works similarly to RRT. However, it introduced two promising features called near neighbor search and rewiring tree operations
 - Near neighbor operations finds the best parent node for the new node before its insertion in tree
 - Rewiring operation rebuilds the tree within this radius of area k to maintain the tree with minimal cost between tree connections



4.4.1 RRT*核心思想



4.4.2 RRT*算法流程

- 将起点加入树中
- 从搜索空间随机取点扩展树
- 临近空间搜索、树的重新连接
 - 节点在更换父节点时也需要判断连线是否经过障碍物。
- 若树扩展到了目标节点或进入目标区域，便可以在树上找到一条从起始点到终点的路径。
- 继续随机取点优化路径，直至得到最优路径。
 - 可以通过判断更新后得到的最优路径代价值是否收敛至定值来判断是否得到最优路径；为了得到最优路径，随机取点的最大循环次数应当要足够大。

4.4.3 RRT*算法分析

- RRT*在具备完备性、无需建模等优点的同时，还能得到一条较为优化的路径，**如何得到最优路径**
- 得到最优路径所需的迭代次数可能会很多，需要消耗大量计算资源。**那么，如何减少迭代次数？**

4.5 Informed RRT*

Jonathan D. Gammell, Siddhartha S. Srinivasa, Timothy D. Barfoot. Informed RRT: Optimal Incremental Path Planning Focused through an Admissible Ellipsoidal Heuristic. IROS. 2014.*



*Jonathan D.
Gammell*

Oxford Robotics
Institute



*Siddhartha S.
Srinivasa*

University of
Washington Seattle

4.5.1 Informed RRT*核心思想

概念定义:

1. X_{start} : 起点
2. X_{goal} : 终点
3. $f(x)$: 经过状态 x 的最佳路径代价值
4. c_{best} : 当前路径的代价值
5. X_f : 可以优化当前路径的状态的集合 $X_f = \{x \in X \mid f(x) < c_{best}\}$
6. $\hat{f}(x)$: 启发函数, 表示 $f(x)$ 的估计值
7. $X_{\hat{f}} = \left\{x \in X \mid \hat{f}(x) < c_{best}\right\}$

4.5.1 Informed RRT*核心思想

- 由于经过 x 的最佳路径的长度最小为 x 到起点、终点的直线距离和，所以，取

$$\hat{f}(x) = \|x_{start} - x\|_2 + \|x - x_{goal}\|_2$$

- 进而可以得到：

$$X_{\hat{f}} = \{x \in X \mid \|x_{start} - x\|_2 + \|x - x_{goal}\|_2 \leq c_{best}\}$$

该区域为恰好表示为一个椭圆。

- 在该椭圆内取随机点可以优化当前的最佳路径，所以，只需在该椭圆内取点即可对路径进行优化。

4.5.1 Informed RRT*核心思想

椭圆区域的算法表示为:

$$X_{\hat{f}} = CLx_{ball} + x_{center}$$

其中:

$$x_{center} = \frac{x_{start} + x_{goal}}{2}$$

$$x_{ball} \in X_{ball} = \{x \in X \mid \|x\|_2 \leq 1\}$$

$$L = \text{diag} \left\{ \frac{c_{best}}{2}, \frac{\sqrt{c_{best}^2 - c_{min}^2}}{2} \right\}$$

$$C = U \text{diag}\{1, \det(U), \det(V)\} V^\top$$

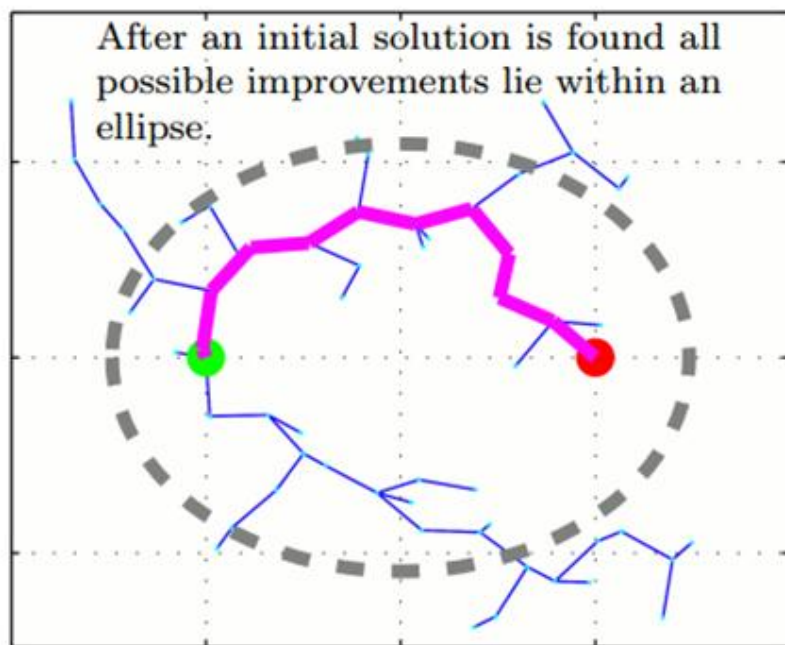
而 U 、 V 由矩阵 $M = a_1 \begin{bmatrix} 1 & 0 \end{bmatrix}$ 经过奇异值分解 $M = U\Sigma V^\top$ 得到。

其中,

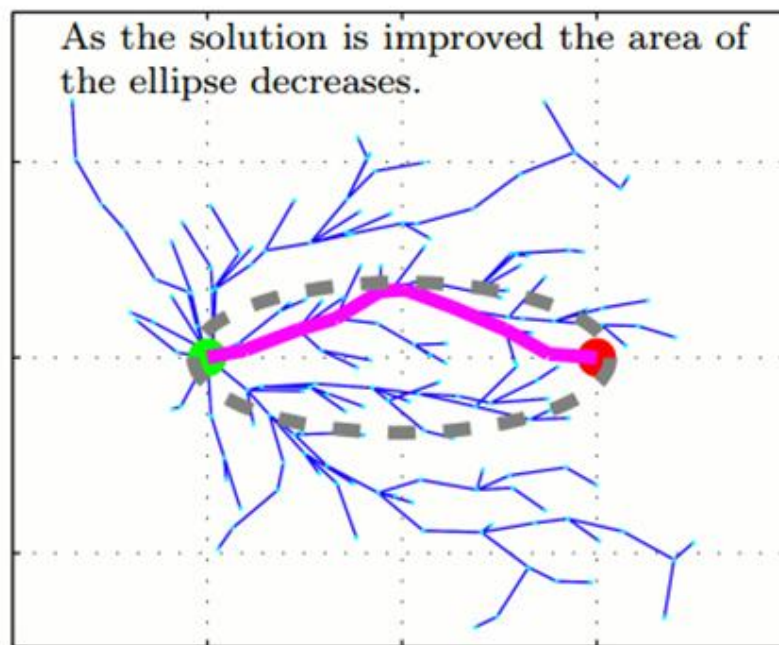
$$a_1 = \frac{x_{goal} - x_{start}}{\|x_{goal} - x_{start}\|_2}$$

4.5.2 Informed RRT*算法流程

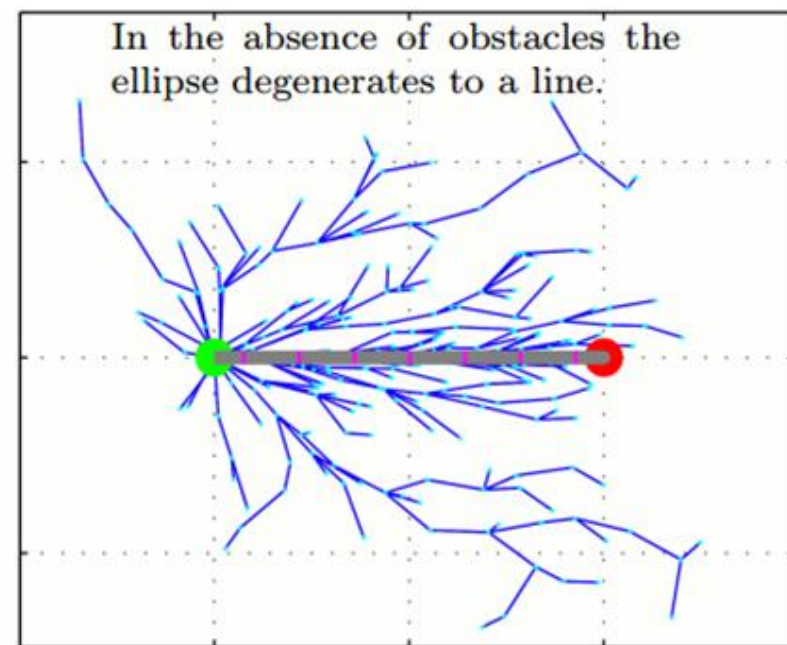
- 按照RRT*的思路得到可行路径。
- 根据已有最优路径代价值得到椭圆区域，在椭圆区域内随机取点
- 随着路径的优化，椭圆区域不断减小直至得到最优路径。



59 iterations, $c_{\text{best}} = 148.24$

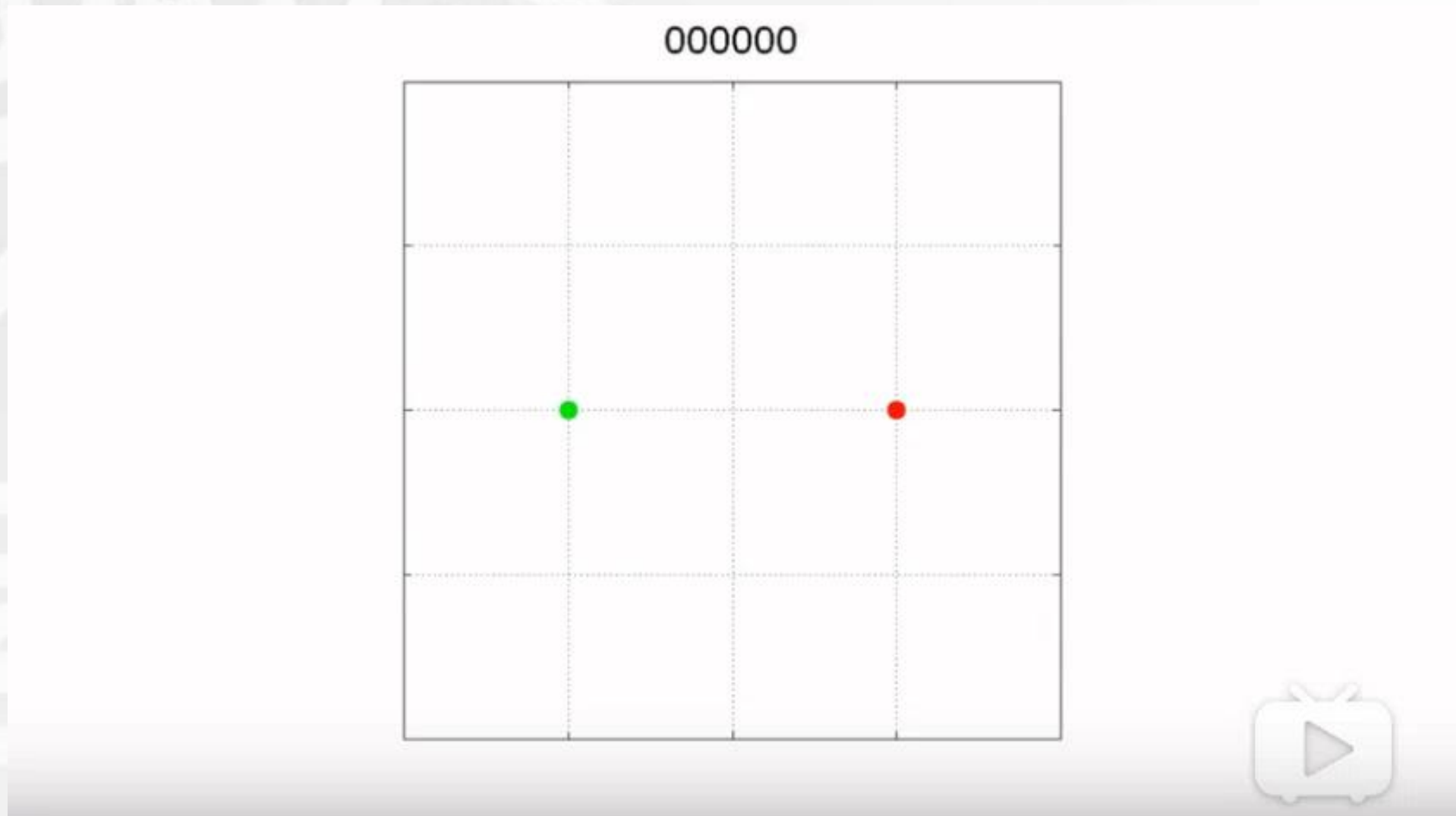


175 iterations, $c_{\text{best}} = 107.12$



1142 iterations, $c_{\text{best}} = 100$

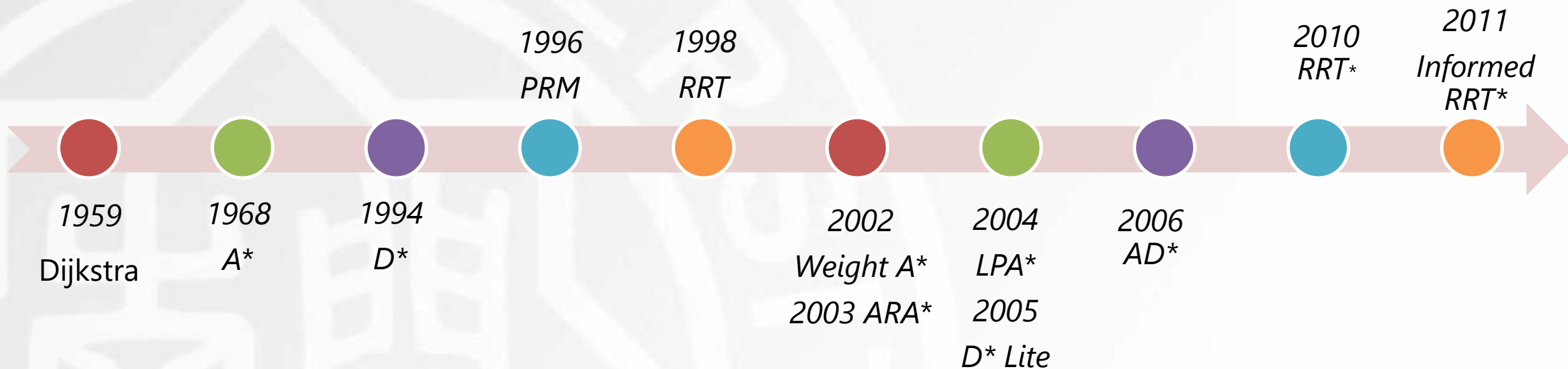
4.5.3 Informed RRT*算法仿真



4.5.4 Informed RRT*算法分析

通过不断缩小随机点的范围，Informed RRT*算法可以大大缩短得到最优路径所需要的时间，在计算资源的节省上要大大优于RRT*算法。同时，Informed RRT*也具备完备性、无需建模等优点，因而该算法实用性较高。

4.6 全局路径规划算法发展历程



05

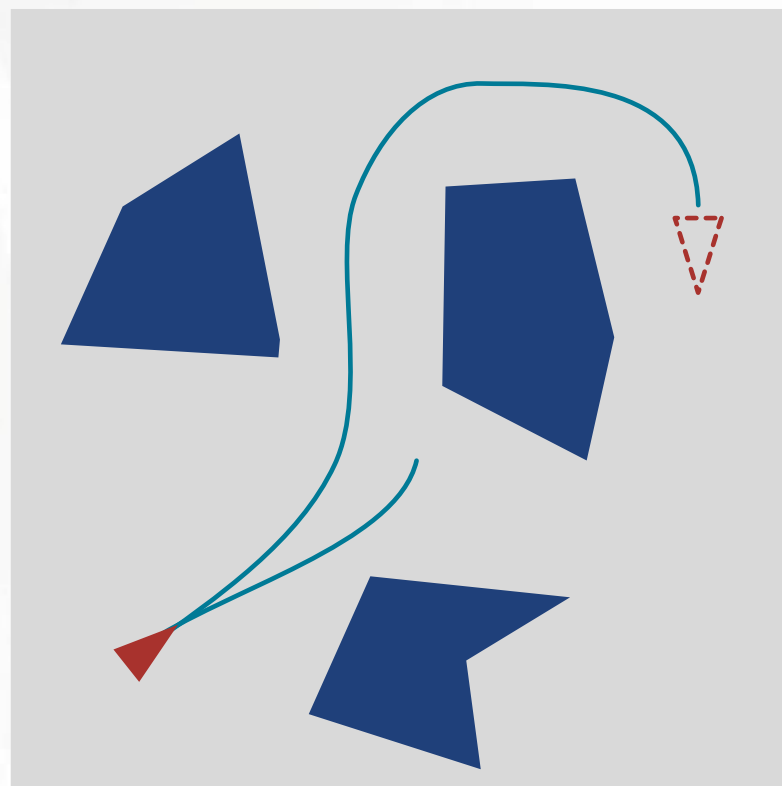
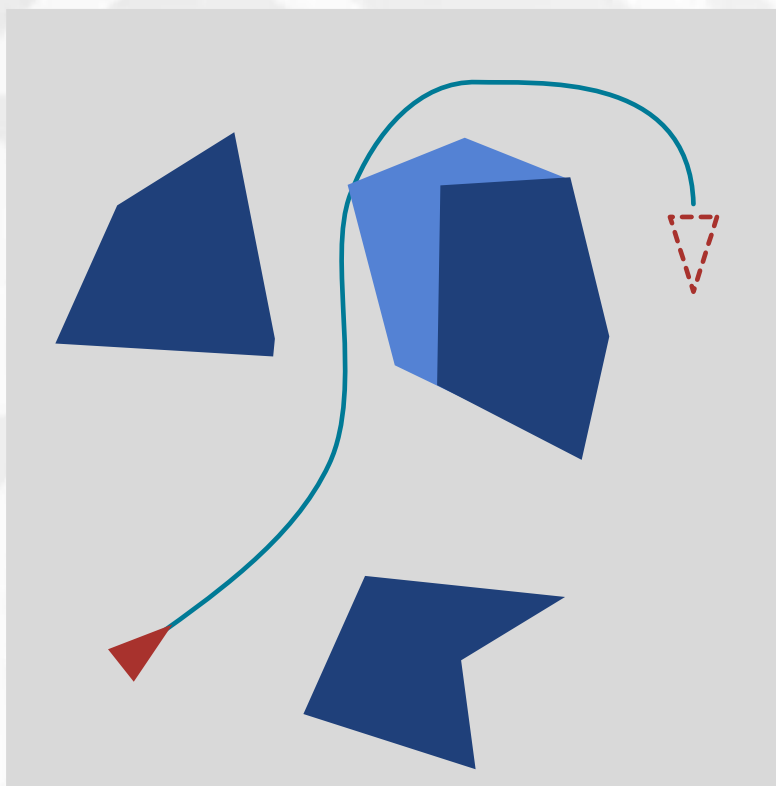
面向避碰的局部路径规划算法

Local Path Planning for Collision Avoidance

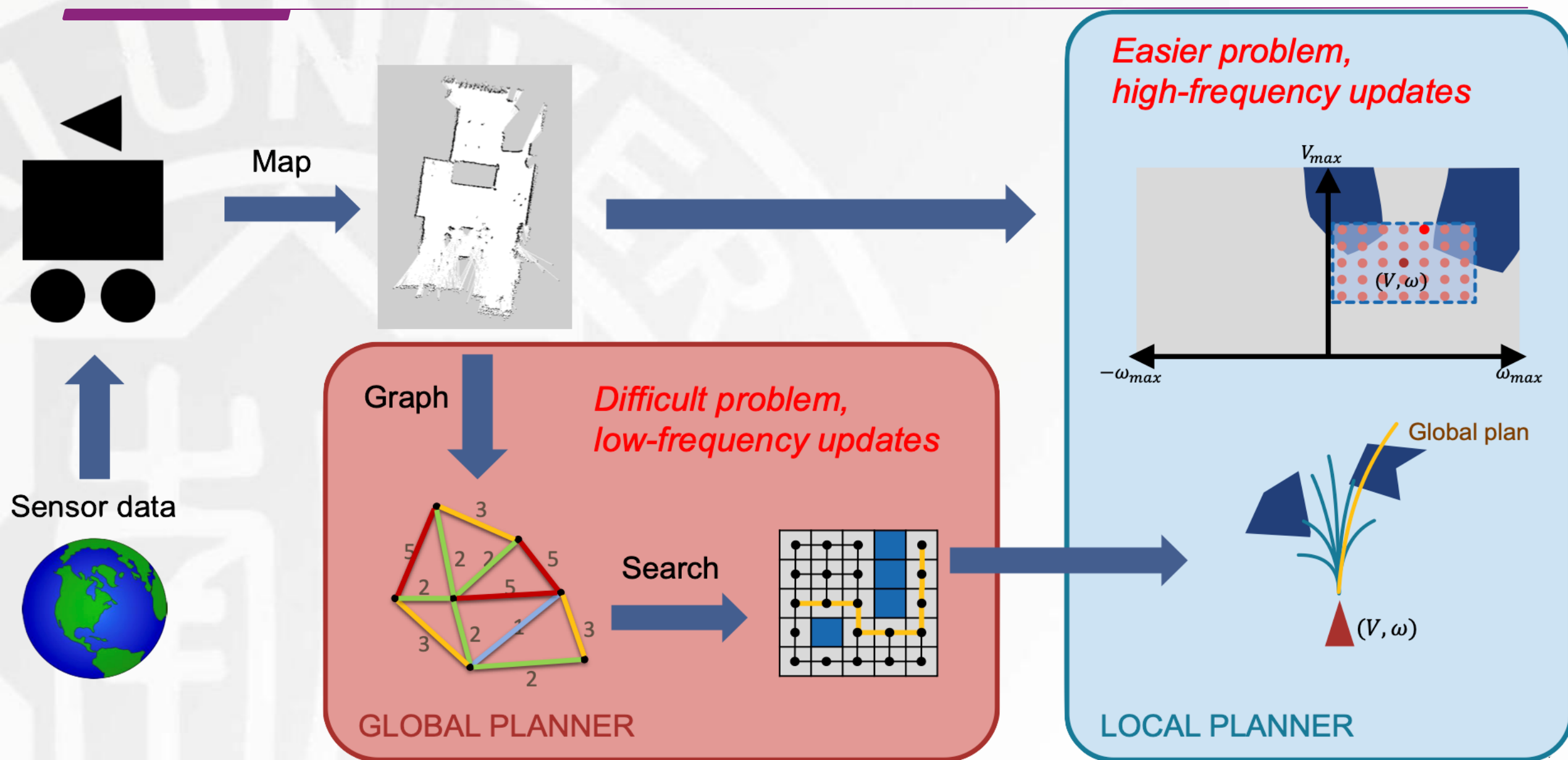
5 机器人局部路径规划意义

全局路径规划问题：

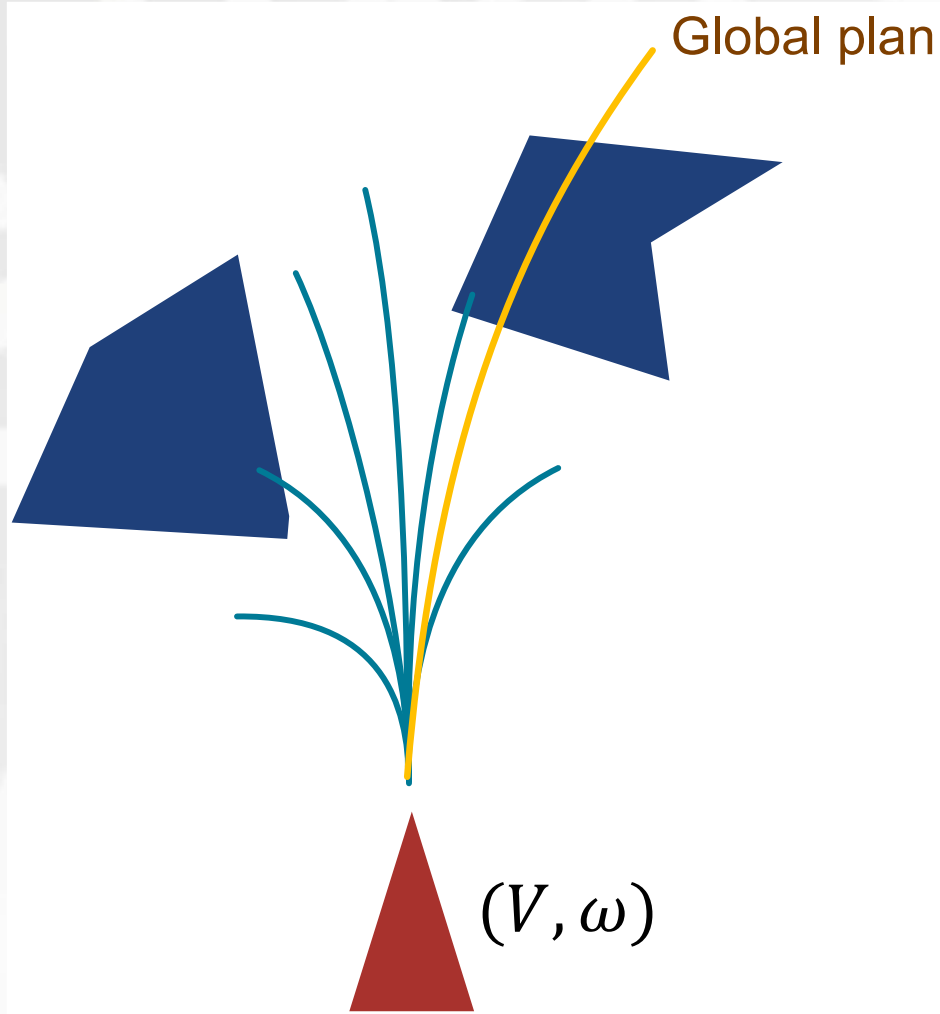
- 全局地图为静态地图，未考虑机器人实际运行时环境的变化，如动态障碍物等
- 机器人运行时受机器人建模误差、计算误差、里程计测量误差、建图误差等影响，机器人运行时会偏离轨迹



5.1 机器人路径规划架构



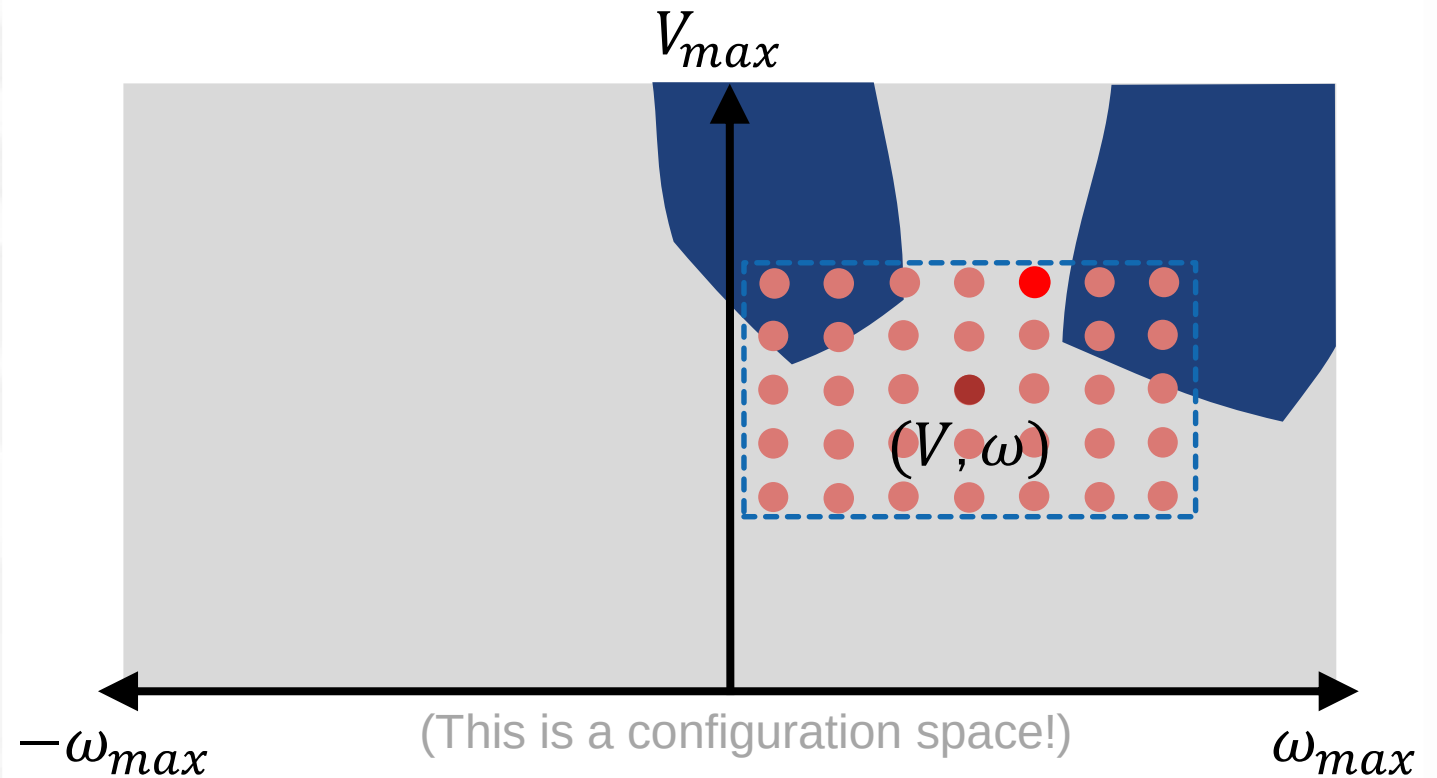
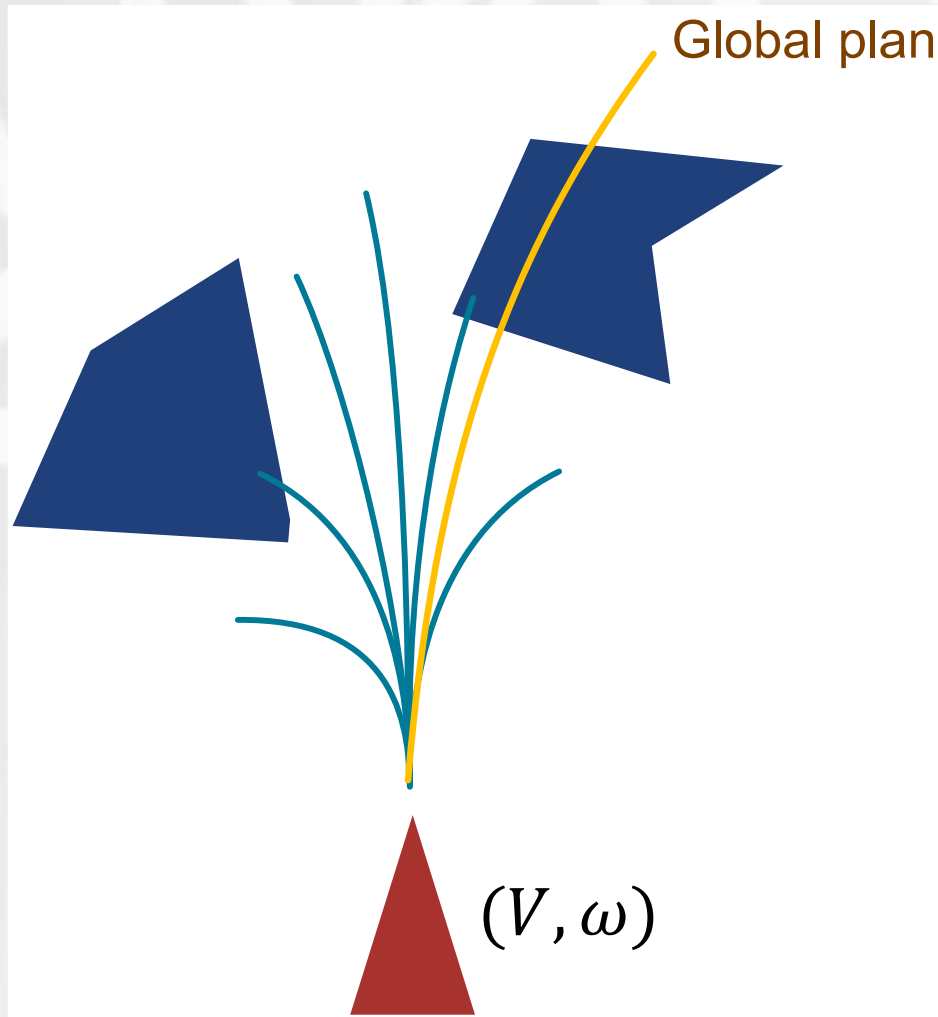
5.2 DWA(Dynamic Window Approach)



- Generating full time-varying trajectories for $V(t)$ and $\omega(t)$ is still very challenging
- If we assume (V, ω) are constant for a fixed Δt , each local path in the future is a circular arc segment
- This can be easily considered as a type of velocity configuration space

Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. IEEE Robotics & Automation Magazine, 4(1), 23-33.

5.2 DWA(Dynamic Window Approach)



- 在一定的时间窗口下，依照机器人当前状态（位置、速度、加速度等）生成多条轨迹
- Maximise utility metric (usually maximise speed, minimise distance to goal, maximise distance from obstacles) across configuration samples

5.2 DWA基本思路

■ 机器人轨迹（运动）估计(龙哥库塔法):

$$\begin{cases} \Delta\theta = \omega\Delta t \\ \Delta x = v\Delta t \cos\left(\theta_t + \frac{\Delta\theta}{2}\right) \\ \Delta y = v\Delta t \sin\left(\theta_t + \frac{\Delta\theta}{2}\right) \end{cases}$$

■ 速度采样:

□ 移动机器人速度约束

$$V_m = \{v \in [v_{\min}, v_{\max}], \omega \in [\omega_{\min}, \omega_{\max}]\}$$

□ 移动机器人执行器能力约束

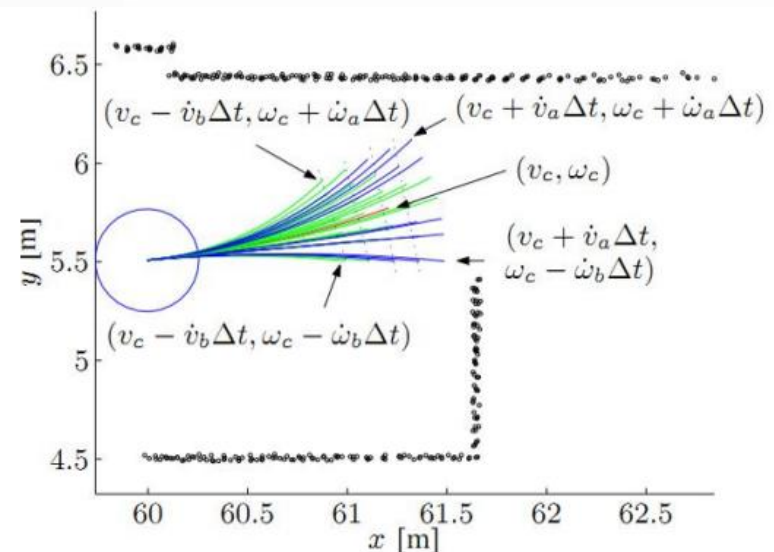
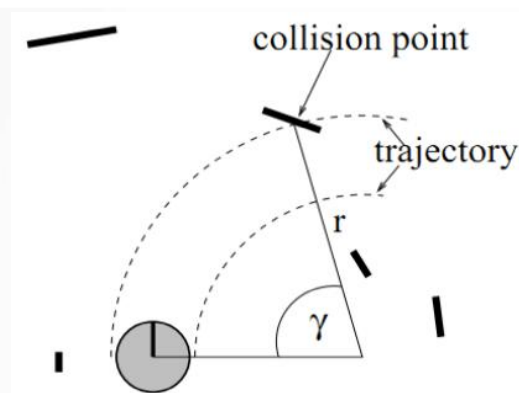
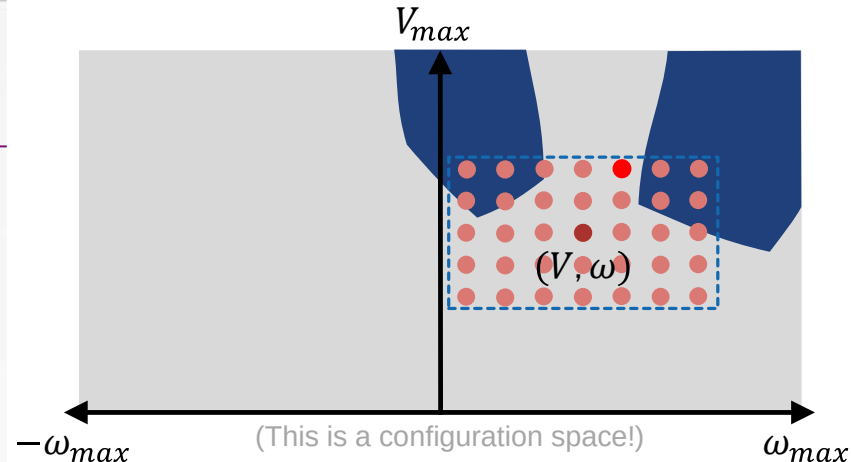
$$V_d = \left\{ (v, \omega) \left| \begin{array}{l} v \in [v_c - \dot{v}_b \Delta t, v_c + \dot{v}_a \Delta t] \wedge \\ \omega \in [\omega_c - \dot{\omega}_b \Delta t, \omega_c + \dot{\omega}_a \Delta t] \end{array} \right. \right\} \text{ (其中 } (v_c, \omega_c) \text{ 为当前速度)}$$

□ 环境约束

$$V_d = \left\{ (v, \omega) \left| v \leq \sqrt{2 \cdot \text{dist}(v, \omega) \cdot \dot{v}_b} \wedge \omega \leq \sqrt{2 \cdot \text{dist}(v, \omega) \cdot \dot{\omega}_b} \right. \right\}$$

其中 $\text{dist}(v, \omega)$ 表示 (v, ω) 对应轨迹上离障碍物最近的距离

注意：这个条件并不是在采样一开始就能得到，需要计算轨迹后判断当前采样的速度能否在碰到障碍物之前停下来，如果能停下来则该速度为可接受admissible，否则抛弃



5.2 DWA基本思路

■ 评价函数:

$$G(v, \omega) = \sigma (\alpha \cdot heading(v, \omega) + \beta \cdot dist(v, \omega) + \gamma \cdot velocity(v, \omega))$$

□ 方向角评价函数 $heading(v, \omega)$

机器人当前设定的采样速度下，达到模拟轨迹末端时的朝向与目标间的夹角

□ 空隙 $dist(v, \omega)$

机器人当前设定的采样速度下，模拟轨迹与最近的障碍物间的距离

□ 速度 $velocity(v, \omega)$

当前采样速度大小（在一次采样中速度大小不变）

■ 指标的归一化处理

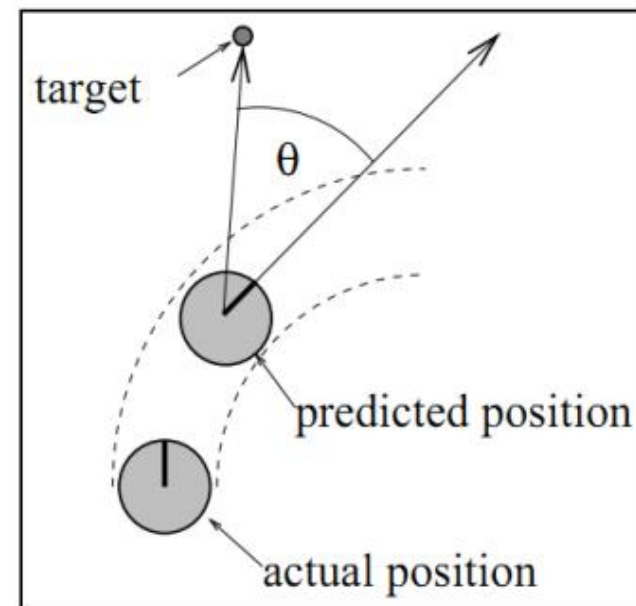
□ 多目标加权求优过程中经常应用的方法

□ 多个优化指标在相对变化量上的影响一致

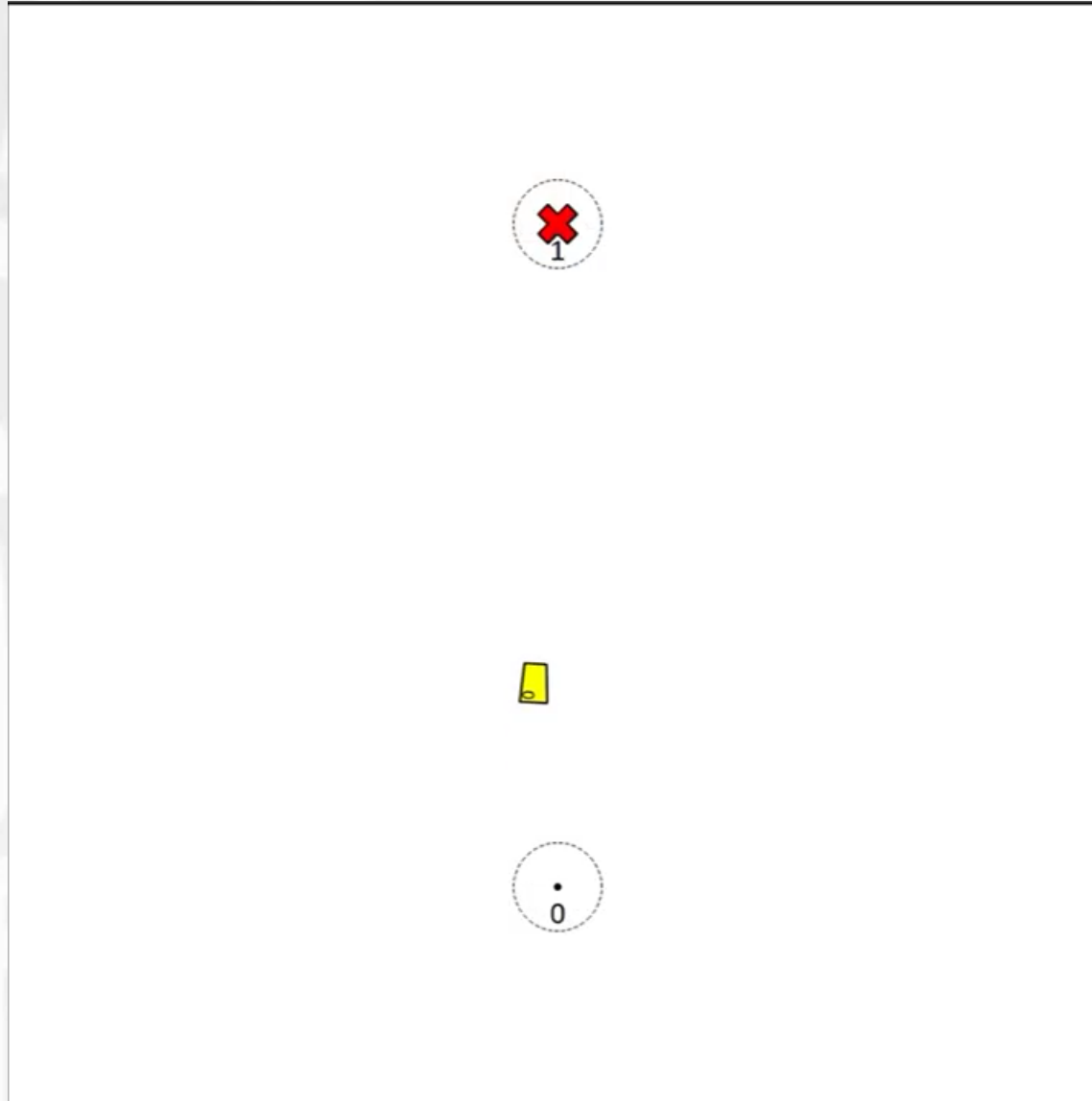
$$Norm_{head(i)} = \frac{heading(i)}{\sum_{i=1}^n heading(i)}$$

$$Norm_{dist(i)} = \frac{dist(i)}{\sum_{i=1}^n dist(i)}$$

$$Norm_{velocity(i)} = \frac{velocity(i)}{\sum_{i=1}^n velocity(i)}$$



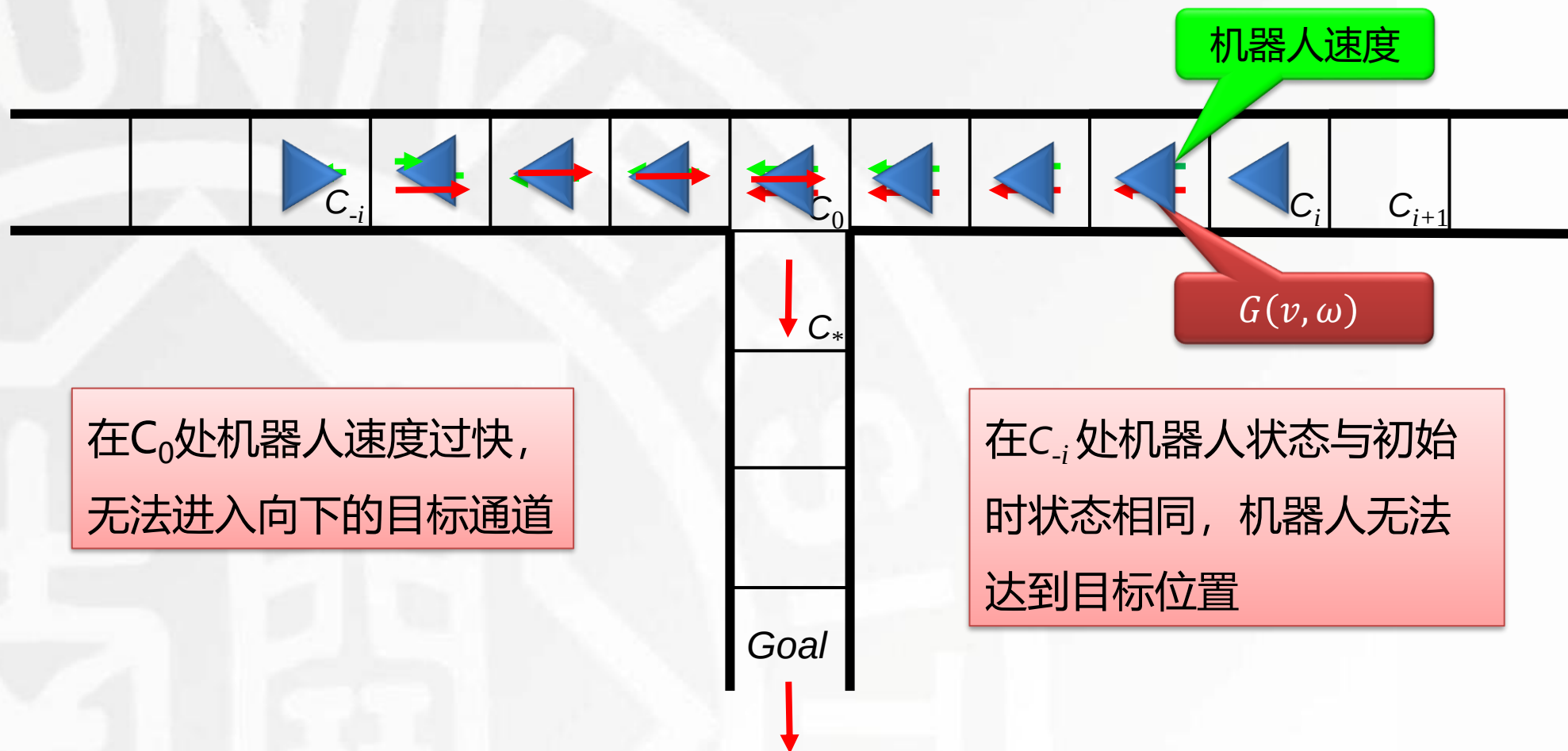
5.2 DWA



5.2 DWA算法总结

- 可以快速根据环境信息作出反应
 - 可有效避免机器人与环境发生碰撞
 - 计算复杂度低，对硬件计算资源需求低
 - 具有较高的实用性，可有效地在实际应用中使用
-
- 计算结果与选择权重相关，有可能得不到全局最优解
 - 局部最优的结果有可能使机器人陷入死锁，无法达到目标点

5.2 DWA算法问题



$$G(v, \omega) = \sigma (\alpha \cdot heading(v, \omega) + \beta \cdot dist(v, \omega) + \gamma \cdot velocity(v, \omega))$$

5.2 DWA算法问题

■ DWA算法在实际应用过程中的问题:

- 可预见性差，在接近门口的位置速度过快



5.3.1 TEB (Timed Elastic Band)核心思想

TEB核心思想:

- 连接起始、目标点，并让这个路径可以变形，变形的条件就是将所有约束当做橡皮筋的外力。
- **Elastic-Band**: 起始点、目标点状态由用户/全局规划器指定，中间插入N个控制橡皮筋形状的控制点（机器人位姿）

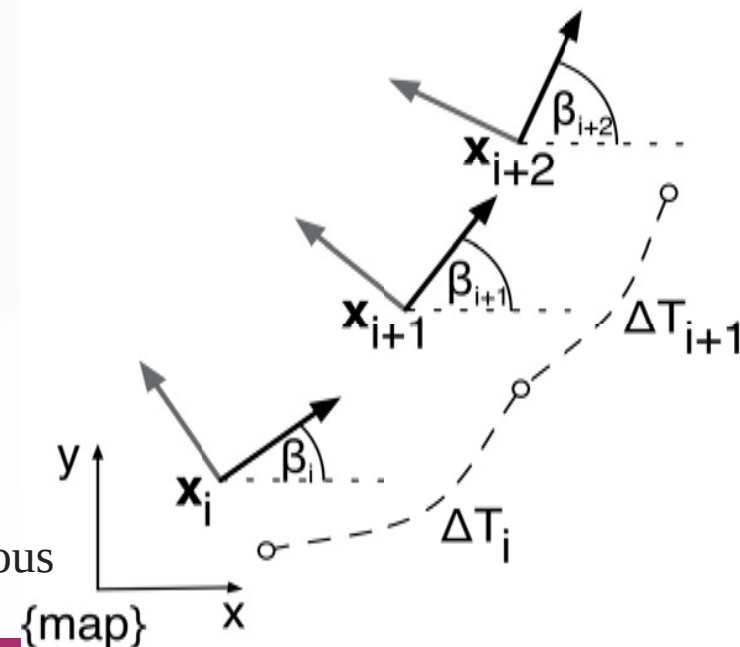
$$Q = \{X_i \mid X_i = \{x_i, y_i, \beta_i\}^T \in \mathbb{R}^2 \times \mathcal{S}^1\}_{i=0, \dots, n}$$

- **Timed-Elastic-Band**: 为控制橡皮筋形状的控制点赋予时间 $\tau = \{\Delta T_i\}_{i=0, \dots, n-1}$

$$B := (Q, \tau)$$

- 通过加权多目标优化获取最优的路径点，即最优的B

$$f(B) = \sum_k \gamma_k f_k(B) \Rightarrow B^* = \operatorname{argmin}_B f(B)$$



C. Roesmann, et al., Trajectory modification considering dynamic constraints of autonomous robots, ROBOTIK 2012, 7th German Conference on Robotics, 74-79

5.3.2 TEB约束目标函数组成

TEB约束目标函数(1):

■ 跟随路径和避障约束

$$d_{\min,j} \leq r_{p_{\max}}, \quad d_{\min,j} \geq r_{o_{\max}}$$

其中, $d_{\min,j}$ 为与第 j 个全局路径点或第 j 个障碍物点 z_j 间的最小距离, $r_{p_{\max}}$ 为距离全局路径允许的最大偏差, $r_{o_{\max}}$ 为距障碍物的允许的最小距离

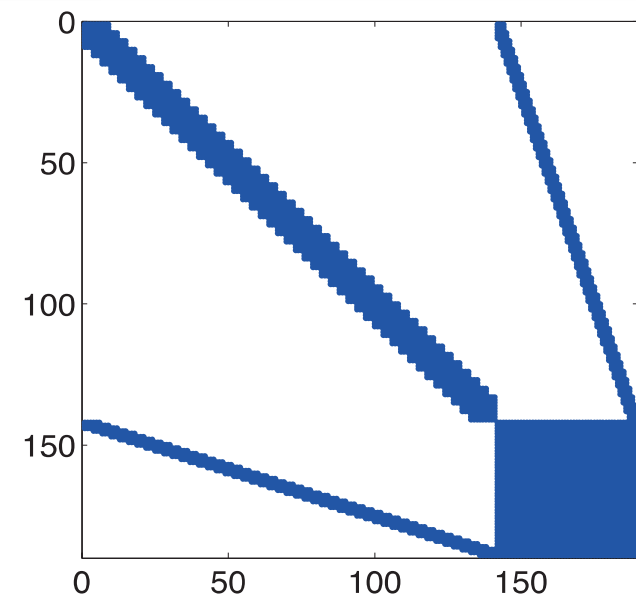
- 不等式约束不便于优化——等式约束
- 每个目标函数只与elastic band中的某几个连续状态有关, 而非整条band。故 J 为稀疏矩阵
- For 47 configurations, the first 141 states corresponds to 47 configurations X_i and states 142-189 are the related time differences ΔT_i .
- 采用分段连续, 可微分的代价函数计算破坏约束的惩罚值

$$e_{\Gamma}(x, x_r, \epsilon, S, n) \simeq \begin{cases} \left(\frac{x - (x_r - \epsilon)}{S} \right)^n & \text{if } x > x_r - \epsilon \\ 0 & \text{otherwise} \end{cases}$$

其中, S 表示缩放, n 表示多项式阶数, ϵ 表示限界值邻域范围

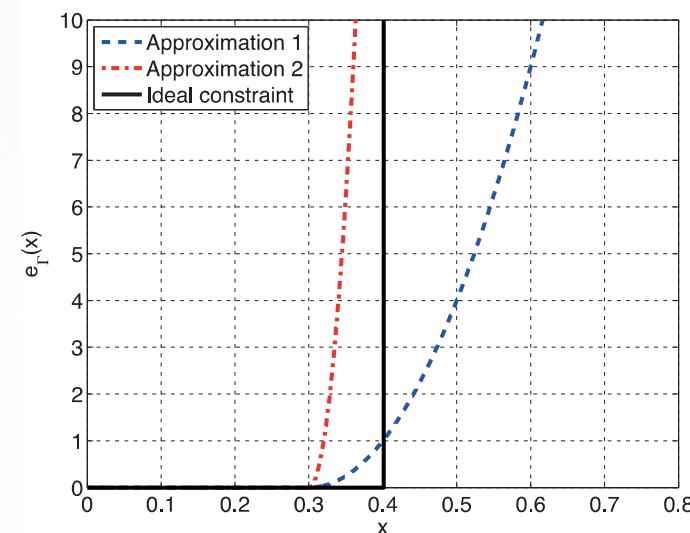
$$f_{\text{path}} = e_{\Gamma}(d_{\min,j}, r_{p_{\max}}, \epsilon, \mathbf{S}, \mathbf{n}), \quad f_{\text{ob}} = e_{\Gamma}(-d_{\min,j}, -r_{o_{\max}}, \epsilon, \mathbf{S}, \mathbf{n})$$

注意: 目标函数的梯度可视为对elastic band施加的外力



nz = 5481

TEB system-matrix is still sparse with 15 percent of non-zero elements.



an approximation of the constraint $x_r = 0.4$.

5.3.2 TEB约束目标函数组成

TEB约束目标函数(2):

■ 速度和加速度约束

$$v_i \simeq \frac{1}{\Delta T_i} \left\| \begin{pmatrix} x_{i+1} - x_i \\ y_{i+1} - y_i \end{pmatrix} \right\|, \quad \omega \simeq \frac{\beta_{i+1} - \beta_i}{\Delta T_i}$$

$$a_i \simeq \frac{2(v_{i+1} - v_i)}{\Delta T_i + \Delta T_{i+1}}, \quad \dot{\omega}_i \simeq \frac{2(\omega_{i+1} - \omega_i)}{\Delta T_i + \Delta T_{i+1}}$$

□ 两轮差速机器人运动学:

$$v_{\omega_r, i} = v_i + L\omega_i, \quad v_{\omega_l, i} = v_i - L\omega_i$$

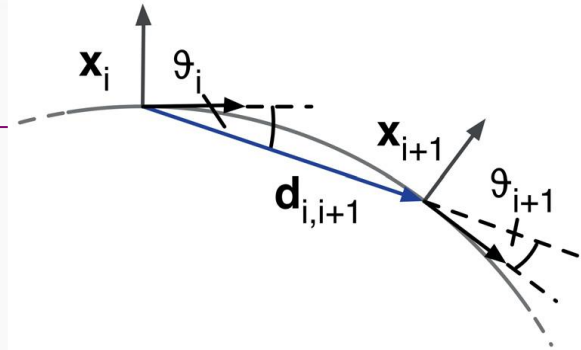
□ 电机速度加速度限制:

$$-v_{\max} \leq f_v(B) \leq v_{\max}, \quad -a_{\max} \leq f_a(B) \leq a_{\max}$$

□ 不等式约束不便于优化——等式约束

$$f_v(B) = e_{\Gamma}(-f_v(B), -v_{\max}, \epsilon, S, n) \quad f_a(B) = e_{\Gamma}(-f_a(B), -a_{\max}, \epsilon, S, n)$$

5.3.2 TEB约束目标函数组成



TEB约束目标函数(3):

■ Non-holonomic运动学约束

- 两轮差速机器人在平面运动只有两个自由度，运动学约束使得机器人以若干弧段组成的平滑的轨迹运动
- 相邻的两个构型应在弧段的两端: 构型 X_i 与运动方向 $d_{i,i+1}$ 的夹角 θ_i 应与构型 X_{i+1} 与运动方向 $d_{i+1,i+2}$ 的夹角 θ_{i+1} 相等

$$\theta_i = \theta_{i+1} \Rightarrow \begin{pmatrix} \cos \beta_i \\ \sin \beta_i \\ 0 \end{pmatrix} \times d_{i,i+1} = d_{i,i+1} \times \begin{pmatrix} \cos \beta_{i+1} \\ \sin \beta_{i+1} \\ 0 \end{pmatrix} \quad d_{i,i+1} := \begin{pmatrix} x_{i+1} - x_i \\ y_{i+1} - y_i \\ 0 \end{pmatrix}$$

$$f_k(X_i, X_{i+1}) = \left\| \left[\begin{pmatrix} \cos \beta_i \\ \sin \beta_i \\ 0 \end{pmatrix} + \begin{pmatrix} \cos \beta_{i+1} \\ \sin \beta_{i+1} \\ 0 \end{pmatrix} \right] \times d_{i,i+1} \right\|^2$$

要注意机器人运动在180度附近的越界的情况

□ 转弯半径约束:

$$r(x_k, x_{k+1}) = \left| \frac{v_k}{\omega_k} \right| \simeq \frac{\|d_{i,i+1}\|}{|\beta_{i+1} - \beta_i|}, \quad f_r(X_i, X_{i+1}) = e_\Gamma(r(x_k, x_{k+1}), r_{\max}, \epsilon, \mathbf{S}, \mathbf{n})$$

■ 最小时间约束

$$f_t = \left(\sum_{i=1}^n \Delta T_i \right)^2$$

5.3.2 TEB算法优化思路

$$Q = \{X_i | X_i = \{x_i, y_i, \beta_i\}^T \in \mathbb{R}^2 \times \mathcal{S}^1\}_{i=0, \dots, n}$$

$$\tau = \{\Delta T_i\}_{i=0, \dots, n-1}$$

$$B := (Q, \tau)$$

$$f(B) = \sum_k \gamma_k f_k(B) \Rightarrow B^* = \operatorname{argmin}_B f(B)$$

$$f_{path} = f_{path}(x_p)$$

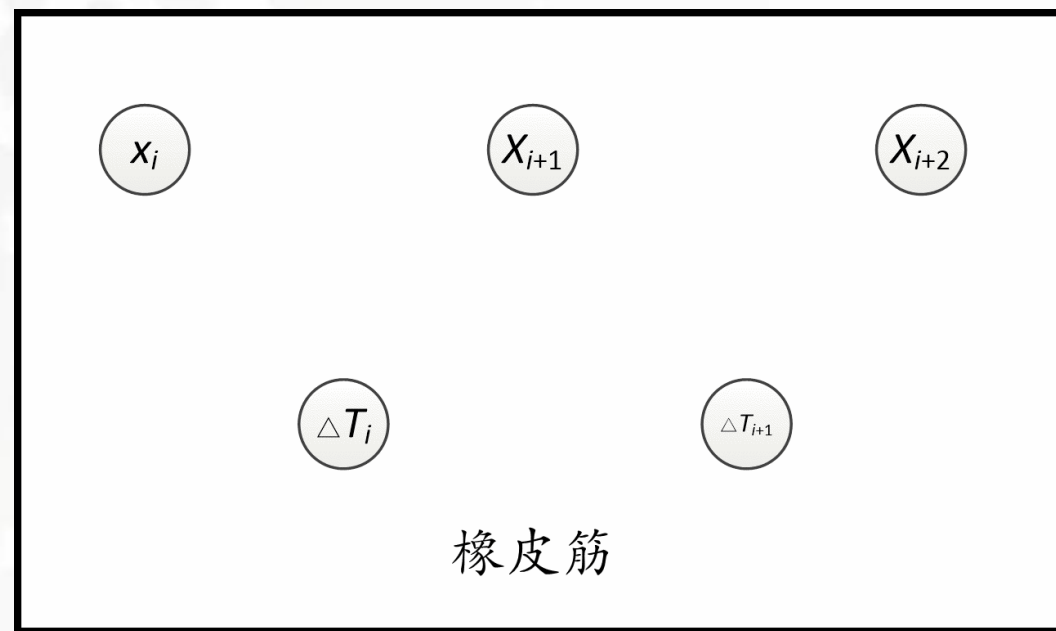
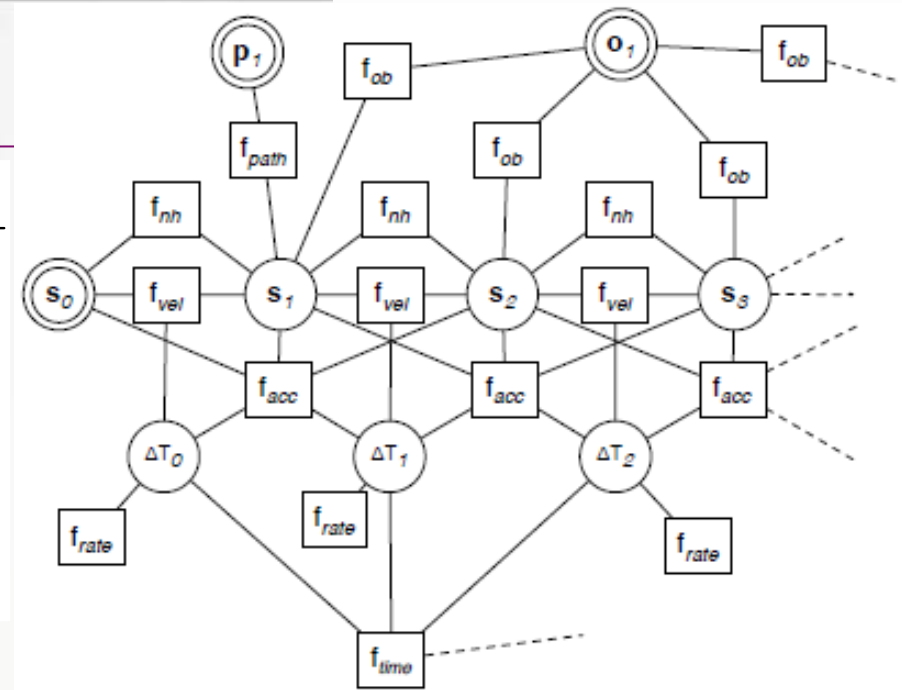
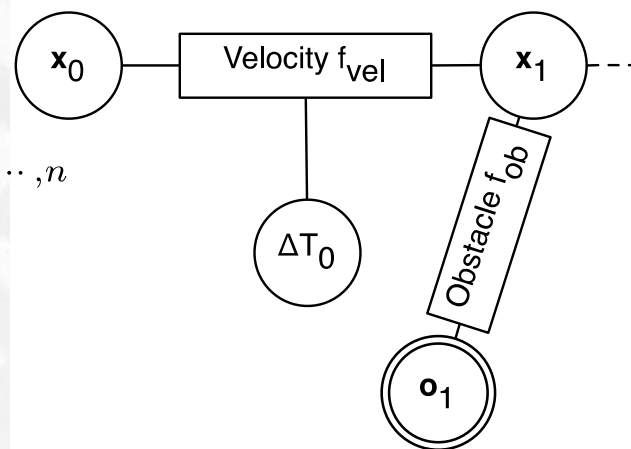
$$f_{ob} = f_{ob}(z_k)$$

$$f_v = f_v(X_i, X_{i+1}, \Delta T_i)$$

$$f_a = f_a(X_i, X_{i+1}, X_{i+2}, \Delta T_i, \Delta T_{i+1})$$

$$f_r = f_r(X_i, X_{i+1})$$

$$f_t = \left(\sum_{i=1}^n \Delta T_i \right)^2$$



5.3.4 TEB算法流程

TEB约束目标函数:

■ 跟随路径和避障约束

$$f_{path} = e_{\Gamma}(d_{\min,j}, r_{p_{\max}}, \epsilon, S, n)$$

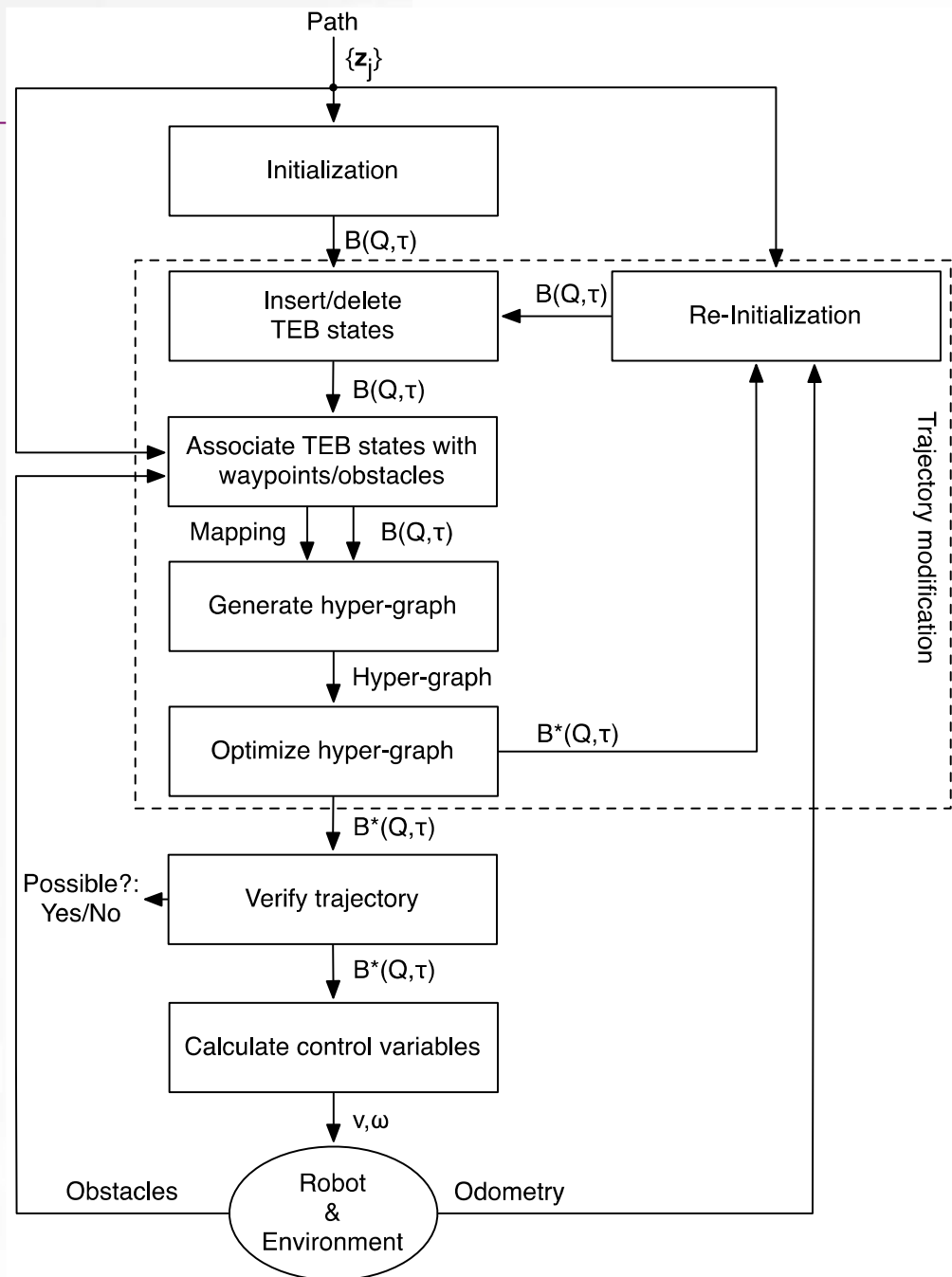
$$f_{ob} = e_{\Gamma}(-d_{\min,j}, -r_{p_{\max}}, \epsilon, S, n)$$

其中, $d_{\min,j}$ 为与第 j 个全局路径点或第 j 个障碍物点 z_j 间的最小距离, $r_{p_{\max}}$ 为距离全局路径允许的最大偏差, $r_{o_{\max}}$ 为距障碍物的允许的最小距离

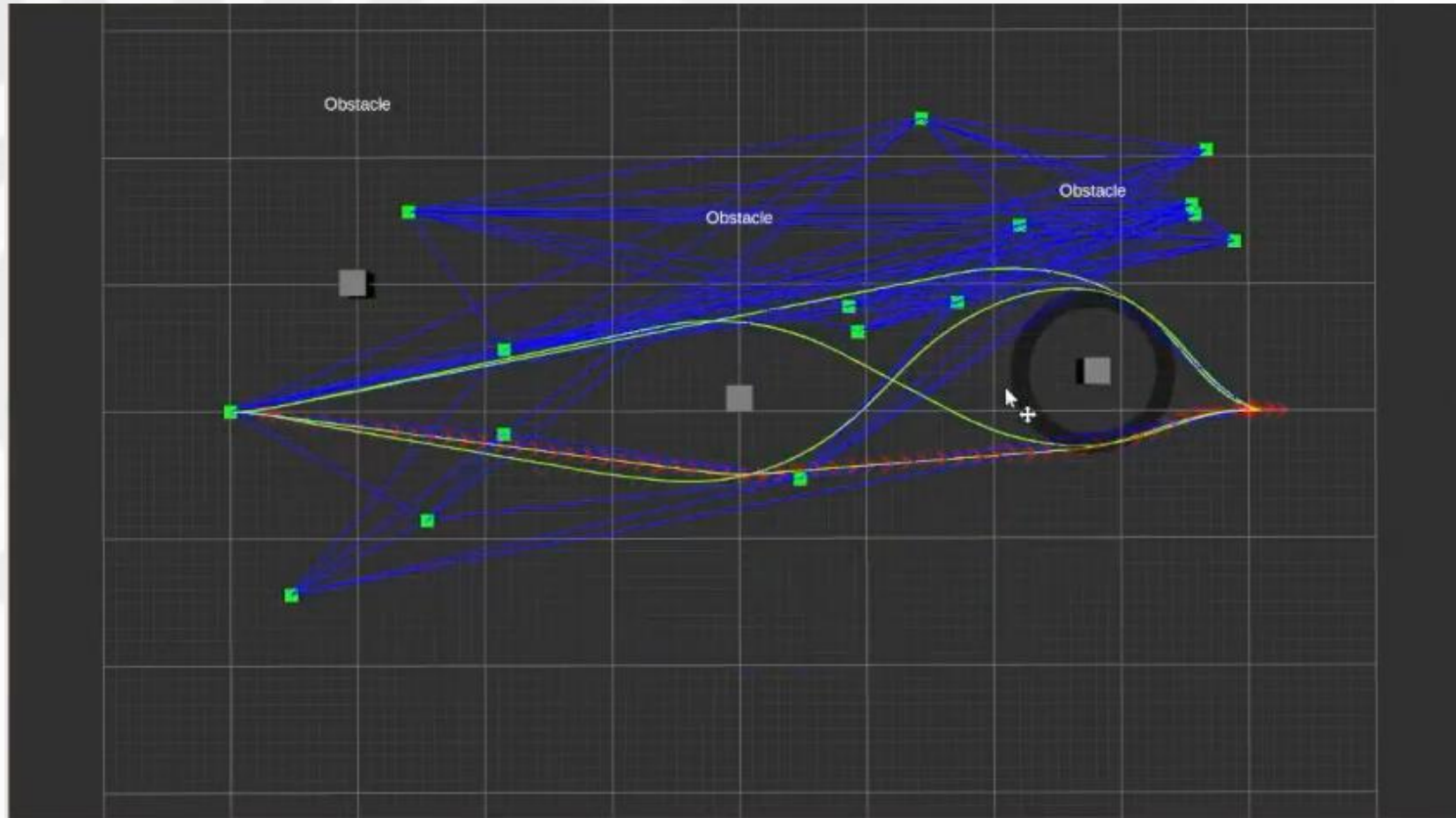
注意: 目标函数的梯度可视为对elastic band施加的外力

■ 速度和加速度约束

■ 起始点、目标点状态由用户/全局规划器指定, 中间插入N个控制橡皮筋形状的控制点 (机器人位姿)



5.2 TEB



06

路径平滑与轨迹规划算法

Path Smoothing and Trajectory Planning

6.1 Path Smoothing

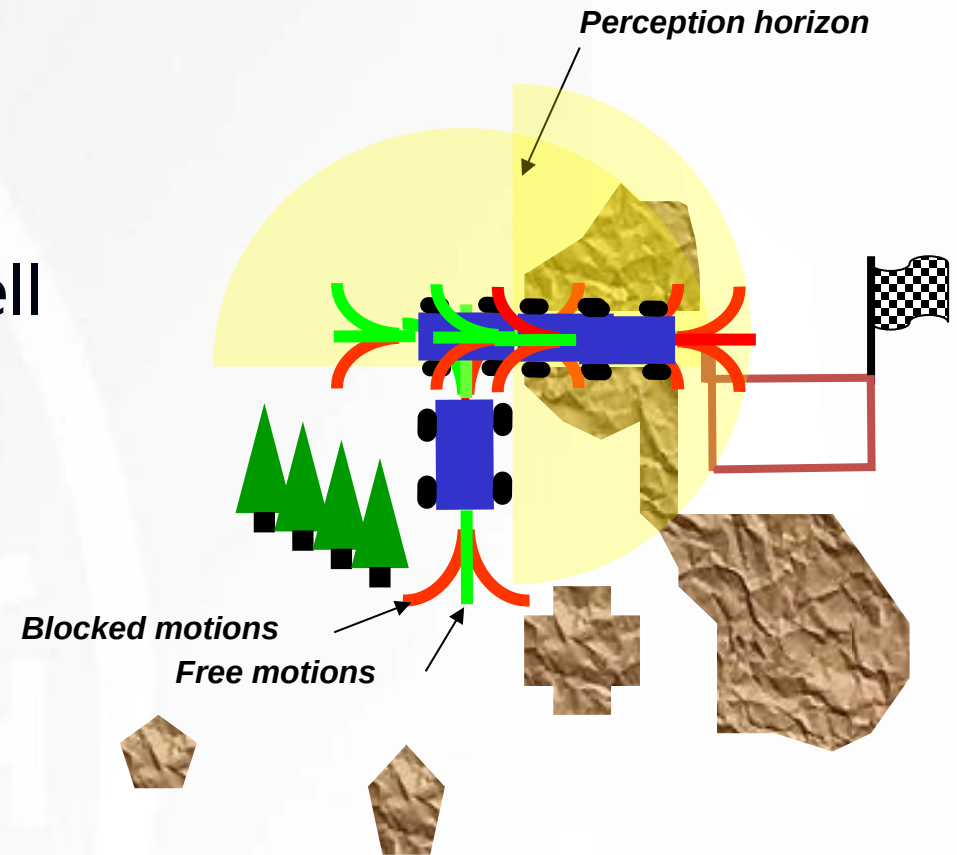
Local Collision-free Path Planning

■ Pros:

- Local motion evaluation is fast
- In sparse obstacles, works very well

■ Cons:

- Search space differences
- Local mini



6.1.1 Path Smoothing

■ Performance Requirement

- Efficiency
- Precision

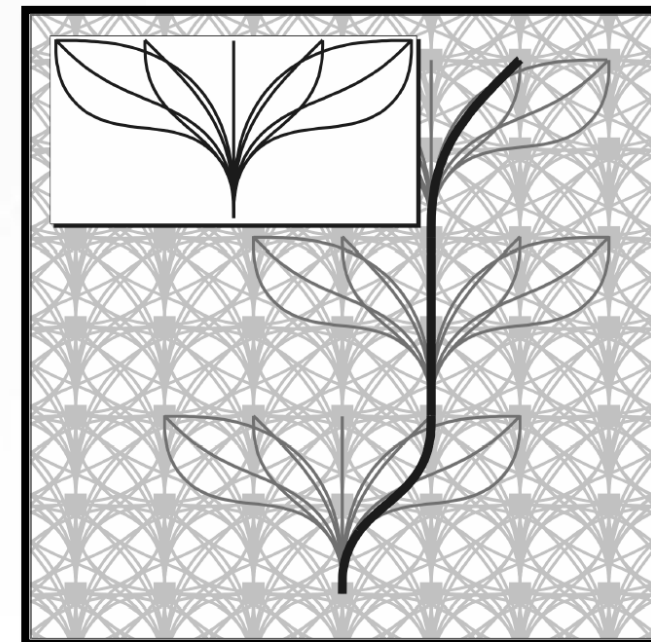
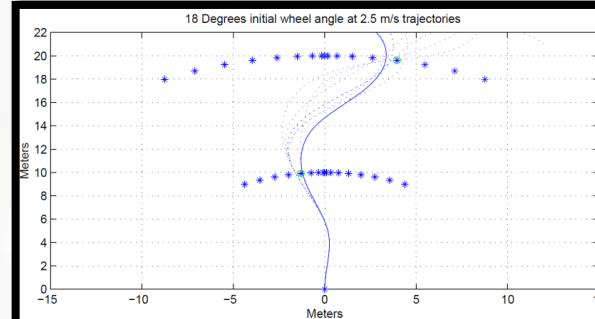
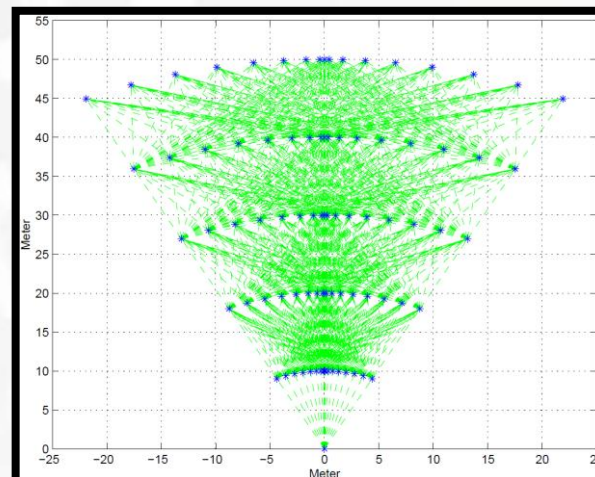
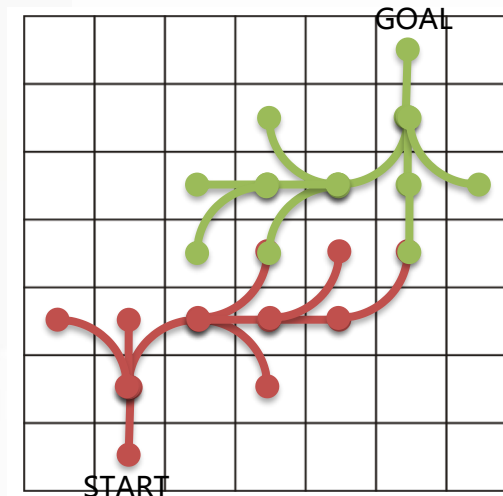
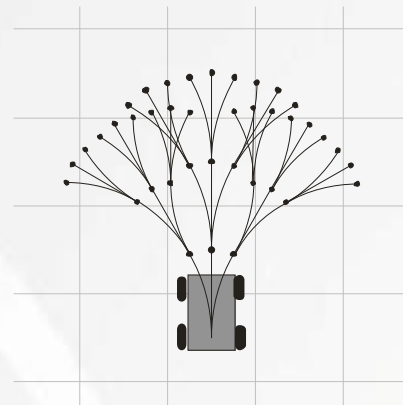
■ Constraints by robot

- Degree of Maneuverability
- Kinematics
- The capacity of actuators

■ Moves with the robot

■ Fixed to the world

■ Circle is suitable?



6.1.2 路径光滑的基本约束

■ 初始、终止状态约束

- 初始、终止位置——曲线的起始、终止位置
- 初始、终止姿态——曲线的起始、终止位置的切线方向

■ 曲线光滑：

□ 曲线连续

- ✓ 机器人物理属性

□ 曲率连续

- ✓ 机器人运动学约束：非完整性属性

□ 曲率导数连续

- ✓ 执行器约束：电机输出力矩连续：机器人加速度连续

□ 曲率导数最小

- ✓ 机器人执行精度要求：角加速度最小，避免机器人侧滑

6.1.3 Kinematics Constraints

■ 差速机器人运动学模型

$$v = \frac{\dot{\phi}_r + \dot{\phi}_l}{2} r, \quad \omega = \frac{\dot{\phi}_r - \dot{\phi}_l}{2l} r$$

■ 瞬时旋转中心 (ICR: Instantaneous Center of Rotation)

$$R = \frac{\dot{\phi}_r + \dot{\phi}_l}{\dot{\phi}_r - \dot{\phi}_l} l = \frac{v}{\omega}$$

■ 曲率

$$\rho = \frac{\omega}{v} = \frac{|\ddot{y}|}{(1 + \dot{y}^2)^{\frac{3}{2}}} \quad for \quad y = f(x)$$

$$\rho = \frac{|\dot{p}(t)\ddot{q}(t) - \dot{q}(t)\ddot{p}(t)|}{(\dot{p}(t)^2 + \dot{q}(t)^2)^{\frac{3}{2}}} \quad for \quad \begin{cases} x = p(t) \\ y = q(t) \end{cases}$$

6.1.4 曲线形态 (1) ——圆弧

- 已知机器人初始位姿及停止位姿分别为: $x_0 = \begin{bmatrix} P_{x_0} \\ P_{y_0} \\ \theta_0 \end{bmatrix}$, $x_f = \begin{bmatrix} P_{x_f} \\ P_{y_f} \\ \theta_f \end{bmatrix}$
- 初始、终止状态约束

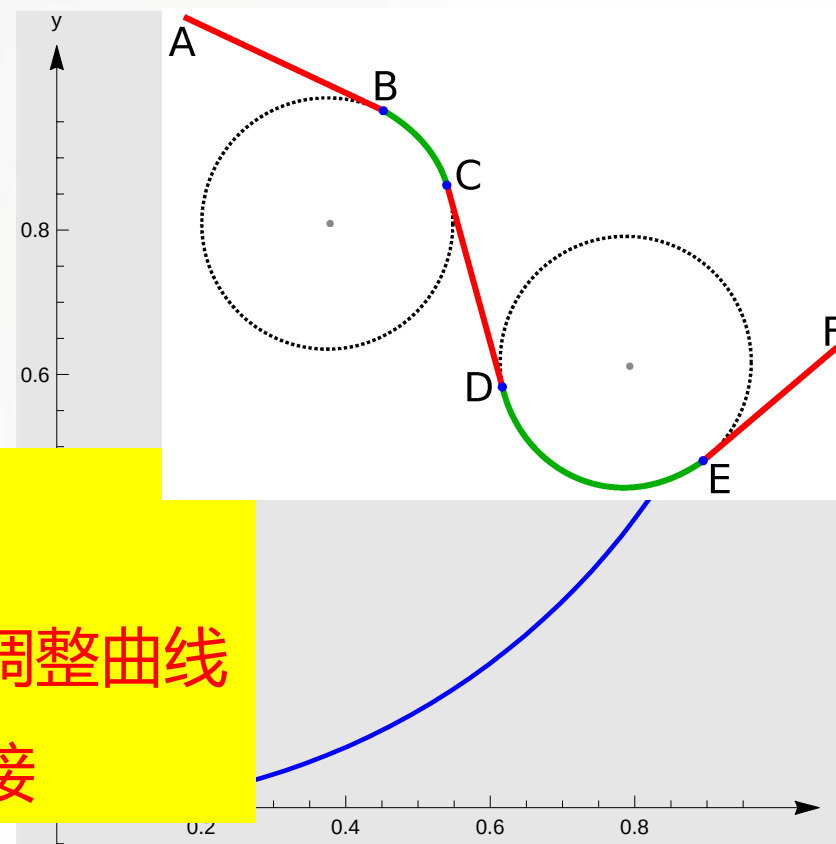
- 初始、终止位置——曲线的起始、终止位置
- 初始、终止姿态——曲线的起始、终止位置的切线方向

- 曲线光滑:

- 曲线连续
- 曲率连续
- 曲率导数连续
- 曲率导数最小

唯一解

- ◆ 无法根据障碍物实际情况调整曲线
- ◆ 无法与其他段轨迹进行衔接



6.1.4 曲线形态 (2) ——贝塞尔曲线

■ 一阶贝塞尔:

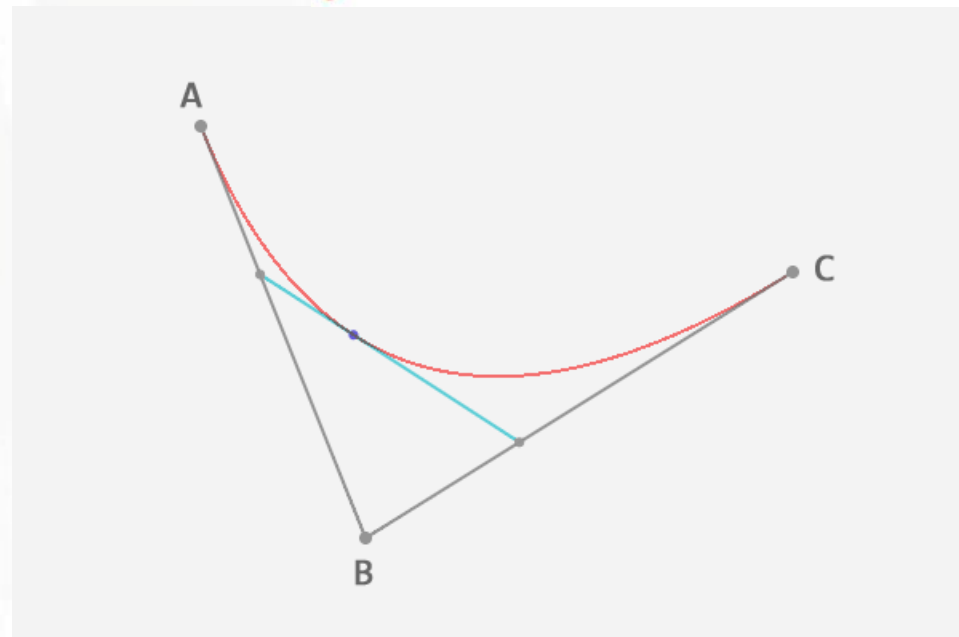
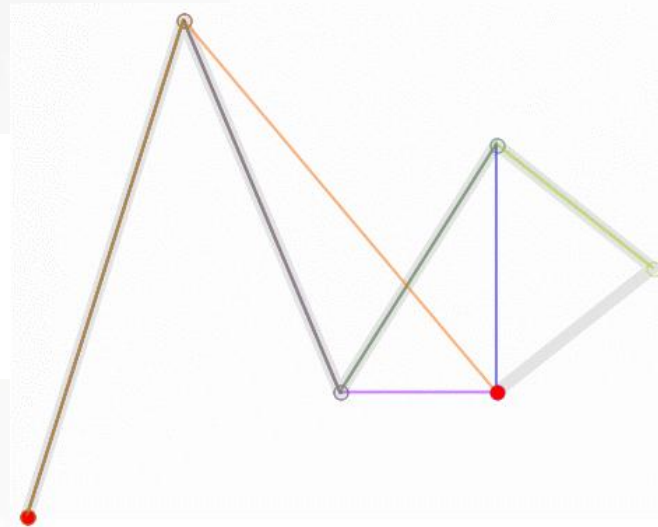
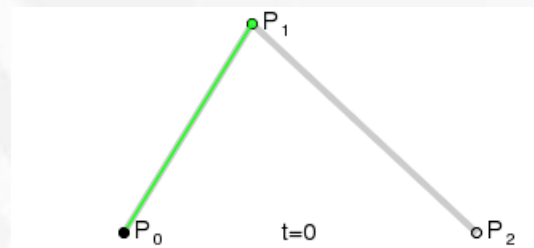
$$\mathbf{B}(t) = (1 - t)\mathbf{P}_0 + t\mathbf{P}_1$$

■ 二阶贝塞尔:

$$\mathbf{B}(t) = (1 - t)^2\mathbf{P}_0 + 2(1 - t)t\mathbf{P}_1 + t^2\mathbf{P}_2$$

■ 高阶贝塞尔:

$$\mathbf{B}(t) = \sum_{i=0}^n \mathbf{C}_n^i \mathbf{P}_i (1 - t)^{n-i} t^i$$



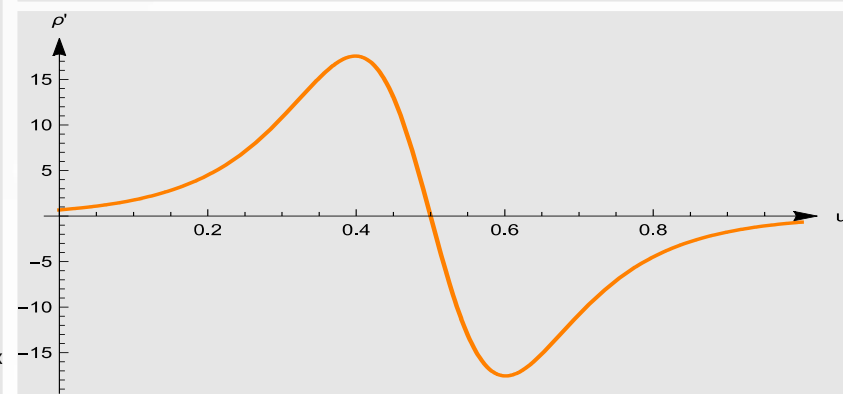
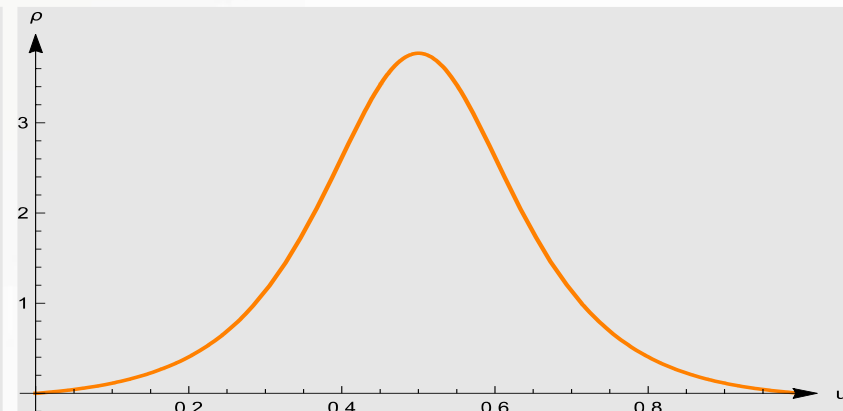
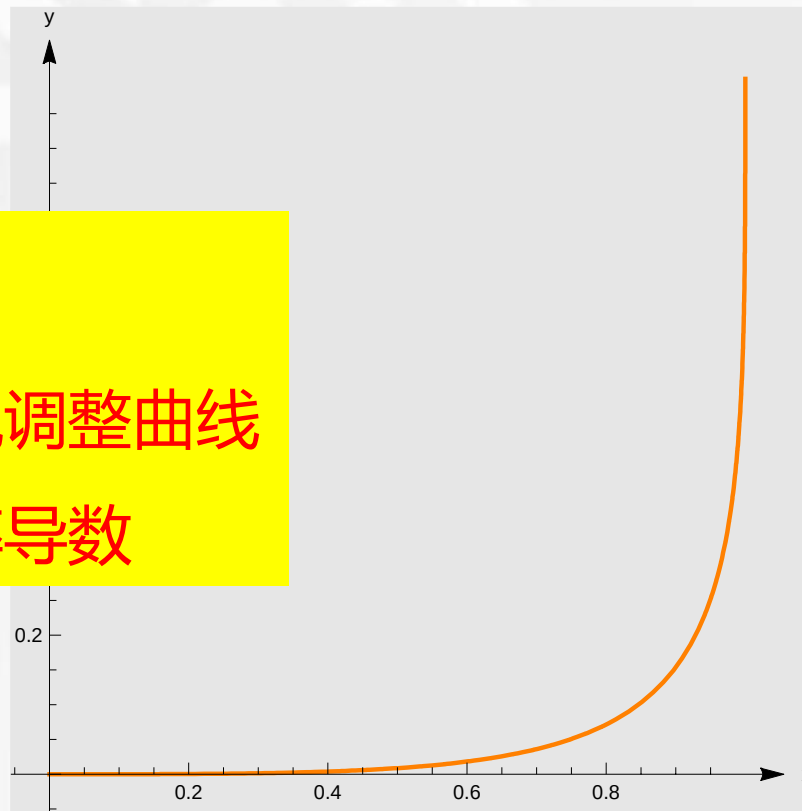
6.1.4 曲线形态 (3) ——三阶贝塞尔曲线

■ 三阶贝塞尔:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3$$

唯一解

- ◆ 无法根据障碍物实际情况调整曲线
- ◆ 无法优化最大曲率及曲率导数



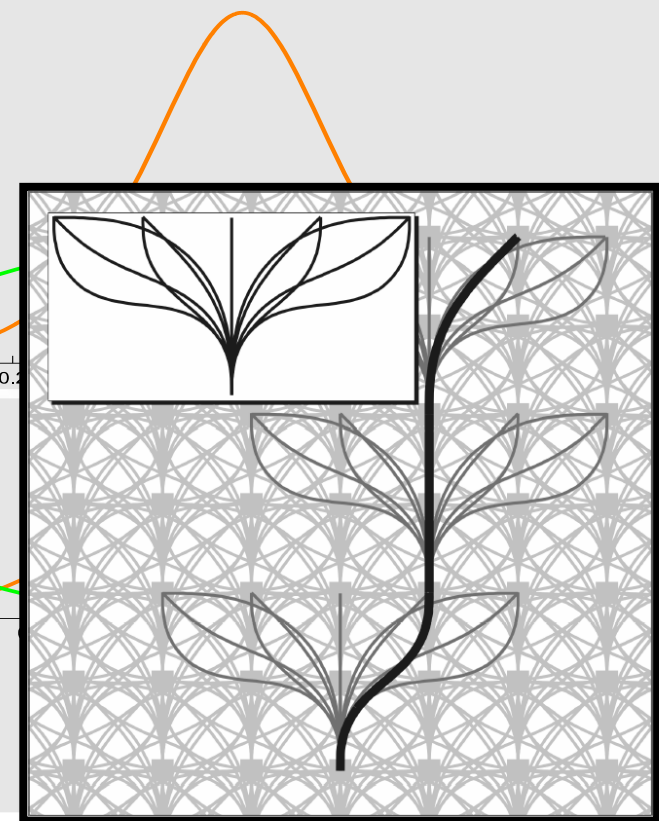
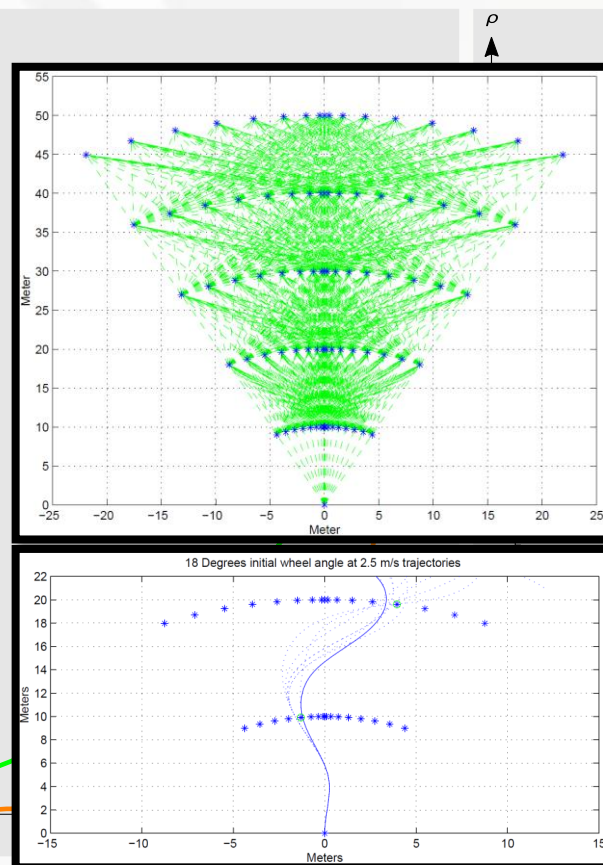
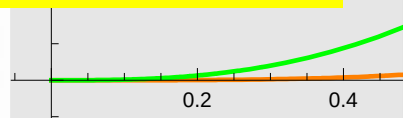
6.1.4 曲线形态 (3) ——五阶贝塞尔曲线

■ 五阶贝塞尔:

$$\mathbf{B}(t) = (1-t)^5 \mathbf{P}_0 + 5(1-t)^4 t \mathbf{P}_1 + 10(1-t)^3 t^2 \mathbf{P}_2 + 10(1-t)^2 t^3 \mathbf{P}_3 + 5(1-t) t^4 \mathbf{P}_4 + t^5 \mathbf{P}_5$$

多解

- ✓ 可以根据障碍物实际情况调整曲线
- ✓ 可以优化最大曲率及曲率导数
- ◆ 求解困难——无法在线运行



6.2.1 移动机器人轨迹规划

■ 移动机器人轨迹规划目标

以给定约束(执行器速度, 加速度限制)的条件下, 给出沿给定的光滑曲线上**可供机器人执行**的运动命令序列。

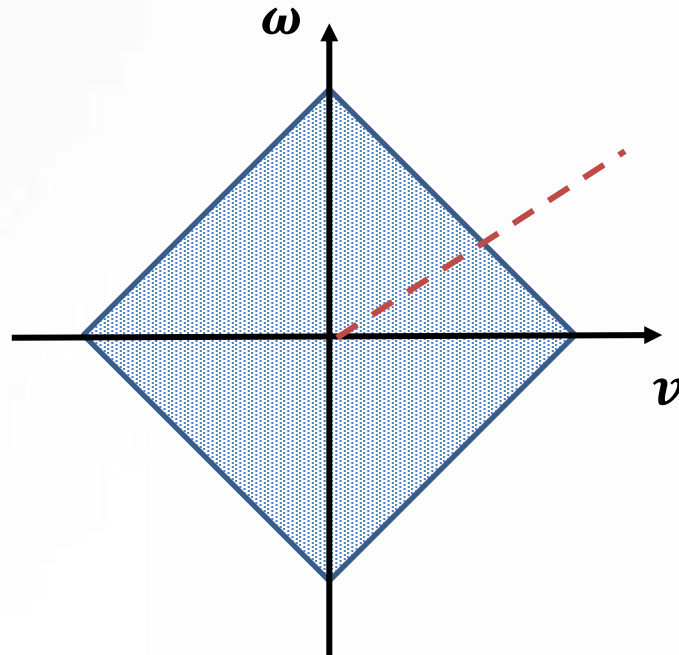
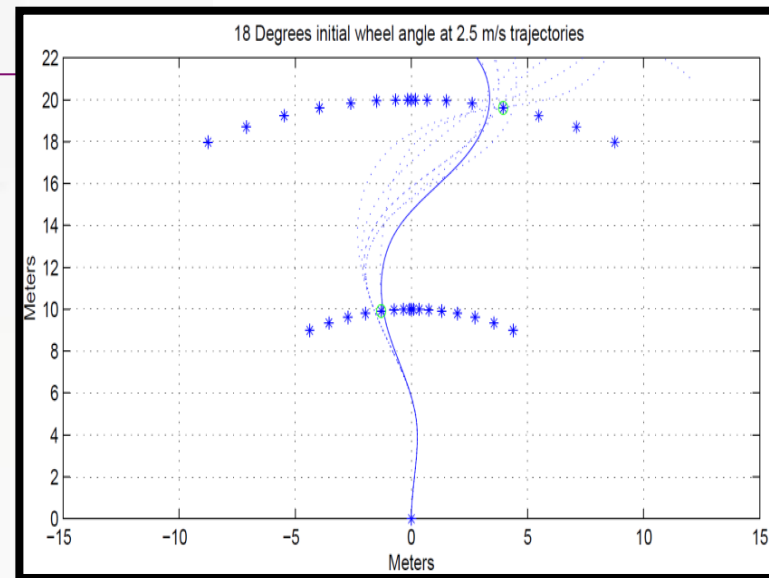
■ 轨迹规划约束条件:

- 初始、终止状态约束: 位置、速度、加速度
- 执行器约束: 机器人车轮速度、加速度限幅 $\dot{\phi}_{\max}, \ddot{\phi}_{\max}$
- 机器人运动学约束: 非完整性属性——线速度与角速度的耦合

$$v = \frac{\dot{\phi}_r + \dot{\phi}_l}{2} r, \quad \omega = \frac{\dot{\phi}_r - \dot{\phi}_l}{2l} r$$
$$R = \frac{\dot{\phi}_r + \dot{\phi}_l}{\dot{\phi}_r - \dot{\phi}_l} l = \frac{v}{\omega}$$

■ 解决思路:

- 给定路径下, 路径上的每个位置的曲率是已知的



6.2.2 移动机器人轨迹规划思路

- Step1: 基于给定路径求取路径上的每个位置对应的曲率

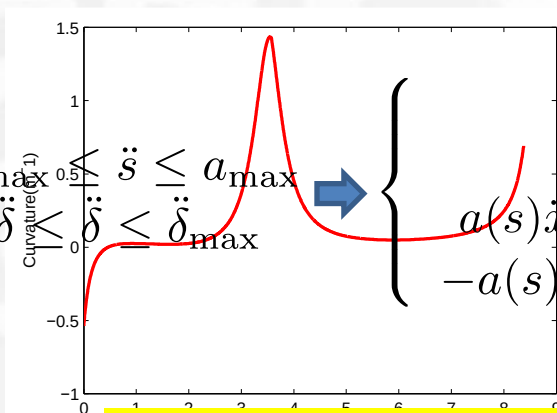
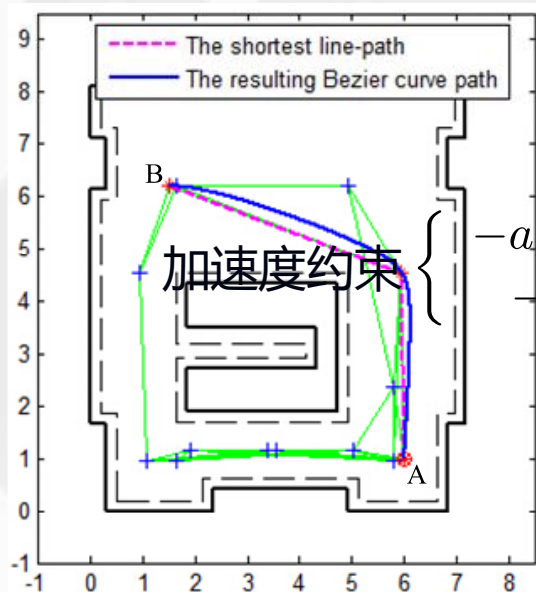
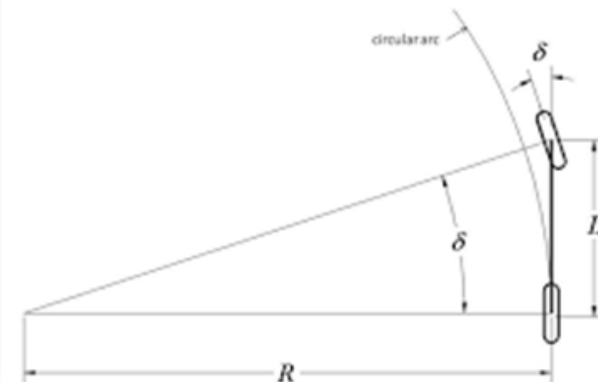
$$\rho = \frac{|\ddot{y}|}{(1 + \dot{y}^2)^{\frac{3}{2}}} \quad \text{for } y = f(x)$$

- Step2: 根据机器人模型, 求取速度上界

- Step3: 根据机器人模型和加速度约束, 求取速度上界MVC(s)

$$\omega = \frac{v}{R} = \frac{v \tan \delta}{L} \quad \omega = k(s)v \Rightarrow \delta = \arctan(Lk(s)) \Rightarrow \ddot{\delta} = \frac{L}{1 + L^2 k^2} \frac{dk}{ds} \ddot{s} + \left[-\frac{2L^3 k}{(1 + L^2 k^2)^2} \left(\frac{dk}{ds} \right)^2 + \frac{L}{1 + L^2 k^2} \frac{d^2 k}{ds^2} \right] \dot{s}^2$$

$$= a(s)\ddot{s} + b(s)\dot{s}^2$$



$$\begin{cases} \ddot{s} - a_{\max} \leq 0 \\ -\ddot{s} - a_{\max} \leq 0 \\ a(s)\ddot{s} + b(s)\dot{s}^2 - \ddot{\delta}_{\max} \leq 0 \\ -a(s)\ddot{s} - b(s)\dot{s}^2 - \ddot{\delta}_{\max} \leq 0 \end{cases}$$

满足约束的最小加速度和最大加速度

$$\alpha(s, \dot{s}) = \max \left(\frac{-\ddot{\delta}_{\max} - b(s)\dot{s}^2}{a(s)}, -a_{\max} \right)$$

$$\beta(s, \dot{s}) = \min \left(\frac{\ddot{\delta}_{\max} - b(s)\dot{s}^2}{a(s)}, a_{\max} \right)$$

为了保证 $\dot{s}$ 存在, 应当要满足 $\alpha(s, \dot{s}) \leq \beta(s, \dot{s})$

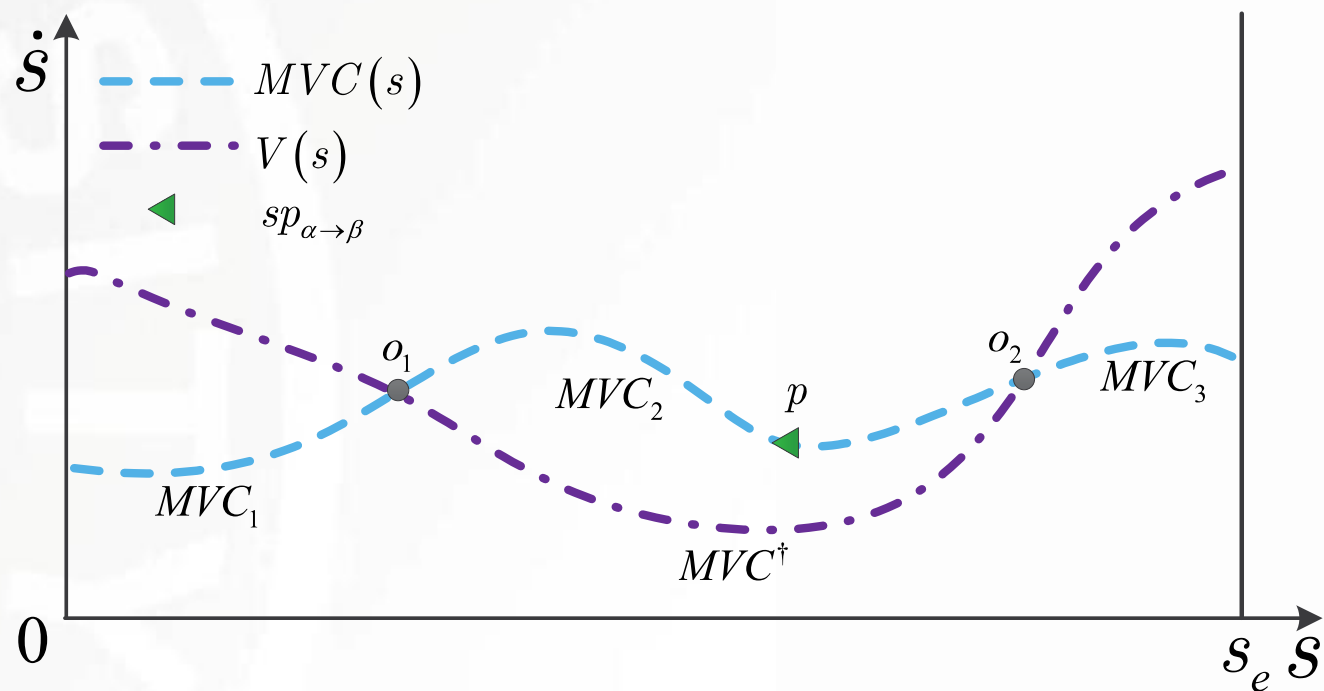
6.2.2 移动机器人轨迹规划思路

根据以上分析, 可知对于 $[0, s_e]$ 上的任意一个 s , 都有一个对应的 $\dot{s}_0$, 当 $\dot{s} > \dot{s}_0$ 时, 加速度将不满足约束条件, 我们将这些点组成的曲线称为最大速度曲线 (MVC), 并且有:

$$MVC(s) = \min\{\dot{s} \geq 0 | \alpha(s, \dot{s}) = \beta(s, \dot{s})\} \quad s \in [0, s_e]$$

进而可得 $MVC(s)$ 的表达式:

$$MVC(s) = \sqrt{\frac{\ddot{\delta}_{\max} + |a(s)|a_{\max}}{|b(s)|}}$$



6.2.2 移动机器人轨迹规划思路

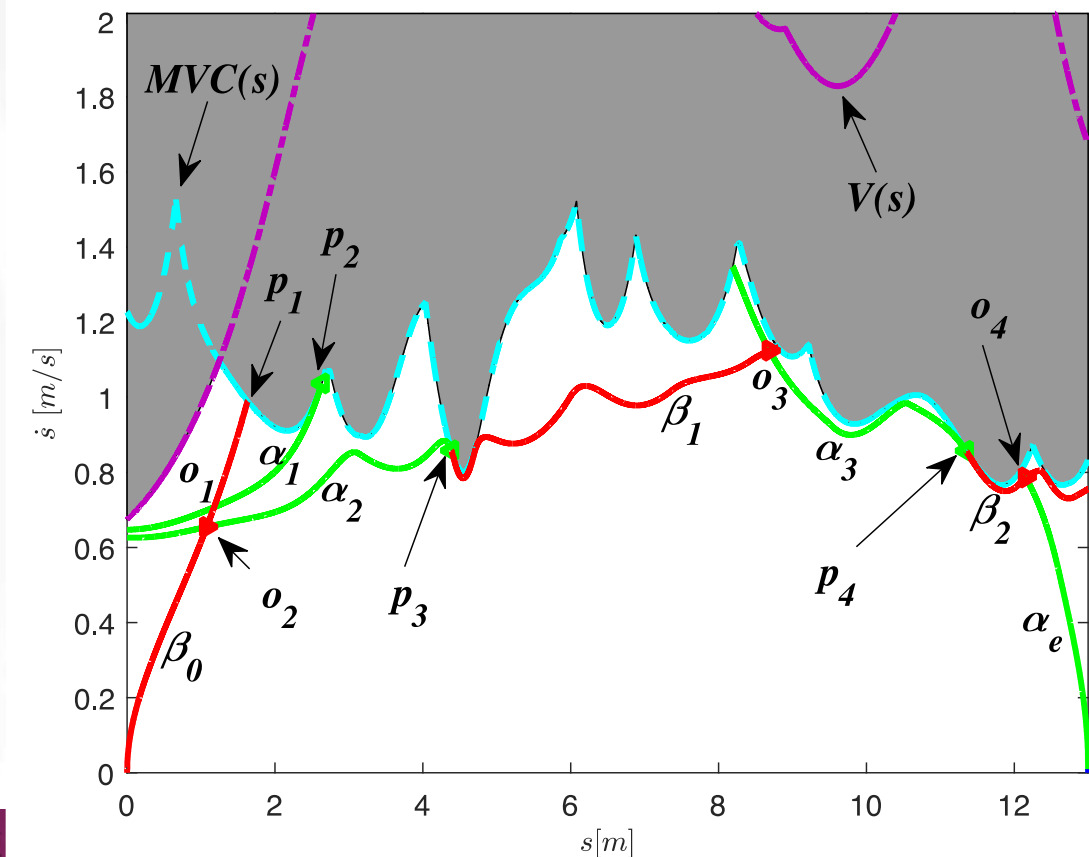
■ Step4 连接起点和终点速度

- 正向搜索：从**起点**开始，以**最大的路径加速度** $\beta(s, \dot{s})$ 进行正向数值积分
- 反向搜索：从**终点**开始，以**最小的路径加速度** $\alpha(s, \dot{s})$ 进行反向数值积分

满足约束的最小加速度 $\alpha(s, \dot{s})$ 和最大加速度 $\beta(s, \dot{s})$

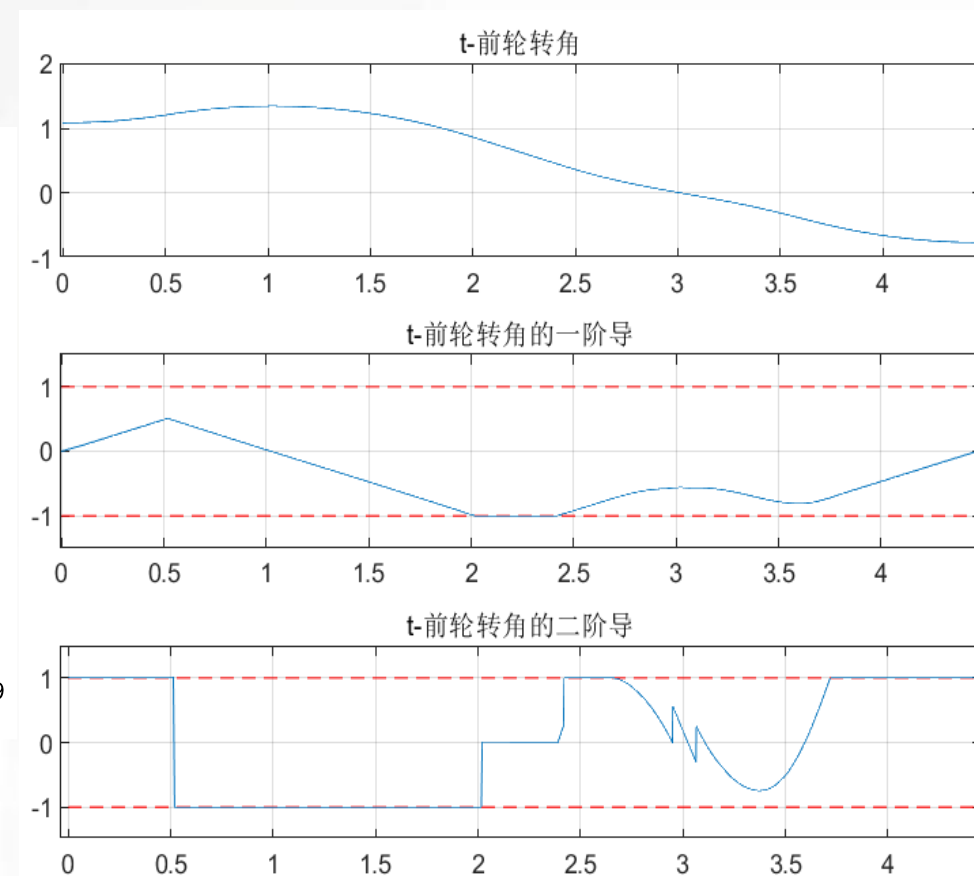
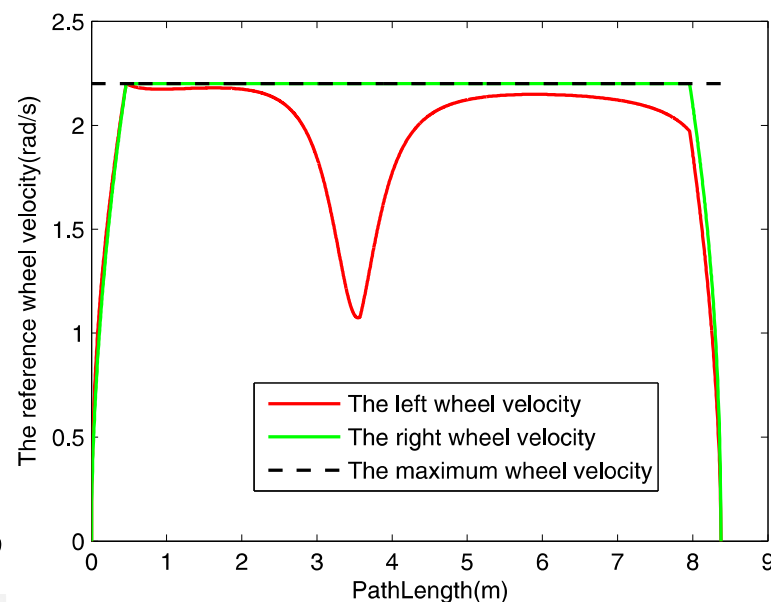
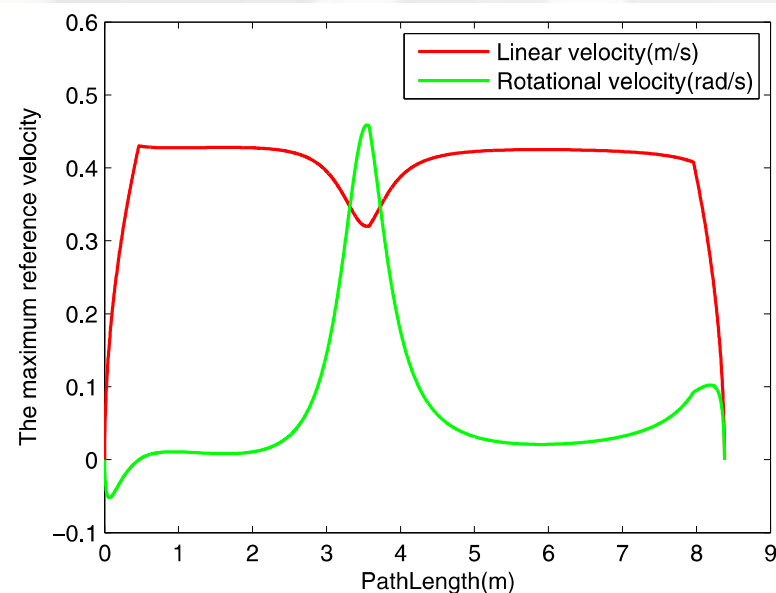
$$\alpha(s, \dot{s}) = \max \left(\frac{-\ddot{\delta}_{\max} - b(s)\dot{s}^2}{a(s)}, -a_{\max} \right)$$

$$\beta(s, \dot{s}) = \min \left(\frac{\ddot{\delta}_{\max} - b(s)\dot{s}^2}{a(s)}, a_{\max} \right)$$

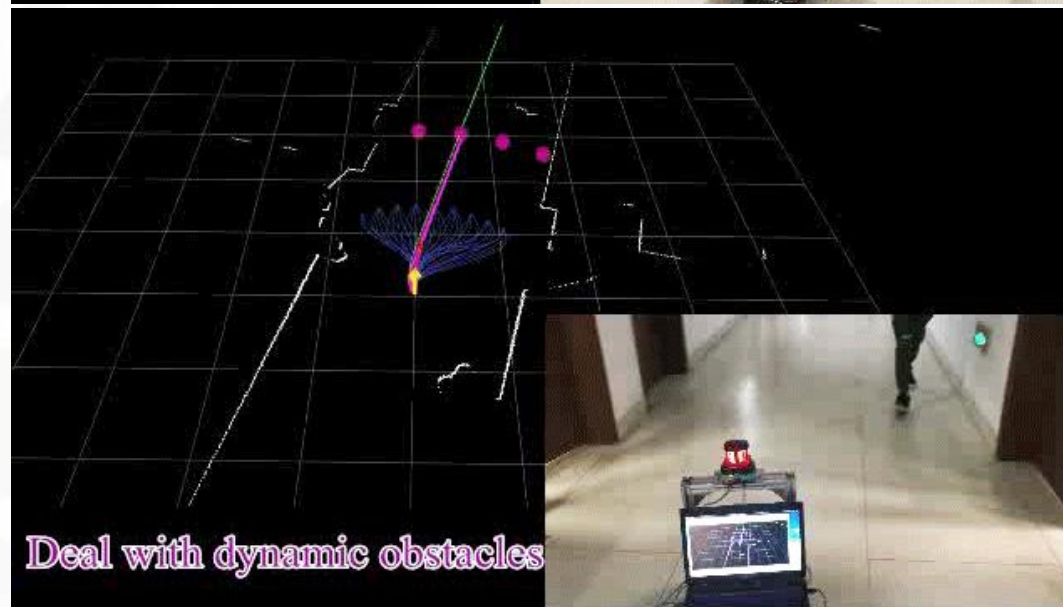
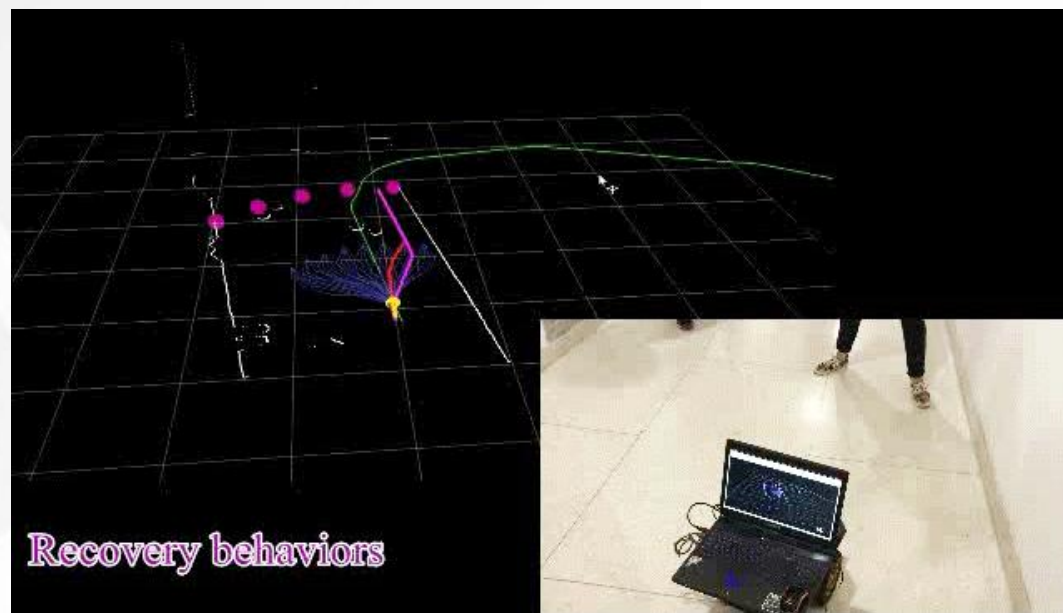
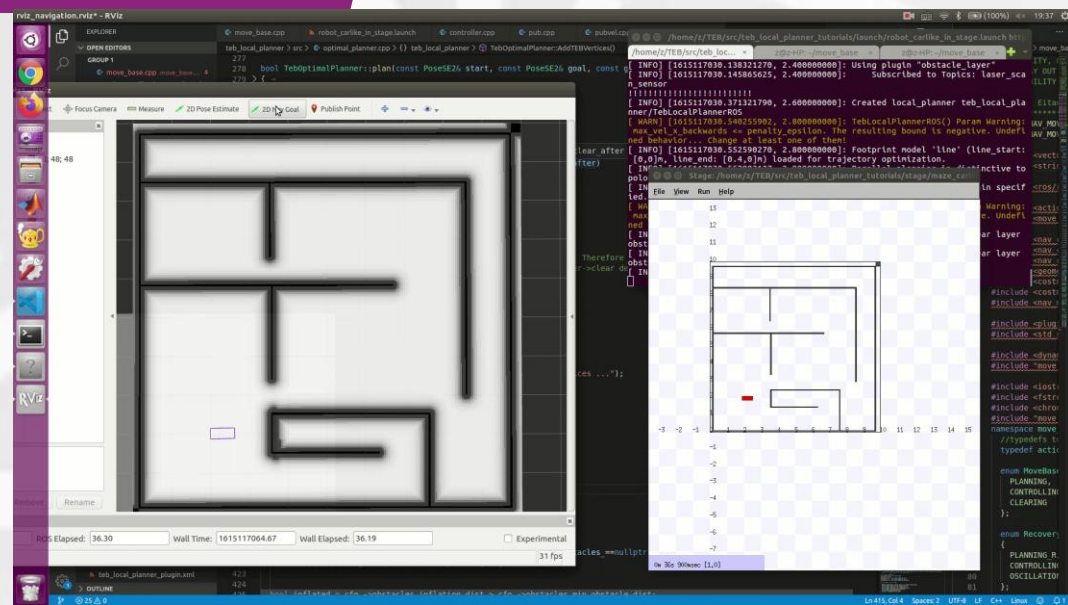


6.2.2 移动机器人轨迹规划思路

- Step5 根据求取的路径上每个点的速度值求取角速度值 $\omega = k(s)v$
- Step6 依据机器人运动学模型，求取机器人执行器控制输入



6.2.3 移动机器人轨迹规划实验



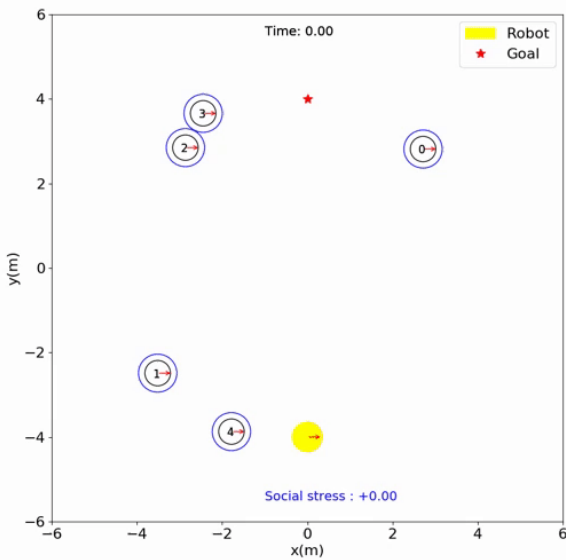
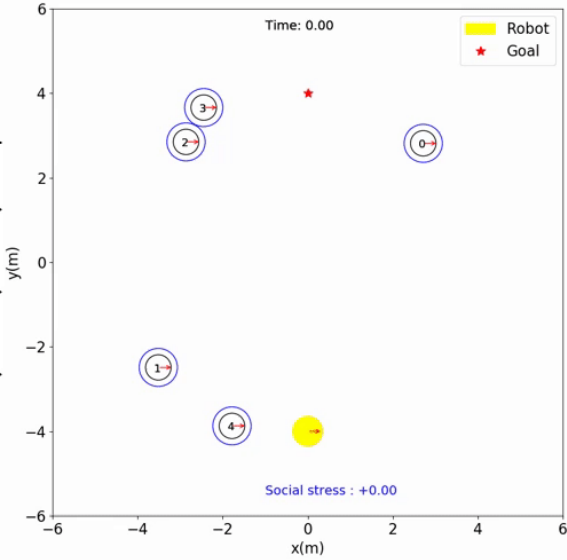
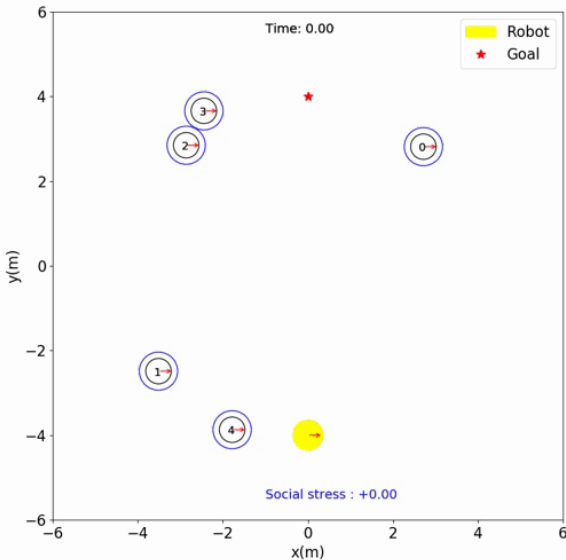
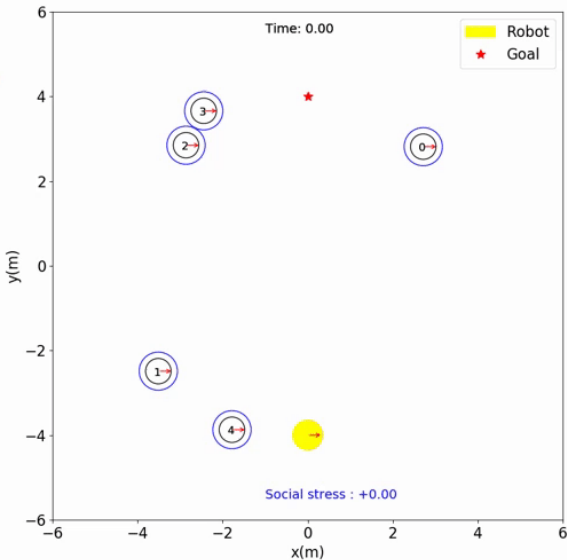
6.3 最新的移动机器人轨迹规划技术

Simple Mode

CASE #2

CADRL | LSTM-RL
SARL | ssDRL

Scene size	10m×10m
Number of pedestrian	5
Sensing range	None





南开大学
Nankai University



天津市智能机器人技术重点实验室
Tianjin Key Laboratory of Intelligent Robotics
南开大学机器人与信息自动化研究所
Institute of Robotics & Automatic Information System

第六章 移动机器人运动规划技术

Motion Planning Algorithms for Mobile Robot

孙雷 教授

Tel:13512967601

Email: sunl@nankai.edu.cn

2026年6月8日